

A LATEX FORMAT FOR DISSERTATIONS: THIS IS THE ETD VERSION OF THE TITLE PAGE

By

Full Legal Name

Dissertation

Submitted to the Faculty of the
Graduate School of Vanderbilt University
in partial fulfillment of the requirements
for the degree of

DOCTOR OF PHILOSOPHY

in

Major

Graduation Month Day, Year

Nashville, Tennessee

Approved:

Committee Chair, Ph.D.

Committee member, Ph.D.

Committee member, Ph.D.

Committee member, Ph.D.

Copyright © 2021 Full Legal Name
All Rights Reserved

The dedication page is optional. The Copyright page is also optional. If you do not use them, please delete them. If you have a dedication, then center it in the middle of this page. If no Copyright page, begin printing page numbers here, using lower case Roman numerals and continue consecutive Roman numeral numbering throughout the preliminary pages.

ACKNOWLEDGMENTS

The acknowledgements page is optional. Acknowledgement of grant and contract support may be included on this page. This is also where you thank the people that made your work possible: advisers, your committee, mom, friends, etc. For example, Vanderbilt University would like to thank Eli Hootener for creating the original $\LaTeX$ template for dissertations and Haley Adams for reorganizing and updating the document.

TABLE OF CONTENTS

	Page
LIST OF TABLES	viii
LIST OF FIGURES	ix
1 Introduction	1
1.1 Motivation	1
1.2 Research Contributions	2
1.3 Dissertation Organization	2
2 Towards Human-AI Teaming for Cybersecurity	4
2.1 Introduction	4
2.2 Related Work	5
2.3 Teaming Architecture & Methods	6
2.3.1 System Architecture	7
2.3.2 Explanation Generation	8
2.3.3 LLM Chatbot	10
2.3.4 Dashboard	10
2.4 Case Study	12
2.5 Conclusion	14
3 Charon: An End-to-End Infrastructure for Connecting AI@Edge to HPC	16
3.1 Introduction	16
3.2 Use Case	17
3.2.1 Background	17
3.2.2 Core Capabilities in Edge-to-HPC	19
3.3 Approach	19
3.3.1 Resource Arbitration for AI@Edge	20
3.3.2 AI@Edge Reconfiguration	22
3.4 Experimental Evaluation	24
3.4.1 Setup	24
3.4.2 AI Inference Management and Control	25
3.4.3 Adaptive Control Using Different AI Models	26
3.4.4 Discussion	27
3.4.4.0.1 Resource Optimization	27
3.4.4.0.2 Model Size and Quantization	28
3.4.4.0.3 Container Run Time and AI Inference-as-a-Service	29
3.4.5 Toward Broader Edge-to-HPC Applications	29
3.5 Related Work	30
3.5.1 Resource Autoscaling	30
3.5.2 Edge Computing Platform	31
3.6 Conclusion	31
4 A Reinforcement Learning Approach for Performance-Aware Reduction in Power Consumption of Data Center Compute Nodes	33

4.1	Introduction	33
4.2	Related Work	35
4.3	Background	36
4.3.1	Reinforcement Learning	37
4.3.2	Performance Measurement	37
4.3.3	Intel RAPL	37
4.4	Methodology	38
4.4.1	Problem Definition	38
4.4.2	Mathematical modeling and MDP identification	39
4.4.3	Reward function	40
4.4.4	RL Agent Training	41
4.5	Empirical Evaluation of Reinforcement Learning Approach	41
4.5.1	Evaluation Strategy	42
4.5.2	Implementation of RL Agent and Experimental Platform	42
4.5.3	Experiment Details	43
4.5.4	Empirical Results and Discussion	45
4.5.5	Analysis of Results	46
4.6	Conclusion	48
5	Offline Reinforcement-Learning-Based Power Control for Application-Agnostic Energy Efficiency	54
5.1	Introduction	54
5.2	Background	56
5.2.1	Offline Reinforcement Learning	57
5.2.2	Measuring Application Performance through Hardware Performance Counters	57
5.2.3	Capturing Application Progress in the Dataset	58
5.3	Problem Statement	58
5.4	Methodology	59
5.4.1	Dataset Collection: Capturing Application Performance through Hardware Performance Counters	60
5.4.2	Augmenting the Dataset with Application Progress	60
5.4.3	Training the RL Agent using Offline Collected Dataset	61
5.4.4	RL Model Evaluation	62
5.5	Applying the Methodology to a Use Case	62
5.5.1	Experimental Hardware and Tools Used	62
5.5.2	Compute-intensive Workloads used in Offline RL Model Training	63
5.5.3	Data Collection and Offline Training	64
5.6	Empirical Evaluations	65
5.6.1	Evaluation of the Proposed Framework	66
5.6.2	Comparison with Existing Methods	68
5.7	Background and Related Work	70
5.7.1	Background on Offline Reinforcement Learning	70
5.7.2	Comparison with Related Efforts	70
5.8	Conclusions	72
6	Decay-Driven Reinforcement-Learning-Based Power Control for Single Node Energy Efficiency	74
6.1	Introduction	74
6.2	Background	75
6.2.1	Offline Preference-Driven Multi-Objective Reinforcement Learning	75
6.2.1.1	Multi-Objective Value Functions and Preference Conditioning	75
6.2.1.2	Preference-Driven Bellman Optimality Operator	75
6.2.1.3	Offline Learning and the Preference Alignment Problem	76

6.2.1.4	From Preference Alignment to Decay-Driven Control	76
6.2.2	Application-Aware Performance Measurement	76
6.2.3	Measuring Application Performance through Hardware Performance Counters . . .	77
6.2.4	Proxy Applications and Dataset Partitioning	77
6.2.4.1	Calder Based Similarity Analysis	78
6.3	Problem Statement	78
6.4	Methodology	80
6.4.1	Data Collection	80
6.4.2	Offline Training	80
6.4.3	Evaluation	80
6.5	Experimental Setup	80
6.5.1	HPC Platform and Software Stack	80
6.5.2	Workloads	81
6.5.3	Offline Data Collection	81
6.5.4	Decay-Driven Preference	82
6.5.5	Policy Network Architecture	82
6.5.6	Offline Training	82
6.5.6.0.1	Bellman backup.	83
6.5.6.0.2	Training objective.	83
6.5.6.0.3	Optimiser.	83
6.5.7	Online Evaluation	83
6.6	Results	83
6.6.1	Preference-Conditioned Operating Points	83
6.6.2	The Decay Percentage as a User-Controllable Bound	83
6.6.3	Application-Specific Observations	84
6.6.3.0.1	STREAM benchmarks.	84
6.6.3.0.2	NPB benchmarks.	84
6.6.4	Generalisation to Unseen Workloads	84
6.7	Related Work	84
References		89

LIST OF TABLES

Table

Page

LIST OF FIGURES

Figure	Page
--------	------

CHAPTER 1

Introduction

This dissertation presents a collection of research contributions at the intersection of autonomous cyber defense, edge-to-HPC computing, and energy-efficient high-performance computing (HPC). Together, these works address the growing complexity of modern cyber-physical and computing systems, where intelligent automation must be paired with human oversight, and where energy efficiency must be achieved without sacrificing application performance.

1.1 Motivation

Modern computing infrastructure spans a wide spectrum—from edge devices collecting scientific or operational data, to high-performance computing clusters running exascale workloads, to enterprise networks defended by autonomous cyber agents. Each of these domains faces a common challenge: how to design intelligent systems that are simultaneously powerful, efficient, trustworthy, and human-interpretable.

Autonomous Cyber Defense and Human-AI Teaming. As enterprise and operational technology (OT) networks grow in scale and complexity, autonomous cyber defense agents (Blue agents) have emerged as a promising capability, offering continuous monitoring and rapid response. However, fully autonomous defense remains insufficient: agents can suffer from adversarial manipulation, false positives, and a lack of mission-level context. This dissertation addresses this gap by developing a comprehensive Human-AI Teaming framework for cybersecurity, where humans and AI agents collaborate synergistically, each compensating for the other’s limitations.

Edge-to-HPC Computing Infrastructure. Scientific workflows at large research facilities—such as X-ray beamline experiments at the Argonne Advanced Photon Source (APS)—increasingly require real-time data processing at the edge combined with high-fidelity model retraining on HPC systems. This dissertation presents Charon, a middleware framework designed to bridge this gap by enabling adaptive resource management across the edge-to-HPC computing continuum.

Energy Efficiency in HPC Nodes. As exascale computing becomes a reality, the energy demands of HPC data center nodes continue to grow. A common approach to reducing energy consumption is to limit the power supplied to hardware components when workloads experience bottlenecks elsewhere in the system. This dissertation presents a progression of reinforcement learning (RL) based approaches for performance-aware power management of HPC compute nodes—beginning with online RL, advancing to application-agnostic offline RL, and culminating in a preference-driven multi-objective framework that allows users to

specify their acceptable performance degradation as a simple percentage.

1.2 Research Contributions

The specific research contributions presented in this dissertation are:

1. **Human-AI Teaming for Cybersecurity (Chapter 2):** A comprehensive framework for Human-AI Teaming in cyber defense, integrating a Blue agent with explainable AI (XAI) methods including SHAP-based explanations, counterfactual reasoning, and an LLM-powered chatbot interface. The platform supports operator overrides, risk-aware action scoring, and trust calibration through a unified dashboard.
2. **Charon: Edge-to-HPC Infrastructure (Chapter 3):** An end-to-end infrastructure connecting AI@Edge workloads to HPC resources, enabling dynamic resource arbitration, real-time control of edge applications, and timely AI model retraining. The framework is demonstrated in the context of X-ray beamline experiments at the Argonne APS.
3. **Online RL for Performance-Aware Power Management (Chapter 4):** A reinforcement learning approach for dynamic power capping of HPC compute nodes using the Intel RAPL interface, trained with a Proximal Policy Optimization (PPO) agent on a mathematical model of compute node behavior and validated using the STREAM benchmark.
4. **Offline RL for Application-Agnostic Energy Efficiency (Chapter 5):** An offline RL-based power controller trained on pre-collected execution traces, capable of learning an application-agnostic energy efficiency policy without requiring online interaction with the target system. The approach is validated across multiple benchmarks spanning memory-bandwidth-bound and compute-bound workloads.
5. **Decay-Driven Multi-Objective RL for Energy Efficiency (Chapter 6):** The Decay-Driven Multi-Objective Reinforcement Learning (DDMORL) framework, a single offline-trained controller that allows users to specify their maximum tolerable performance degradation as a percentage. The framework eliminates the need for multiple agents or manual preference tuning and produces well-structured Pareto fronts in the energy-degradation trade-off space.

1.3 Dissertation Organization

The remainder of this dissertation is organized as follows. Chapter 2 presents the Human-AI Teaming platform for cybersecurity. Chapter 3 describes the Charon infrastructure for edge-to-HPC computing. Chapters 4, 5, and 6 present the progression of reinforcement learning approaches for HPC energy efficiency. Each

chapter is self-contained, presenting the motivation, related work, methodology, experimental evaluation, and conclusions of the corresponding research contribution.

CHAPTER 2

Towards Human-AI Teaming for Cybersecurity

Abstract

Advances in autonomous cyber defense have enabled intelligent “Blue agents” capable of monitoring large-scale networks, detecting anomalies, and rapidly deploying defensive actions. However, operational realities in enterprise, operational technology (OT), and green-field environments demonstrate that fully autonomous cyber defense remains insufficient due to contextual ambiguity, adversarial manipulation risks, and the need for strategic judgment. This paper presents a comprehensive framework for Human-AI Teaming for cyber defense, integrating a Blue agent with live network observations, risk analytics, and interactive human decision support. A unified dashboard provides multi-resolution cyber situational awareness, including node compromise states, attack traces, and dynamically scored defensive action options. Humans intervene when risk metrics, network health indicators, or mission priorities deem autonomous actions insufficient or unsafe. The platform incorporates risk-management methods as well as human-agent interaction datasets to support trust calibration and learning cycles. We demonstrate these Human-AI teaming capabilities of our platform through a case study, where we also show when and how a human can override agent’s actions. The paper illustrates that well-designed Human-AI coordination can reduce cognitive load on defenders, improve resilience to adversarial attacks, and balance autonomous speed with human’s contextual domain knowledge.

2.1 Introduction

Modern cyber defense operations face an unprecedented scale, velocity, and complexity of threats across enterprise networks, operational technology (OT) infrastructures, and emerging green-field digital environments. Despite mature monitoring tools and extensive visibility into network topology, device states, and workflow dependencies, the growing sophistication of cyber adversaries requires a defense strategy that is both proactive and adaptive. Autonomous cyber-defense agents (referred to here as Blue agents) have emerged as a promising capability, offering continuous monitoring, rapid mitigation, and the ability to detect subtle anomalies unnoticed by humans.

Yet, autonomous agents alone cannot ensure resilient defense. They can suffer from false positives and false negatives, remain vulnerable to adversarial machine-learning attacks, and often lack the overall mission context or understanding of broader operational constraints. Defensive actions taken without contextual awareness – such as isolating key OT assets or resetting mission-critical processes – may cause more harm than the attack itself. Meanwhile, human cyber operators – with rich contextual understanding, creativity, and

ethical judgment – are still constrained by cognitive limits, fatigue, and response speed. Thus, a systematic approach to Human-AI teaming can overcome these limitations faced by humans and AI agents alone.

The core idea of Human-AI teaming is to leverage complementary strengths. AI agents provide highly scalable and automated cybersecurity, 24/7 vigilance, and sophisticated anomaly detection. Humans provide the application context, multi-domain reasoning, and strategic decision-making. The teaming objective is not merely human oversight of AI, but a synergistic workflow where humans and agents continuously support, supervise, and enhance each other. Roles become dynamically adjustable: humans may be *in-the-loop* for high-risk events, *on-the-loop* for autonomous operations requiring oversight, or *out-of-the-loop* for routine low-impact anomalies.

To operationalize this paradigm, we build upon existing infrastructure developed in RAMPART Neema et al. (2024) including: live network observations, monitoring tools, simulation and emulation environments, asynchronous agent execution, risk-scoring methodologies, and model-based network provisioning systems. The proposed system is a unified dashboard offering deep insights through multiple explainable methods into node states, attack trajectories, and available defensive actions. Additionally, each agent action is annotated with the action-specific explanations (generated using shapley values Lundberg and Lee (2017) and counterfactual explanations Sokol and Flach (2019)) and an impact on overall cybersecurity risk profile of the network Neema et al. (2021).

This allows human operators to choose to interject when risk crosses critical thresholds, when mission priorities require human judgment, or when the agent’s recommended actions present unacceptable operational tradeoffs. System-level datasets of these interactions are logged for training next-generation agents, calibrating trust, and enabling transparent explainability workflows.

In this paper, we present a comprehensive Human-AI teaming platform for cybersecurity. Section II reviews the related work in Human-AI teaming and explainable AI. Section III presents the teaming architecture and methods developed in this research work including: the overall system architecture, approach to generate explanations for agent actions, design of an LLM-powered interaction tool, and an intuitive dashboard for human interaction with the agent. A case study is given in Section IV to illustrate the Human-AI teaming and when and human can override the agent’s action is also presented. Finally, Section V concludes and provides directions for future work. Overall, this research work provides novel implementation methods and approaches for designing a comprehensive Human-AI teaming system for cyber defense based on explainable methods.

2.2 Related Work

The integration of agentic AI-systems capable of autonomous reasoning, planning, and action represents a transformative shift in cybersecurity operations Lazer et al. (2026). However, the dual-use nature of these technologies necessitates structured engineering frameworks to ensure effective human-agent teaming. Evertsz and Thangarajah propose a BDI-based methodology that uses diagrammatic representations to present contextualized team cognition to humans, facilitating their role as peers rather than mere supervisors of artificial agents Evertsz and Thangarajah (2020). A primary barrier to successful teaming is the "black-box" nature of deep learning models, which often leads to alert fatigue and a lack of trust among security analysts. Recent research emphasizes Explainable AI (XAI) Kalimuthu (2025) as the critical bridge to resolving this opacity. For example, studies utilizing SHAP and LIME demonstrate that providing feature-level justifications for IDS alerts can significantly improve forensic analysis and decision-making Kalimuthu (2025). Furthermore, evidence from user studies indicates that hybrid UI strategies combining confidence visualizations with structured rationales are most effective at calibrating user trust and improving decision accuracy. Ultimately, reliability in autonomous cybersecurity is increasingly defined as a socio-technical property. Adebayo's HITL-XAI framework Adebayo (2026) embeds human expertise at strategic junctures of the cyber kill chain, using XAI-driven uncertainty quantification to trigger human intervention only when the AI reaches its limits. This collaborative paradigm is echoed in governmental principles for Operational Technology (OT), which stress that humans must remain responsible for functional safety through continuous oversight and fail-safe mechanisms. Together, these papers advocate for a shift from passive reporting to symbiotic collaboration, where AI amplifies human capacity while human experts provide the essential context for high-stakes reasoning.

2.3 Teaming Architecture & Methods

The concept of Human-AI Teaming for cybersecurity is motivated by limitations on both sides of the human-machine spectrum. Cyber agents excel in high-frequency detection, large-scale log processing, and consistent execution of defensive playbooks, making them invaluable during rapidly unfolding attacks such as lateral movement, credential escalation, or large-scale scanning. However, these agents lack contextual awareness, cannot interpret multi-domain implications (e.g., mission importance of an OT asset), and may recommend actions that inadvertently disrupt operations. In contrast, human defenders possess strong contextual understanding, creativity, intuition, and ethical judgment, especially important during complex or novel attacks. Yet they struggle with the speed and volume of modern cyber engagements and can become overloaded or inconsistent under stress.

For example, let's say a Blue agent detects a privilege escalation on a subnet supporting admin systems

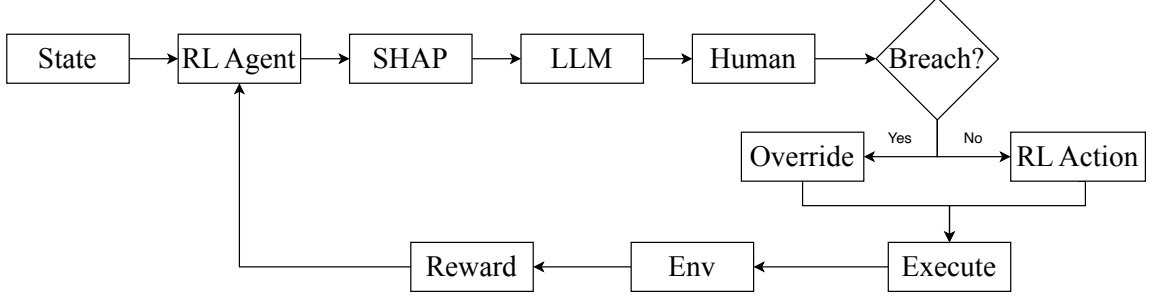


Figure 2.1: Human–AI cyber defense control loop with intervention condition.

and safety controllers. It decides to isolate the segment as there is a 92% mitigation chance and only a 35% risk of alerting the attacker. However, what the agent doesn't consider is that full isolation would raise operational risk during a maintenance cycle. So, with this contextual knowledge the human narrows the response to a subset of hosts and applies rapid policy updates, preserving mission continuity while mitigating the threat. The agent executes, monitors, and logs the action, showing that collaboration, not full autonomy, is essential.

To allow for this collaboration, we develop a system where human operators can override AI teammates and inject alternative actions whenever they disagree on attack or defense trajectories. To support issue detection and diagnosis, we provide two forms of explanations-direct and counterfactual-for agent actions, accessible through an LLM-based chatbot for ease of use. Finally, we integrate these explanations into a unified dashboard, where each action is annotated with its probability of success, expected execution time, likelihood of attacker detection, and impact on overall risk profile of the network Neema et al. (2021). A control loop in our developed system is shown in Figure 2.1.

2.3.1 System Architecture

The system architecture is based on the CAGE-2 enterprise topology Kiely et al. (2023) and utilizes RAM-PART, a simulator to train and evaluate autonomous cyber defense agents in realistic enterprise and OT-integrated network environments. It is deployed on an OpenStack cloud environment and consists of 15 virtual machine instances: one high-capacity Defender node and fourteen additional hosts distributed across user, enterprise, and operational subnets. The enterprise subnet contains mission-critical servers (enterprise0, enterprise1, enterprise2), the user subnet contains five user workstations (user0 – user4) representing typical entry points for adversarial compromise, and the operational subnet hosts OT-integrated nodes (op.host0 - op.host2 and op_server0) running power grid and chemical plant control workloads. Gateway nodes interconnect these subnets, enforcing routed communication paths and enabling realistic lateral movement across network boundaries. A separate control network provides management-plane connectivity for orchestration,

monitoring, and persistent Velociraptor communication between all endpoints and the Defender. The Defender node, provisioned with elevated compute resources, hosts the Velociraptor server and serves as the execution environment for the blue team actions, telemetry aggregation, logging, and reward evaluation. The entire topology is instantiated using OpenStack services including Nova for compute provisioning, Neutron for virtual networking and routing, and HEAT templates for declarative orchestration, enabling reproducible deployment of vulnerable services and controlled network communications. This emulated infrastructure preserves realistic TCP/IP routing, service discovery, exploit execution, and defensive response mechanisms, thereby providing a high-fidelity hybrid IT-OT environment for evaluating autonomous and human-in-the-loop cyber defense within RAMPART.

2.3.2 Explanation Generation

Users may request one of two types of explanations for red or blue agent actions: direct explanations (“Why does [agent] choose [action]?”) or counterfactual explanations (“Why does [agent] NOT choose [action] instead of [alternate action]?”). To generate these explanations, we first apply VIPER Bastani et al. (2018a) to distill the agent’s policy into a simple, representative decision tree. We then use this tree to produce either a SHAP-based direct explanation Lundberg and Lee (2017) or a counterfactual explanation Sokol and Flach (2019), depending on the user’s query.

RL policies are often too large and complex for human users to understand. Extracting RL policy as a decision tree can make it simple, intuitive, and easier to interpret Bastani et al. (2018b). The initial tree may be empty, supplied by the user, or carried over from a previous exercise. When the agent executes an action, we generate an abstract state that captures the context in which the action occurred. The features of this abstract state are binary high-level, human-readable concepts (supplied by the user or the environment) that the agent can access when selecting actions. For example, we may track concepts such as activity, scan, and exploit detection on nodes in the network Emerson et al. (2024). We determine true features in the abstract state by pattern matching against the agent’s low-level observation.

We then add the abstract {state, action} pair to the training data for the decision tree and refit the tree using Gini impurity as the split criterion Pedregosa et al. (2011). Each internal node tests whether an abstract feature is true or false and each leaf corresponds to a possible agent action. Each action at a leaf is weighted by its importance, defined as the difference between its Q-value and the minimum Q-value among all available actions. At the end of the exercise (e.g., an episode or game), we evaluate the resulting decision tree by using its representative policy (rather than the base policy) to simulate multiple attacks or defenses on the network and averaging its environmental reward. If this new tree outperforms a tree trained on all prior data excluding the current episode, the new tree is retained as the representative policy. Otherwise, the

previous tree remains as the base. Repeating this update procedure after each exercise maintains an up-to-date, interpretable approximation of the underlying policy for explanation.

When the agent selects an action and the user requests a direct explanation of that selection, we apply SHAP Lundberg and Lee (2017) to the decision tree and return all abstract features with non-zero Shapley values. Shapley values originate from cooperative game theory and quantify the contribution of each feature to a model’s prediction by averaging its marginal impact across all possible feature subsets. SHAP estimates each feature’s average marginal contribution on a decision by comparing the model’s prediction for the given action to a baseline prediction (the average over all actions). If removing a feature substantially changes the prediction, that feature receives a large-magnitude SHAP value. SHAP values are positive when a feature supports the chosen action and negative when it opposes it. Users can then inspect the features that most strongly support or oppose the action, ordered by their absolute impact.

Example 1: Consider the user query: “Why does the blue agent remove access from ‘user3’ when there was a scan detected on ‘enterprise1’?” The user may be puzzled about why ‘user3’ is targeted instead of ‘enterprise1’. Applying SHAP to this decision yields the following normalized feature influences: scan of ‘enterprise1’ detected (0.655), no direct activity detected on ‘enterprise0’ (0.273), ‘enterprise1’ (0.038), ‘defender’ (-0.004), ‘user1’ (-0.007), ‘user2’ (-0.011), and a previous remove-action on ‘user3’ (-0.014). These values indicate that the decision to remove access from ‘user3’ is primarily supported by the fact that a scan has occurred on ‘enterprise1’ but no follow-on activity has yet been observed on ‘enterprise1’ or ‘enterprise0’. This suggests that the red agent may already have a foothold on ‘user3’ (or possibly ‘user4’). The action is weakly opposed by the absence of activity elsewhere in the network and by the fact that the blue agent has already tried removing access from ‘user3’ once before, but not strongly enough to prevent, repeating this defensive action.

If the user requests a counterfactual explanation Sokol and Flach (2019) for a given action, we generate a set of alternative actions along with the agent’s rationale for not choosing them. We first enumerate all other actions with non-zero Q-values that the policy could have selected and order them by Q-value. For each alternative action, we use the decision tree to compute the minimal change in the abstract state required for the agent to select that action instead of the original one. Concretely, we identify the closest common ancestor of the two actions in the tree and report the sequence of decision nodes that must change to reach the alternative leaf. Users can then inspect the required feature changes and determine whether the agent should have taken an alternative action, either because its learned policy is flawed or because it lacked accurate state information when making the decision.

Example 2: Consider the user query, “Why does the blue agent remove access from ‘user3’ when a scan is detected on ‘enterprise1’?” The user may believe that removing access for ‘user3’ is not the appropriate

action and wants to understand possible alternative actions and when the agent would choose them. In this scenario, the agent’s policy also allows the actions of restoring ‘enterprise1’ and removing ‘enterprise1’. The counterfactual explanation clarifies that the agent would have restored ‘enterprise1’ if there had been a prior exploit on ‘enterprise0’. The agent would have removed ‘enterprise1’ if there were activity on ‘enterprise0’ or ‘user3’, no activity on ‘user2’, no prior exploits on ‘enterprise0’, and no scans detected on the operational server. These explanations highlight that if the user believes, unlike the blue agent, that they know the location of the red agent, then a different action might be preferred.

2.3.3 LLM Chatbot

Presenting users with raw numerical outputs, however, could result in significant misunderstanding. To address this, we developed an LLM-based chatbot interface that supports natural-language question answering. When a user wants to know about a particular agent action, we construct a query prompt by collecting the current abstract state and the action taken by the agent. For counterfactual questions, we additionally extract the possible alternative actions from the agent policy. We then instantiate one of two templates: “Why does the agent choose [action] when in [state]?” (direct explanation) or “Why does the agent choose [action] instead of choosing [alternate action] when in [state]?” (counterfactual explanation). The LLM receives this prompt along with network information (e.g., machine connectivity) and the decision tree that encodes the alternative policy. It returns an explanation that describes why the action was taken or why an alternative was not taken in human-understandable natural language. The LLM does not generate explanations independently; instead, it translates structured outputs derived from SHAP and decision-tree policies into natural language. In future work, for safety-critical deployment outside of the simulation environment, we plan to validate LLM outputs by ensuring consistency with the original feature set and prevent the introduction of unsupported information.

Example 3: Consider the query: “Why does the blue agent remove access from ‘user3’ when there was a scan detected on ‘enterprise1’?” The generated prompt is: “Here is a decision tree showing the actions of a blue agent trying to stop a red agent on a network: [decision tree]. Here is information about the network: [network information]. Explain the following: “Why does the agent remove ‘user3’ when ‘enterprise1’ is scanned and all other features are false?” The LLM responds: “This could be for several reasons: (1). ‘User3’ might be communicating with ‘Enterprise1’, which could potentially be compromised by the red agent. By removing ‘User3’, the blue agent aims to cut off any potential communication between ‘User3’ and ‘Enterprise1’. (2). ‘User3’ might be a vulnerable node in the network that could serve as an easy entry point for the red agent to move deeper into the network. By removing ‘User3’, the blue agent prevents this possibility. In summary, the blue agent removes ‘User3’ when there is no activity detected at ‘Enterprise0’, a scan is being performed on ‘Enterprise1’, and ‘Enterprise1’ shows some ongoing activity

Network Visualizer

exp emu	episodes 1	steps 100	blueagent cardiff	redagent blinaagent	workflow chemical_plant	gamemode evaluation
user raja3	password *****	url http://10.101.2.95:5000	scenario cage2			
project kw-castle2	key castle-control	Apply Config Start Stop Restart Pause				running

Config applied. Worker running.

Figure 2.2: Dashboard to parse the experiment attributes.

(indicating potential compromise or malicious action) as a precautionary measure to secure the network.”

2.3.4 Dashboard

The dashboard integrates the backend processes mentioned above using FastAPI endpoints. It is important to interact with the run-time processes in a user/operator friendly manner. The dashboard consists of an argument parser console, chatbot, a network visualizer and a status bar that aids the decision making by the operator or users.

The LLM logs (shown in Figure 2.7) is integrated into a browser-based operator dashboard served by a FastAPI application and accessed via a standard web interface. To enable meaningful human-in-the-loop interaction without interrupting simulation continuity, the dashboard implements a *pause-and-explain* mechanism controlled by a dedicated pause toggle. When the operator activates the pause, the simulation halts at the conclusion of the current iteration and the system enters a two-phase explanation flow. In the first phase, the dashboard immediately renders the current network state and SHAP-derived feature attributions, while displaying a visual loading indicator in place of the action selection panel; this communicates that the system is actively generating an explanation and prevents premature interaction. Concurrently, the backend asynchronously queries the LLM with a prompt constructed from the current iteration context, including the blue and red agent actions extracted from the simulation state. In the second phase, once the LLM returns its response, the explanation text is pushed to the UI via a thread-safe snapshot bridge, the action selection panel becomes active, and the *Resume* button is enabled. This sequencing ensures that operators always read the explanation before they commit to an action or continue the simulation, enforcing the intended Human-AI collaboration loop. The synchronization between the simulation worker thread and the operator dashboard is managed through a pair of threading primitives. A `pause_event` signals when the simulation has reached a pause point, while an `action_selected_event` blocks the worker until the operator has either selected an override action or clicked *Resume*. A boolean flag `llm_ready` gates the availability of the *Resume* button in the UI, preventing the operator from advancing the simulation before the explanation has been delivered. The dashboard polls the backend at *300 ms* intervals for snapshot updates (network graph, SHAP

values, available actions) and at 1 s intervals for pause and readiness status, providing near-real-time feedback without overloading the server. Once an explanation has been displayed, it persists in the UI panel for the duration of the pause regardless of subsequent snapshot updates, so the operator can continue reading while reviewing the network visualization.

The operator action selection panel renders the set of candidate blue agent actions as clickable buttons. Selecting an action immediately records the operator’s choice and signals the worker to resume with that action, bypassing the agent’s default policy for that iteration. If no action is selected within a configurable timeout window, the simulation resumes automatically with the agent’s original recommendation. This design preserves operational tempo while still allowing operators to intervene when the LLM explanation or SHAP attribution highlights a concern.

Together, the SHAP attributions, the decision tree policy extraction, and the LLM chatbot form a layered explanation system. SHAP values identify *which features* drove the agent’s decision; the decision tree renders the *policy logic* in human-readable form; and the LLM chatbot translates both into *natural language narrative* tailored to the operator’s query. This layered approach reflects best practices in human-centered XAI, providing multiple complementary explanation modalities Ye et al. (2023) while keeping the operator in meaningful control of the simulation’s trajectory.

2.4 Case Study

To show human intervention impact, we compare rewards from a fully agent-driven trajectory with one where a human overrides action using additional contextual knowledge. This knowledge may include domain information unavailable to the RL agent, such as suspicious network traffic, historical attack patterns, or operational priorities known to human defenders.

This study is intended as a proof-of-concept demonstration of Human-AI teaming framework rather than a statistically rigorous evaluation. In our scenario, the blue agent plays against a red agent using its learned policy, and we record the reward obtained when the blue agent acts autonomously. The blue agent selects actions according to its reinforcement learning (RL) policy, while the red agent executes adversarial actions within the same environment. Figure 2.3 shows a representative environment state during the attack scenario.

Eventually, the red agent performs the action `PrivilegeEscalate_Enterprise2` while the blue agent selects `Remove_Op_Server0`. As shown in Figure 2.4, this decision results in a negative step reward of -0.1000 , indicating that the autonomous policy did not adequately mitigate the ongoing threat.

The dashboard highlights that the decision tree (DT) model disagrees with the RL agent’s action and recommends an alternative such as `Ent0_DecoyTomcat`, as illustrated in Figure 2.5. The SHAP-based explanation panel provides feature-level insight into why the RL policy chose (with a higher probability)

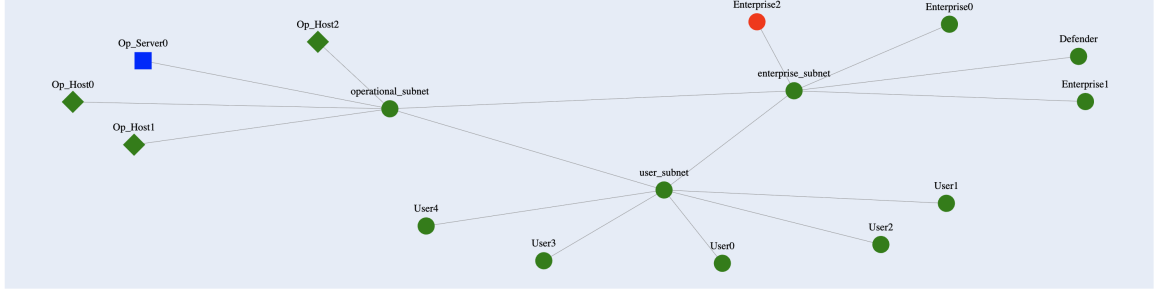


Figure 2.3: Environment state during the attack scenario, illustrating the network topology and the red agent’s activity on Enterprise2.

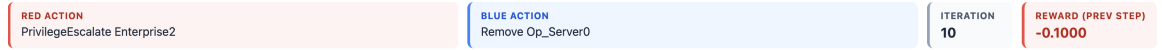


Figure 2.4: Dashboard view showing the red and blue actions at Iteration 10 and the resulting negative reward.

the action `Remove Op_Server0`, emphasizing inactivity-related features and prior user activity patterns.

However, the SHAP ranked features do not directly address the immediate adversarial activity occurring on ‘Enterprise2’. The resulting negative reward demonstrates a mismatch between the learned policy’s prioritization and the evolving attack context.

Next, we have a human operator observe a replay of this game, forcing the blue agent to take the same actions as before until the first human intervention. At each decision point, the operator is informed that an alternative action may be selected whenever the operator disagrees with the agent’s chosen action. Figure 2.6 illustrates the interface that allows the operator to override the agent’s action.

When the red agent escalates privileges on ‘Enterprise2’, the human operator may determine, based on contextual knowledge, that the blue action should address the compromised enterprise node rather than remove an operational server. Whenever an override occurs, the game proceeds as if the agent selected the human’s alternative action, and the agent resumes selecting actions autonomously until the next intervention. This process yields a second trajectory and a reward that reflects agent performance with human intervention.

Finally, Figure 2.5 conceptually illustrates the expected rewards that the human-assisted trajectory could have returned.

By comparing these two rewards, we quantify the effect of human intervention on the agent’s performance. In the illustrated scenario, the autonomous agent’s action leads to a negative reward, whereas a human-informed alternative guided by awareness of the active attack on ‘Enterprise2’ better aligns with the immediate defensive objective.

The impact of human intervention is maximized when the agent’s policy is suboptimal due to factors such as limited training coverage, noisy or incomplete observations, distributional shift, or modeling errors.



Figure 2.5: Decision tree representing the RL policy.



Figure 2.6: Human override interface enabling intervention when the operator disagrees with the agent's selected action.

In such cases, the Human-AI teaming framework enables strategic overrides that incorporate external domain expertise, improving overall performance as reflected by higher cumulative rewards along the intervention-enabled trajectory.

As part of our ongoing work, we are extending this framework to incorporate human actions into the training and evaluation loop to improve network resilience. We collect human intervention data during interactive cyber-defense exercises and use overrides and alternate strategies to enhance agent learning and assess their impact on system performance. By comparing agent trajectories with human interventions using metrics such as cumulative reward, recovery time, and resilience to adversarial behavior, our aim is to quantify the benefits of Human-AI teaming across scenarios and user groups.

2.5 Conclusion

We presented novel methods and approaches to design a comprehensive Human-AI teaming system for cyber, leveraging unique capabilities and mitigating limitations. The system developed is grounded in quantitatively generated explanations that build trust between humans and agents that can help better align the agent's use in real-world, critical cybersecurity applications. Our ongoing research work includes designing games and pre-training tasks that can help build trust between humans and AI, and developing highly interpretable systems that make it easier for human operators to interact with. In addition, we are working to extend the teaming environment accordingly for deeper situational awareness about the presence and behavior of AI agents as well as Human-AI collaboration through agentic AI configurations for coordination and control. Future work will include controlled user studies and statistically significant evaluations across diverse attack scenarios.

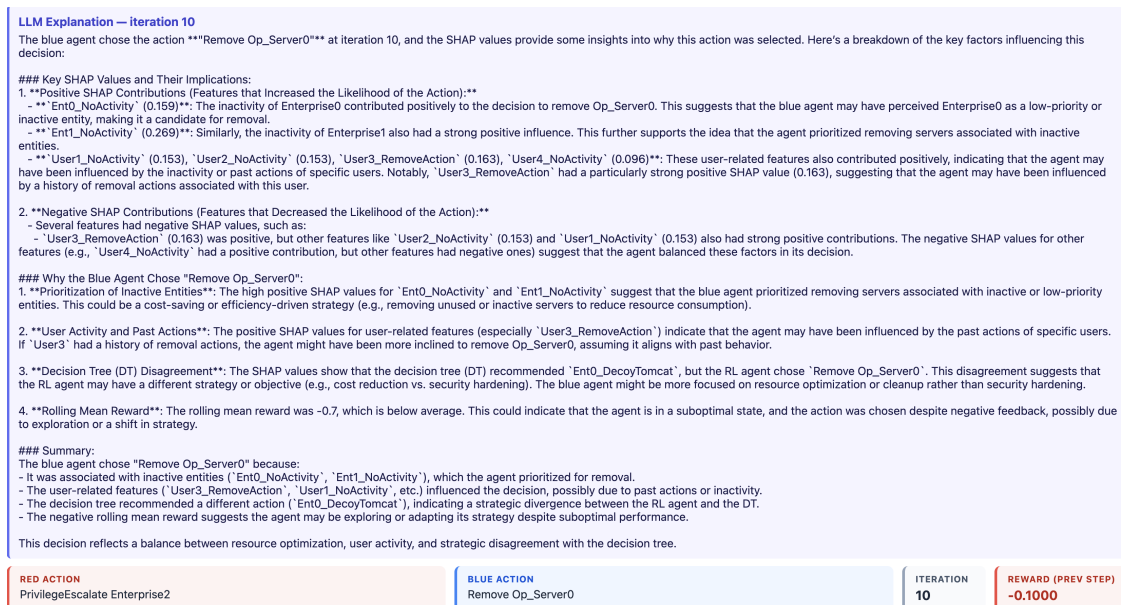


Figure 2.7: LLM logs of human agent to determine the right action.

Acknowledgment

This research was funded, in part, by the Defense Advanced Research Projects Agency (DARPA) through the contract #HR001124C0425. The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the U.S. Government. AI-assisted tools (e.g., language models) were used to support editing and clarity of writing. All technical content and validation were performed by the authors. The authors would also like to thank Kayla Boggess (Leidos) for her insights and contributions.

CHAPTER 3

Charon: An End-to-End Infrastructure for Connecting AI@Edge to HPC

Abstract

Connecting Internet-of-things devices and edge computing to high-performance computing (HPC) systems introduces new challenges to capture the transformative potential of the computing continuum. User workflows in this edge-to-HPC continuum might require dynamic resource provisioning and control at the edge to take advantage of artificial intelligence (AI) inference at the data source (AI@Edge), real-time experiment steering, and model retraining through HPC systems. In this chapter we discuss the case of an edge-to-HPC workflow from beamline science and illustrate how such a workflow can benefit from resource arbitration and reconfiguration. From this driving example we highlight our work toward the design of a framework for AI@Edge resource management and demonstrate the benefits of our approach on an initial experiment, showcasing our ability to observe and adapt resources to meet a target application throughput.

3.1 Introduction

Edge and high-performance computing (HPC) are mutually beneficial components in large-scale scientific experiments. Data volume and computing requirements continue to grow, creating pressure on the computing and network capability of HPC systems. Edge computing can be introduced to process data at the source, lowering the need for sending data to an HPC production system and improving performance. As such, the ability to connect edge resources to an HPC system in a common scientific workflow enables large simulations running on HPC to take filtered data streamed from the edge, reducing latency and infrastructure cost, while still benefiting from the large computing power of HPC for science discovery. The intermediate artifacts of edge computing also provide immediate feedback to users, allowing them to control data processing or even the experiment itself as it occurs. Similarly, algorithms and artificial intelligence (AI) models running on edge computing can be retrained with new data from the experiment on connected HPC resources, adapting the workflow as new scientific needs arise.

The collaboration between edge and HPC, named edge-to-HPC, forms an integrated infrastructure and brings new challenges, ranging from HPC workflow management Dube et al. (2021); Badia et al. (2024) to real-time control of sensors Kandel et al. (2023). These challenges need attention for designing a holistic control of resources in the edge-to-HPC continuum. Application runs and data processing in edge computing are manually executed and curated by users, and their resource requirements are manually controlled and are not propagated outside the edge. We claim that the challenges in edge computing remain as yet unexplored

for finding the potentials and limitations toward a fully connected continuum.

To support the transformative edge-to-HPC computing continuum, we describe here a project that investigates (1) how resources on edge devices can be arbitrated dynamically to ensure the performance for experiment-critical applications while being flexible for reconfiguration, (2) how system software can help design AI@Edge applications that leverage HPC platforms to tune machine learning (ML) model performance, and (3) how flexible the user-agnostic system software policies can be, in order to be deployed across a wide range of use cases and hardware setups.

In this paper we focus on the management of edge computing resources under a user-provided performance goal. The contributions of this paper are as follows. First, we identify an existing scientific workflow from a group of collaborating scientists that requires resource management and arbitration at the edge for science-driven control. Second, we design a framework that captures the performance of both system and user applications while the workflow is running and enables a control algorithm to maximize application throughput. Third, we demonstrate how such a control algorithm can be designed and how such capability for adaptive edge-computing performance can fit in the continuum, identifying challenges for future resource management on the scientific edge. The framework is open-sourced in Github¹.

The following sections describe the scientific workflow that explains our motivation and opportunities for edge-to-HPC, the proposed framework for edge computing, experiment results that highlight the mechanism of the control, and a summary of our approach and future direction.

3.2 Use Case

We have been working closely with scientists at the Argonne Advanced Photon Source (APS) to understand their current applications and workflows that would need an integrated research infrastructure including edge and high-performance computing to fully utilize next-generation X-ray beamline facilities. After a series of discussions with the scientists, we identified some user applications as future workloads and recognized their requirements for running them across the continuum (see Fig. 3.1.) In this section we use a specific use case of X-ray imaging to explain the need for a computing continuum.

3.2.1 Background

The Argonne APS user facility provides ultrabright, high-energy X-ray beams to scientists to serve scientific experiments ranging from materials science to biology, chemistry, planetary science, and fundamental physics. The facility is now finalizing its recent upgrade Advanced Photon Source (2021) and raises the need for data transportation and storage for their next-generation X-ray beams and state-of-the-art detectors.

¹<https://github.com/perarnau/charon>

Advancing from manual data aggregation and processing, the APS facility is designed to stream data within a centralized computing architecture and control system, linked with HPC systems. Terabytes of data per experiment are automatically sent to HPC data storage and are consumed by scientific simulations running on HPC. With the APS upgrade, scientists expect 10–1000x data volume increase toward petabytes per experiment. Such an increase in data volume demands the advancement of data aggregation and processing capabilities within the edge-to-HPC computing continuum. The introduction of computation within and at the edge in combination with HPC brings *new opportunities and challenges* in expanding the computing continuum and AI-driven control in the new APS operation.

In addition to the high-bandwidth computing and networking systems in the infrastructure, ML-based AI methods at the edge (built, trained, and validated on HPC) will enable scientists to analyze data directly at the edge. Recent AI-based applications Babu et al. (2022); Cherukara et al. (2020); Liu et al. (2022) and dynamic workflows Kandel et al. (2023) have been proposed and developed by APS scientists. These AI techniques are now commonly employed by experimental scientists, running on hardware-accelerated machines for data analysis, often surpassing the performance of traditional numerical computing methods, particularly for feature detection and image reconstruction, such as finding and tracking Bragg peaks and ptychography.

Within the continuum, AI@Edge models can be dynamically reconfigured and play active roles in driving the dynamic adaptation of the scientific instruments to react to changes in experimental conditions. We believe that this new computing paradigm will continue to evolve and will require an integrated end-to-end system to manage resources across scientific workflows. In order to support these new functionalities, fine-grained resource management will need to administer shared computational and storage resources at the edge, while the connections to HPC resources and workflow management will need further improvements. To co-design such an integrated system, we discuss here a recently started project, named Charon, that explores the design space of system software for connecting and adapting resource-constrained AI@Edge workloads to HPC, through fine-grained resource management.

3.2.2 Core Capabilities in Edge-to-HPC

Timely AI Model Retraining on HPC An example of AI-based data processing is image reconstruction from raw X-ray diffraction patterns using small but powerful convolutional neural networks Babu et al. (2022). Fine-tuning and retraining of the neural network per experiment with various input sizes on an HPC system can adapt the model to new measurements during a beamline experiment Babu et al. (2022). A newly obtained dataset from the edge must arrive at the HPC resource, and model retraining must be scheduled in a timely manner, in order to update the model. Anomalies in the data stream can also be identified and extracted by

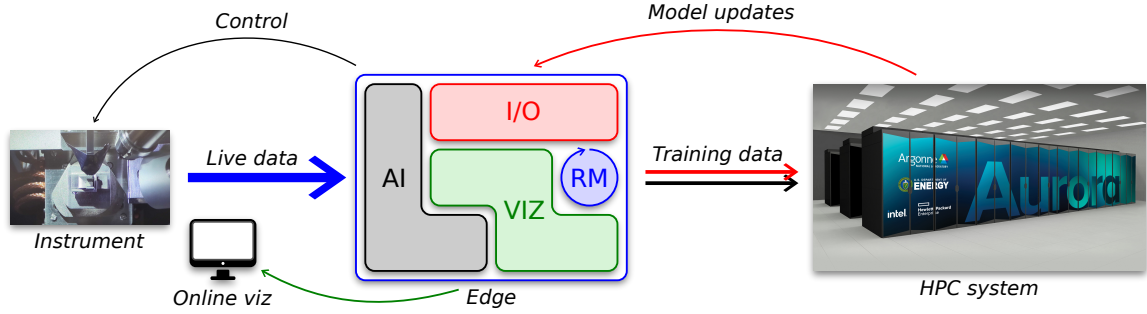


Figure 3.1: Our main use case: an AI@Edge model controls the scientific instrument and processes live data while being periodically updated based on fine-tuning and retraining taking place on HPC resources.

data processing at the edge and selectively added to the dataset, since they might be new patterns that the AI model needs to learn in HPC.

Real-Time Sensor Reconfiguration and Control X-ray diffraction patterns are collected in different positions and angles by moving the sample mount during the experiment for X-ray ptychography and crystallography. Scientists run programs to operate actuators and change the configuration of sensors and the experiment environment (e.g., pressure, temperature). Such actuation and reconfiguration are simple tasks but should be applied immediately and not be interrupted and delayed. Recently, Argonne scientists proposed an AI-enabled workflow to control a detector probe for efficient probe scanning Kandel et al. (2023) in real time using AI@Edge. This AI-driven decision-making demands high-throughput computation for the model to keep up with the sampling rate of the detector. Scaling AI inference and aggregation of the results require precise resource allocation to support real-time control and prevent bottlenecks.

Data Reduction on the Fly APS scientists estimate that the data volume in the post-upgrade era will increase by orders of magnitude toward petabytes per experiment, making filtering out uninteresting data highly desirable. Performing computations on the live data at the edge not only can reduce the final data volume but also can identify interesting phenomena occurring during the experiment. Filtering conditions vary per experiment: under known conditions, we may further reduce the total data volume, whereas exploratory experiments may need the full stream of data for new discovery. In the end, scientists will receive meta information from the edge computation to efficiently index and search through the reduced data. In addition to the capabilities identified above, we expect other workloads, such as live visualization, local AI fine-tuning at the edge, and caching of raw data to persistent storage, to introduce additional resource constraints to the end-to-end infrastructure.

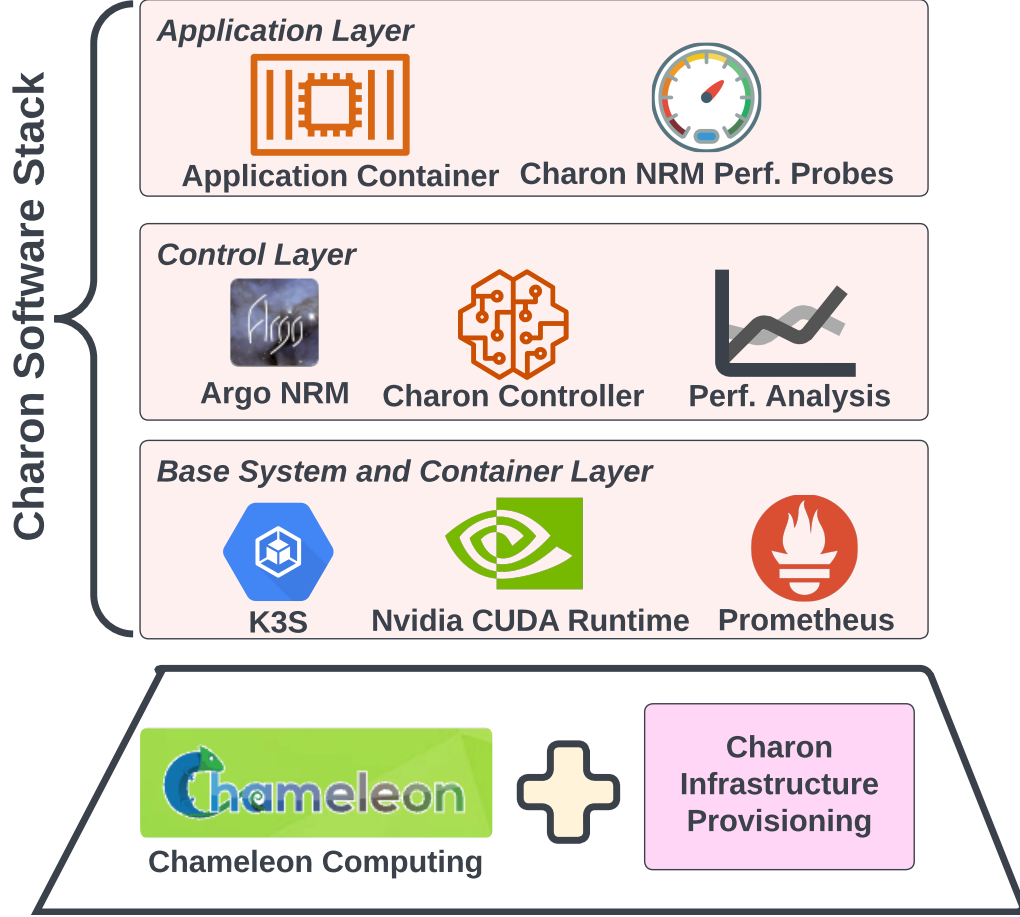


Figure 3.2: Charon software stack. The Charon controller is the main entity governing the resource while monitoring the performance of the system and applications.

3.3 Approach

We design a computing framework targeting scientific edge computing, as shown in Fig. 3.2, to facilitate resource monitoring and control and support the core capabilities described in Section 3.2. We leverage existing tools for the basis of our framework, including Kubernetes for container orchestration, Prometheus for resource monitoring, and a vendor runtime for inference execution. On top of that, we deploy NRM Perarnau et al. (2017), an infrastructure for monitoring and control of arbitrary resources. This infrastructure enables us to deploy a set of custom performance-monitoring capabilities through application instrumentation, as well as custom controls (container deployment) and a custom *control policy* to adapt the system dynamically while observing how applications progress toward a science goal. The following subsections describe the resource management and control delivered through this framework.

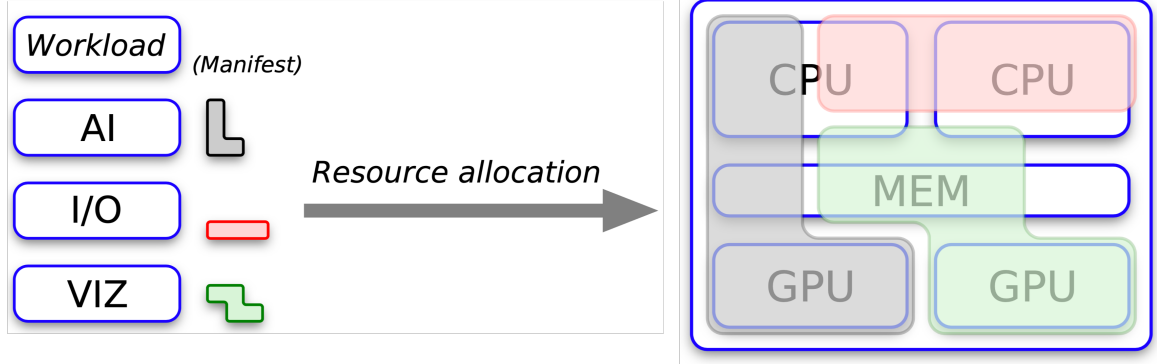


Figure 3.3: System services allocate resources to an AI workload on edge devices. Each component of the workflow describes an initial resource request and performance metrics.

3.3.1 Resource Arbitration for AI@Edge

Our system software runs as a privileged service on each node, monitoring and controlling resources. Each running agent optimizes the system performance through low-level actions in a semi-autonomous closed loop. Each agent has two main duties: 1) translating a set of resource requirements for each AI@Edge workload into a resource arbitration policy for a node and 2) providing methods to dynamically adapt this policy to reach a semi-stable resource allocation during the execution of an experiment.

Resource Allocation and Monitoring Following the design of the Waggle edge scheduler Kim et al. (2022), user workloads deployed on edge nodes are expressed as a set of resources that need to be allocated, as shown in Fig. 3.3. This is not a job scheduler with backfilling policies but, instead, a controller dynamically satisfying the constraints of each component of a workflow. Along with the edge scheduler, we deploy NRM, a system service that centralizes monitoring and control of node-level resources to collect information including self-reported application progress signals and power monitoring through OS-level interfaces. The NRM deployment is modular: since the infrastructure allows for arbitrary sources of information and controls, it can act as a central source of information to push into Prometheus, including application instrumentation to monitor inference performance (image per second), data pipeline performance (data generation rate, and pipeline buffers utilization), as well as CPU and GPU occupation.

By combining the two services (Kubernetes and NRM), we deploy an infrastructure that can launch applications and rebalance their resource needs at run time, both within and across edge nodes. Since Waggle nodes run Linux and we have privileged access to the nodes, most of our control knobs are OS mechanisms for resource allocation. We directly extend the NRM to control applications the way the Waggle scheduler is launching them: as Kubernetes Pods, with the full gamut of resource arbitration mechanisms available under this interface.

Adaptive Optimization Our edge agent optimizes the use of computing resources at run time for performance and stability. Given a set of performance metrics for application components, the agent monitors performance and takes actions to improve computing performance following policies set by users as shown in Fig. 3.4, mostly by rebalancing of resources or the activation/deactivation of certain hardware features. Similarly, prioritizing a specific workload or fair distribution of computing resources across users, throughput, or reliability optimizes computing performance in a multitenant configuration.

Many design choices impact the performance, stability, and flexibility of resource management policies. First, the nature of the inputs has an impact: some metrics are discrete or have fixed domains (min, max, or both), while some others are noisy in value or time (arrival time), available at fixed rates (sampling) or event-based (tracing). A choice can also be made regarding computing statistics across time windows or deriving metrics from multiple signals, while a specific signal can trigger a new decision, or decisions can be made at regular time intervals. Control knobs in our service exhibit similar characteristics.

Second, the policy itself can take various forms, depending on the number of inputs (signals) and outputs (controls) supported, from instantaneous models that directly map signals to control values regardless of the time dimension, to control theory designs such as PID controllers that can smooth out the control over longer periods Cerf et al. (2021a); Hawila et al. (2022a), or reinforcement learning policies Raj et al. (2023a) that can take into account the cost of the entire sequence of actions in their process.

Third, some control loop designs can be more amenable to swapping inputs or outputs, while others embed assumptions about the nature of the system that need to be revisited when applied elsewhere. Some policy designs can offer strong guarantees of the stability of the control when the environment changes, while others require preventing inputs or controls from going out of bounds.

We do not aim for a single policy design that could adapt to any experiment or scientific workflow objective. Instead, our infrastructure is designed to allow users to specify which system observations and controls should be relevant to their use case and the policy itself.

3.3.2 AI@Edge Reconfiguration

Our use case includes AI models deployed at the edge and involves steering instruments. It also includes workflows where the model steering the experiment can be updated at run time based on additional tuning and training done on HPC systems. In these cases, during the lifetime of an experiment, a workflow system will be in charge of sending collected data to HPC resources, retraining the models, and shipping the updated models back to the edge Babu et al. (2022). We envision this pipeline as a set of system services running on edge nodes and in an HPC compute environment and bridging the two environments together. Two main challenges that need to be addressed for such a design to succeed are 1) creating a data and model management

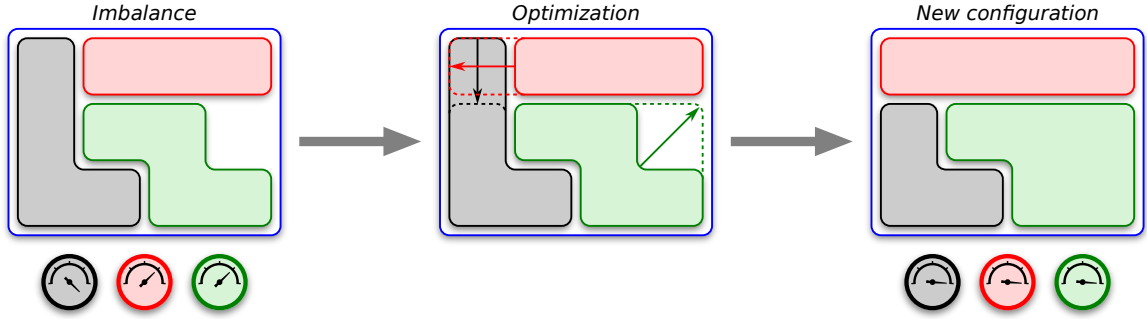


Figure 3.4: Based on the observed performance of the system and available controls (left), our system services periodically optimize the resource allocation (center), resulting in an improved configuration (right).

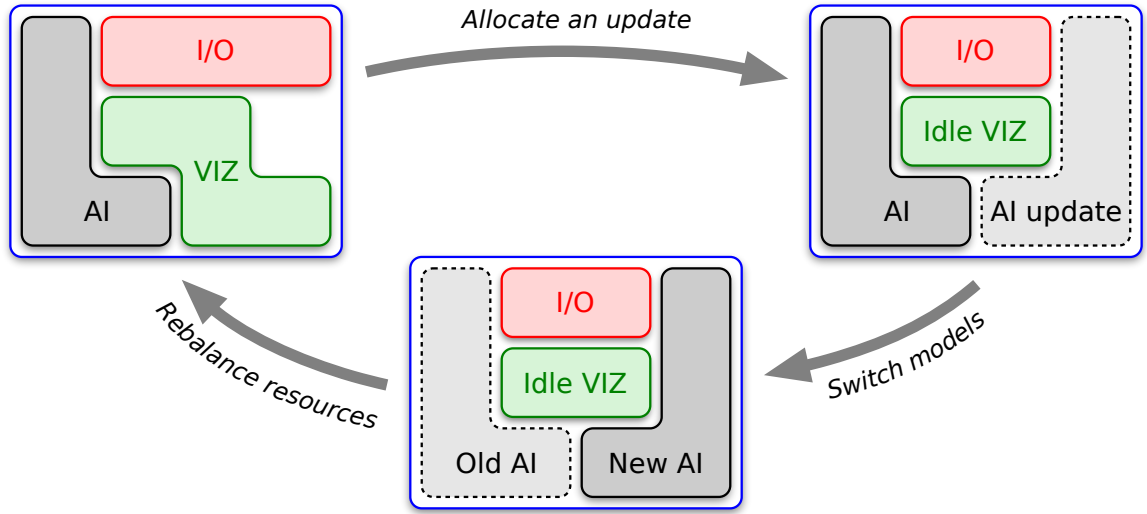


Figure 3.5: Updating the AI@Edge model while ensuring continuous operation and performance: resources need to be reprioritized during and after the update received from an HPC system.

infrastructure that can maintain information provided by users about the curation of data and models and 2) developing methods to update and adapt the configuration of edge devices when such a workflow triggers actions as shown in Fig. 3.5.

An Infrastructure for Model Management Which data to move, how to retrain a model, and when to retrain or replace a model are decisions best made by experts in their respective domain sciences and their particular experiments. This information, however, can be provided to an infrastructure that can then act on such policies while ensuring the performance and reliability of the surrounding environment. A function as a service (FaaS) component in the infrastructure can monitor the state and data produced by an experiment and be dynamically triggered to move the data to an HPC system and launch the tuning of AI components and their update. This infrastructure will have to take into account timing and performance constraints such as the volume of data that needs to be moved (inputs and model), the time needed to move it, and the required

resources. Indeed, we expect that, depending on the models being updated and the nature of both the HPC resources and the edge nodes, such infrastructure might operate at differing timeframes.

Sources of Variability at Run Time and Edge Capabilities The capabilities of edge devices (their hardware configuration) and similarly the capabilities of AI models (architecture, size) are always changing: new AI methods are developed, new accelerators become available, and instruments are updated (e.g., the ongoing APS upgrade). We expect that these changes will have an impact on the constraints and performance of our solution in the future. The flexibility of our infrastructure will be an asset in such cases, as new sources of monitoring data or controls can be added easily.

We acknowledge that some experiments might require real-time response behavior or might involve considering more than just average performance or short-term measurements for proper resource management. We intend to develop the infrastructure further toward those goals.

3.4 Experimental Evaluation

To demonstrate the ability of resource monitoring and control in the Charon framework, we designed a simple experiment in which a single computing cluster runs the scientific AI application Babu et al. (2022) to process live streaming data from a beamline detector. In this experiment, the rate at which the AI model processes X-ray images should ideally be matched with the data generation rate of the detector in order to prevent any data loss. The controller in this experiment is responsible for keeping up with the data generation rate by controlling the number of application instances, which changes the global data processing rate.

3.4.1 Setup

We use the Chameleon Keahey et al. (2020a) testbed to set up a single-node cluster and provision it using the proposed framework. The provisioning installs tools to bring up Nvidia’s GPU support and container run time. At the end of provisioning, the cluster is ready to accept user applications while monitoring their resource usage in real time. Additional tooling and visualization can expose the performance information to users, including Graphana dashboards using the data monitored by NRM and sent to a Prometheus instance.

The application workflow is as follows.

- **Data generator** loads the recorded scientific data of a beamline detector and streams the data as individual frames at the desired rate of 1,000 Hz.
- **Distribution server** takes the stream and distributes frames to any connected consumer instance; the distribution follows a simple round-robin policy among all connected consumers, to balance the load.

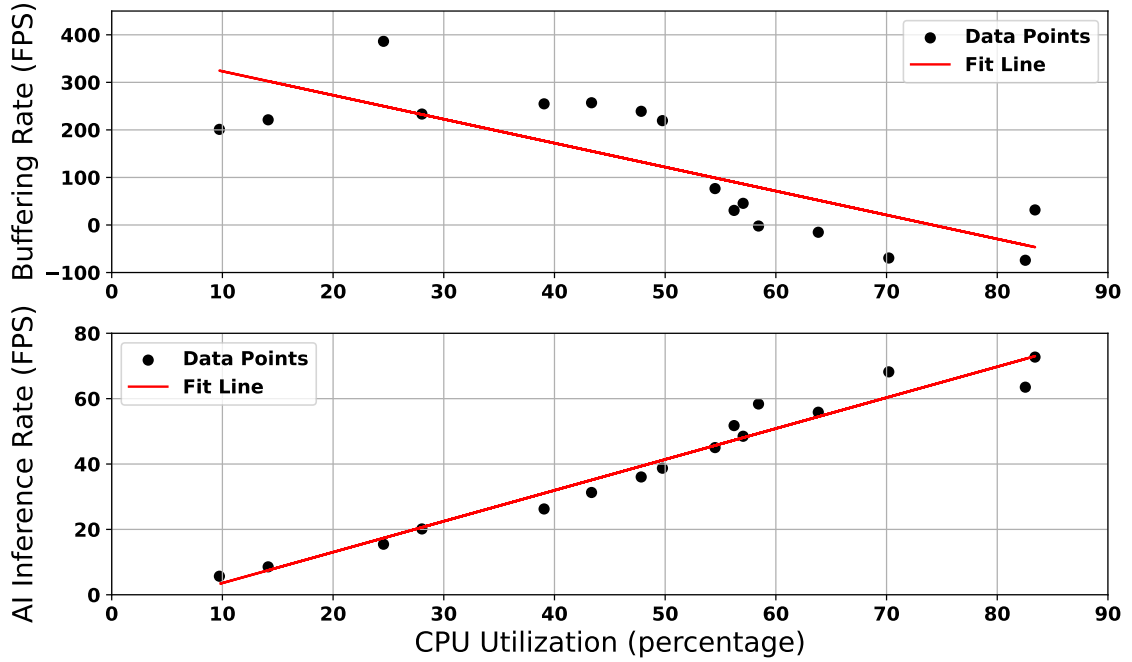


Figure 3.6: Checking the linearity of the controlled variables.

- **Consumer** runs the PtychoNN AI model to process frames; output data frames go to the next stage for aggregation.

This workflow is implemented by using PvaPy, a Python data streaming infrastructure developed by the Argonne APS staff for use with some of their detectors, and mimics a real deployment. Additionally, we instrument the AI@Edge application, that is, the consumer in the workflow, to constantly publish application performance metrics through NRM. The Charon controller then communicates with NRM to analyze incoming metrics and make control decisions. One of the metrics we use in this control experiment is the number of queued frames (received by a consumer but not yet processed), indicating whether the application is behind the data generation rate. An increase in this metric means that the workflow cannot process data frames at the rate of data generation and needs more computing power.

3.4.2 AI Inference Management and Control

First, we evaluate the impact of our controlled resources on overall performance. Since preprocessing of images is currently implemented as a sequential algorithm on the CPU, we consider preprocessing instances (containers) and CPU utilization to be the same metric (utilization between 0 and 100 is rate limiting for one container, between 100 and 200 for two containers, and so on). Note that we did not control the GPU resource because GPU resource division per application instance was unavailable outside of Nvidia's multi-instance

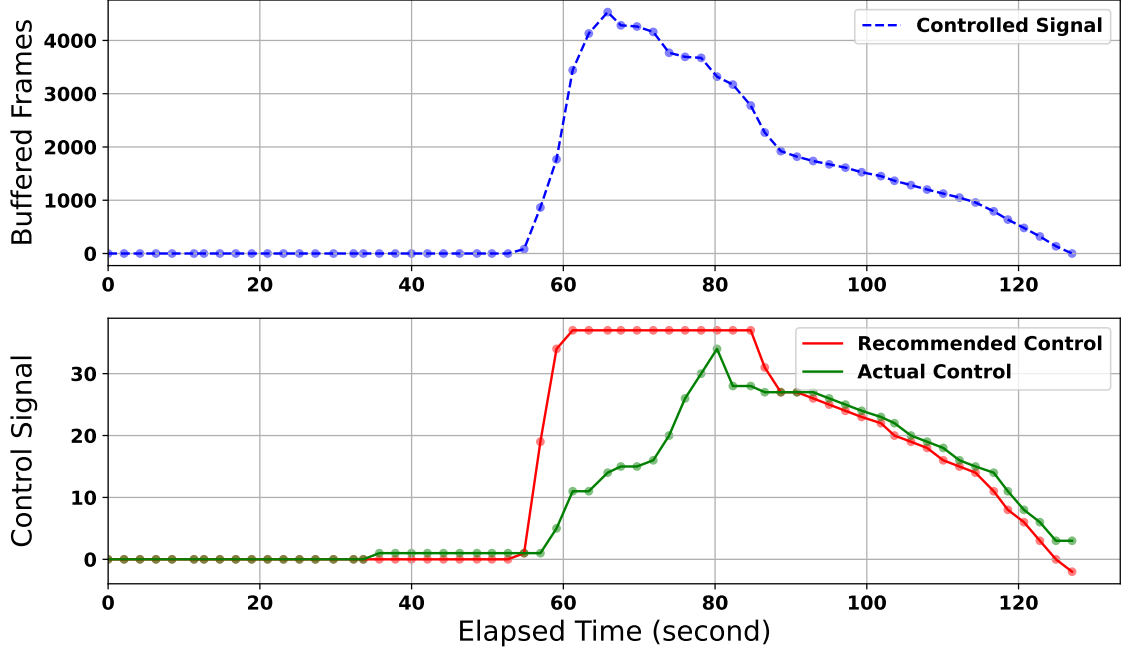


Figure 3.7: Controlling to reduce frame buffering toward zero. $K_p = 1$ and $K_d = 3$ were used. The control signal is emitted to Kubernetes to modify the number of AI@Edge instances (consumers) at run time.

GPU technology Li et al. (2022), which cannot be reconfigured dynamically. Nevertheless, this evaluation is part of the typical characterization process for the design of a controller. Figure 3.6 displays the relationship between inference rate, buffering rate of the data, and CPU utilization. As we allocate more CPU resources, the buffering decreases while the AI inference rate increases. Therefore, we confirm that linearity in the relationship exists, enabling us to design a PID controller.

Given this characterization, we design a proportional-derivative controller, using the number of frames waiting for processing (queue length) as the signal to minimize. The following equation defines our controller: $u(t) = K_p e(t) + K_d \frac{\delta e(t)}{\delta t}$, where $u(t)$ represents the control at time t , which is a proxy for CPU utilization at time t ; K_p is the proportional gain; K_d is the derivative gain; and $e(t)$ is the error between the actual and desired queue values at time t . Figure 3.7 illustrates how such a controller behaves on an experiment where the frame rate of the data generator is set to 1,000 frames per second. The controller waits an initial 20 seconds before starting, upon which new preprocessing containers are launched to respond to the growing queue length. Overall, around 30 container instances are progressively launched to handle the incoming frames. This controller design can be adapted further depending on needs; adjusting the control parameters K_p and K_d will change the aggressiveness of the control, leading to faster convergence but possible oscillation in the control.

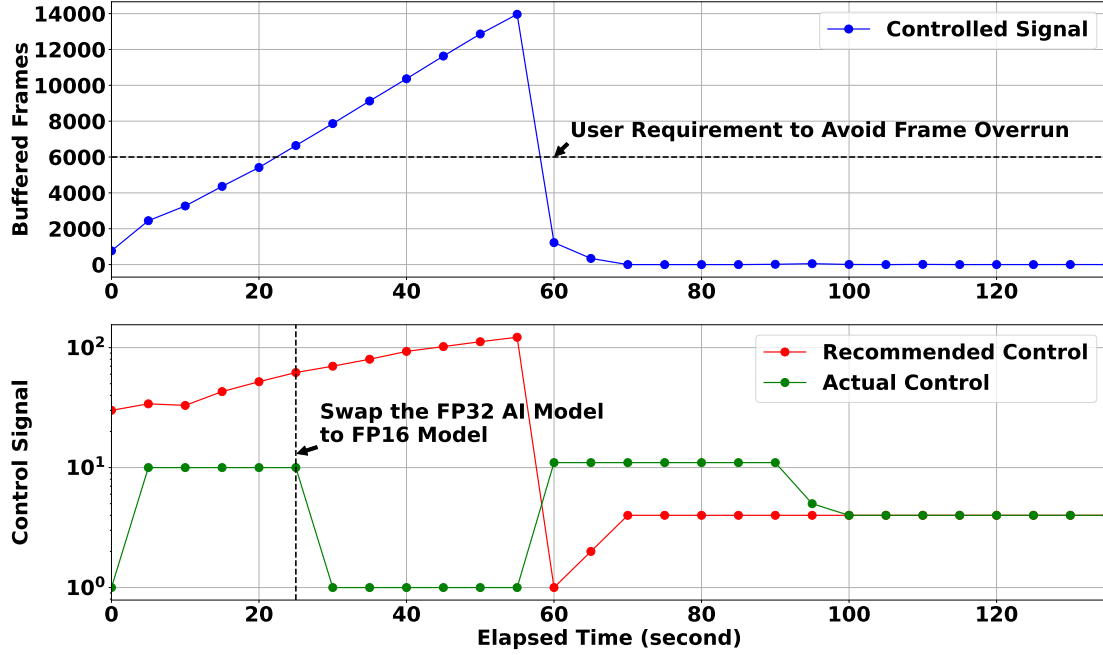


Figure 3.8: Demonstration of an adaptive PD controller, with $K_p = 0.5$ and $K_d = 1$. Per user requirement, the controller swapped the AI model to a smaller one to handle the incoming frame rate that overran the buffer.

3.4.3 Adaptive Control Using Different AI Models

In this experiment, we demonstrate how an adaptive controller can be applied to overcome frame overrun. The PtychoNN AI model is prepared in single-precision floating point (FP32) and half-precision (FP16) after quantization. The reduced precision slightly degrades the AI performance but gains computing efficiency, for example, memory reduction and faster floating-point calculation. We set the experiment with a 400-FPS input stream to make the buffer overrun by incoming frames when using the FP32 model within the provided computing node. Then, we simulate a buffer overrun threshold set by a user, triggering a switch to the FP16 model.

Figure 3.8 showcases the control and performance of this setup. Starting with the FP32 model results in a buffer quickly filling with unprocessed frames. Once the controller detects that the buffered frames signal exceeds the set threshold, FP32 model containers are shutdown and swapped with the FP16 model ones. Control resumes after the transition, this time with reasonable performance, as the FP16 models can handle the incoming frame rate (as they process frames much faster than the FP32 model, even with a fewer number of container instances). Note that due to implementation limitations, frames already sent to a consumer are lost in the model transition process, resulting in a sharp drop in buffered frames. An application, or a streaming data pipeline infrastructure capable of keeping track of frames would prevent this, and require

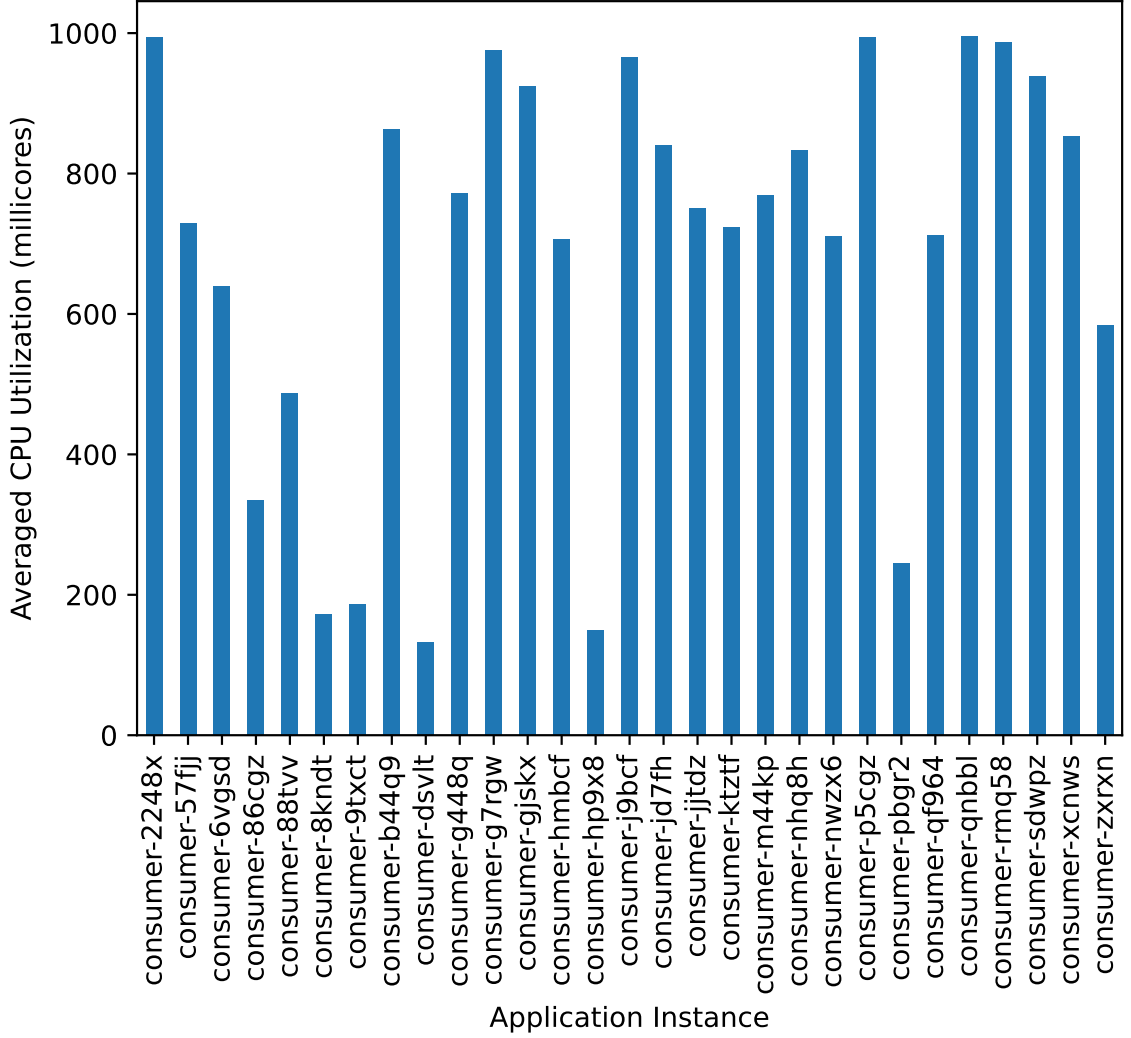


Figure 3.9: Averaged CPU utilization per application instance. Note that 2 of the 31 instances were not measured because they were terminated too early by the controller.

more control after the transition to quickly drain buffers.

3.4.4 Discussion

3.4.4.0.1 Resource Optimization

As shown in Fig. 3.7, the controller in the first experiment (see the subsection 3.4.2) spawned 31 application instances in total, which consumed approximately 20 CPU cores and 23 GB of memory to handle the 1,000-FPS data stream. Each instance was allocated 1 CPU core but used different amounts over the experiment (see Fig. 3.9) because they were spawned at different times and the total X-ray images to process varied over time.

Edge resources are often limited, for example by the physical space available near the experiment. In our current use case at the APS, this could lead to multiple tenants (experiments) sharing edge resources. In such a setup and with such a dynamic behavior for each scientific workflow, finer-grained control of resource allocation would be desired, to ensure performance and fairness to each user.

3.4.4.0.2 Model Size and Quantization

Model selection is one of the main factors in determining where the model runs in the heterogeneous edge-to-HPC continuum and what precision the selection offers. Smaller models process data much faster and run on less performant computing nodes. If a scientific experiment is robust to data quality reduction, model selection as a control action could provide more flexibility in the design and efficient use of AI@Edge systems.

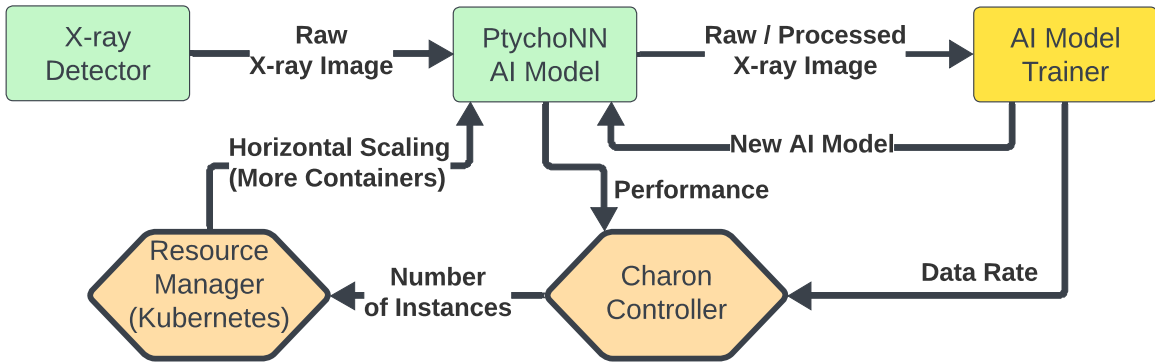
3.4.4.0.3 Container Run Time and AI Inference-as-a-Service

The application instances were a container run by Kubernetes. Each instance loaded the same model and dependent libraries within the container for AI inference. Although the programming model of the proposed framework is simple and easy to use, the initialization of an instance in the Kubernetes container run time took at least several seconds on a server-rack computer. In order to eliminate the duplicated library loading and react to rapid deployment and control of application instances, efficient programming models such as Nvidia Triton inference-as-a-service Savard et al. (2024) can be introduced in the framework. Similarly, enabling adaptation at each step of this scientific pipeline (data collection, preprocessing, inference) or our more developed use case (feature extraction, control, fine-tuning) would also provide opportunities for further performance improvements. We refer in particular to infrastructure stages for which hardware resources can be changed at run time (moldable jobs, heartbeat scheduling Acar et al. (2018)).

3.4.5 Toward Broader Edge-to-HPC Applications

Although the work in this paper targets the APS use case, introduced in Section 3.2, for real-time monitoring and control, the proposed framework and software stack bring fundamentals to support other edge-to-HPC applications. Figure 3.10 illustrates dataflow of the APS use case and the same for another use case, federated learning in edge-to-HPC. The Charon controller, the key fundamental, manages resources under local constraints and accepts inputs from both the edge and counter-part HPC applications. In the APS use case, the controller horizontally scales the AI application by sensing the application throughput. The AI trainer in HPC may set a specific data rate at which the trainer receives data from the edge to train a new model. Similarly, the federated learning (FL) client trains its local model with live data, while the FL server requests the local model weights from the client. The controller in this use case allocates more resources to the client

The APS Experiment



Federated Learning (FL) in AI

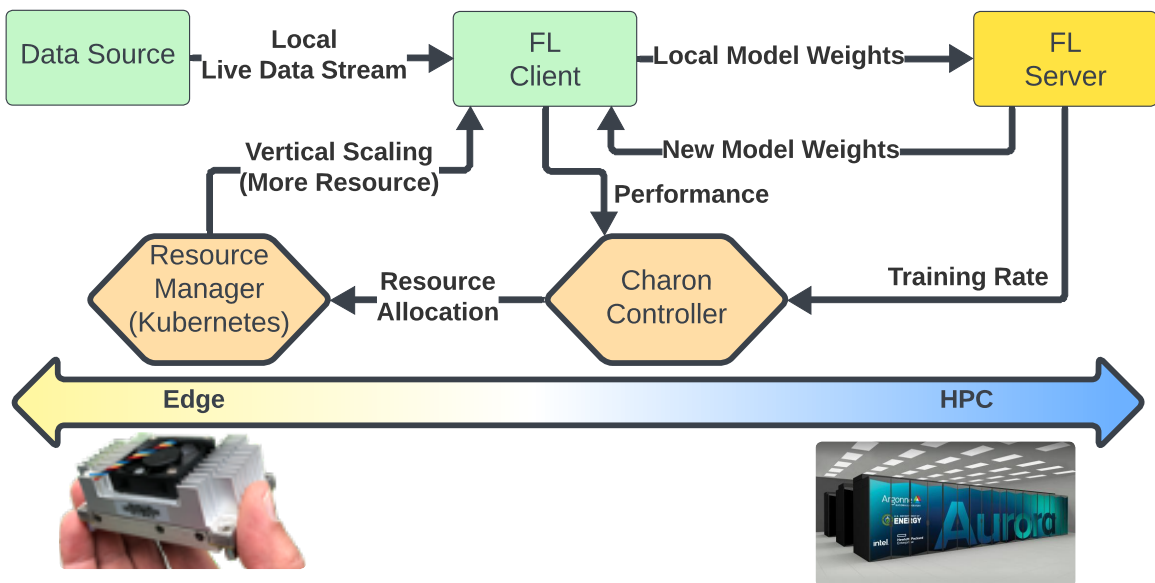


Figure 3.10: Use cases in the Charon framework. The Charon controller drives computation at the edge while communicating with the HPC application for the holistic control in edge-to-HPC.

if the server needs more training weights. This closed control loop allows many edge-to-HPC applications to manage their edge application at the high level in real time.

However, there are challenges in applying the proposed framework to potential edge-to-HPC applications. For example, the framework requires a communication channel to the applications with a descriptive language for the Charon controller to understand user requirements. Moreover, in an open network setup where edge nodes go online and offline and move their geo-location, the entire workflow should keep track of node availability to inject proper control inputs to the Charon controller in the node. Those challenges are left to the implementation to address.

3.5 Related Work

3.5.1 Resource Autoscaling

Computing clusters adjust allocated resources and application instances to meet user requirements. For example, the works using Kubernetes autoscaler Ju et al. (2021); Tran et al. (2022) control the number of application containers and their resource allocation based on reported application performance. However, the autoscaler is based on signals from the application and does not consider metrics at the workflow or infrastructure level. This situation brings the need to connect autoscaling to resource management and control at the macro level in the edge-to-HPC computing continuum. Furthermore, these autoscalers rely on simple policies (thresholds) and do not easily allow for swapping out these policies to a design adapted to the use case, as we are able to do with our infrastructure.

3.5.2 Edge Computing Platform

Edge computing is an active research field and can connect scientific instruments and sensors in the field to large cluster computing. The Waggle edge computing platform Beckman et al. (2016), one of the successful field-deployable nodes designed and built for data collection and analysis at the edge, has scheduling capabilities for multiple users to share node resources and run codes at the edge Kim et al. (2022). Early use of these edge scheduling capabilities in Waggle revealed the need for much finer-grained resource management in order to administer limited computational and storage resources for multiple users. To provide finer-grained resource management, in this paper we adopted the Argo Node Resource Manager Ramesh et al. (2019a), which enables application-specific resource usage monitoring at run time as well as dynamic resource arbitration, enabling tracking and reacting to applications' behavioral changes during the lifetime of a scientific experiment.

3.6 Conclusion

The expanded edge-to-HPC computing continuum enables flexible data processing under dynamic resource constraints but also raises problems in resource management and optimization. We investigated a specific scientific use case and designed a framework for managing edge resources to meet user requirements. In the simple experiments with a proportional-derivative controller, we achieved the target requirement of processing 1,000 X-ray images per second and showcased an adaptive control using model quantization. As we implemented and tested the framework, we discovered potential opportunities for further design and adaptation for more general use in the edge-to-HPC continuum.

One other possible future research direction for this work is the use of AI in the control itself. We are, in essence, building a more complex instrument by using hardware and software to combine user facilities and

HPC systems—an instrument that we intend to control at run time based on a set of complex performance criteria. Both monitoring this instrument and controlling it can be augmented by using AI methods, from generating good performance signals from a collection of sensors to using reinforcement learning to learn a policy that maximizes the value extracted from operating such an instrument. As with any other learning problem, collecting good data and access to a functional environment will be critical, which are precisely some of the expected main outcomes of this prototyping.

Acknowledgment

We thank APS scientists Changyong Park, Antonino Miceli, Tao Zhou, and others for invaluable inputs on their current and future workloads and workflows. Also, we thank Randall Berry, Scott Collis, Rajesh Sankaran, and Nicola Ferrier for supporting the initiation of this project. This material is based on work supported by the U.S. Department of Energy, Office of Science, under contract DE-AC02-06CH11357 and U.S. Department of Energy, Office of Science, under grant DOE-145-SE-PRJ1009506. Experimental results presented in this chapter were obtained using the Chameleon testbed supported by the National Science Foundation.

CHAPTER 4

A Reinforcement Learning Approach for Performance-Aware Reduction in Power Consumption of Data Center Compute Nodes

Abstract

As Exascale computing becomes a reality, the energy needs of compute nodes in cloud data centers will continue to grow. A common approach to reducing this energy demand is to limit the power consumption of hardware components when workloads are experiencing bottlenecks elsewhere in the system. However, designing a resource controller capable of detecting and limiting power consumption on-the-fly is a complex issue and can also adversely impact application performance. In this chapter, we explore the use of Reinforcement Learning (RL) to design a power capping policy on cloud compute nodes using observations on current power consumption and instantaneous application performance (heartbeats). By leveraging the Argo Node Resource Management (NRM) software stack in conjunction with the Intel Running Average Power Limit (RAPL) hardware control mechanism, we design an agent to control the maximum supplied power to processors without compromising on application performance. Employing a Proximal Policy Optimization (PPO) agent to learn an optimal policy on a mathematical model of the compute nodes, we demonstrate and evaluate using the STREAM benchmark how a trained agent running on actual hardware can take actions by balancing power consumption and application performance.

4.1 Introduction

Cloud data centers are increasingly providing a variety of high performance computing nodes including hardware accelerators to support the needs of a range of compute-intensive applications including various machine learning training/inferencing and scientific applications. In these setups, a large number of compute nodes are networked together via high-speed networks to provide high performance computing capabilities. This growth, however, has given rise to several challenges, notably in the areas of energy efficiency. For instance, Darrow et al. in Darrow and Hedman (2009) pointed out that data centers consume 10-50 times the energy per floor space compared to any other commercial building and the consumed energy constitute 2% of the total US energy consumption. Multiple surveys and studies Darrow and Hedman (2009); Koomey et al. (2011); Koot and Wijnhoven (2021); Shalf (2020); Dayarathna et al. (2015) have shown that the global power consumption for computing is expected to increase in the upcoming years.

Consequently, research on data center power consumption has gained considerable attention since the publication of initial statistics on its energy usage, which illuminated the rapidly increasing demands of

servers worldwide Koomey et al. (2011). A more recent survey shows that the growing demands of these data centers are not in proportion with their efficiency despite the technological innovations Koot and Wijnhoven (2021). In other words, the rate at which the power consumption increases across the generations of data centers are higher than the rate of growth in their efficiency. Within the data center, almost 86% of the total power consumption is equally shared between the cooling system and servers Shehabi et al. (2016), which makes the power consumption of compute nodes an important issue that needs to be addressed.

In the context of reducing compute node power consumption, various methods have been proposed that can be broadly classified into two categories: those that focus on minimizing power dissipation and those that focus on minimizing power supply. These approaches aim to reduce the overall energy consumption of compute nodes, while maintaining their performance and functionality. The former is strictly a Very Large Scale Integration (VLSI) design problem, while the latter is a cyber-physical systems problem, which can be dealt with using efficient power management algorithms (i.e., control system design), which is the focus of this paper.

Specifically, we design a method to reduce the consumed power without any noticeable impact on the execution time of a workload. Through our work, we achieve regulation of the average supplied power using a reinforcement learning (RL) agent, trained using a mathematical model which relates the progress made by the application towards the completion of its scientific-goal to the power cap (pcap) of RAPL actuators. Additionally, the goal is to not let the execution time vary significantly when compared to that with maximum power execution.

To that end, we present a real-time, dynamic power management scheme that is based on observing both time-aware and power-aware variables as a solution to the power management problem of the high performance compute nodes. In our work, the RL agent observes the progress made by the application towards completing its scientific goal at each instant and makes a decision on the power requirement for the next time step based on this observation. The action, which is based on this decision, is taken by setting the running average power limit (RAPL) power caps at the computed value. RAPL is an interface available on modern Intel processors to monitor and control the energy consumption of various power domains. Our approach is incorporated in the Argo Node Resource Management (NRM) stack Pete et al. (2021).

The key contributions of this paper are the following:

- A novel RL-based algorithm that relies on the system model for training.
- An RL-based architecture employing the trained model to control the compute node within a desired operational region, while utilizing real-time progress measurements.
- An implementation of the algorithm with an open-source repository for researchers to test and extend

the results.

The remainder of this paper is structured as follows: In Section 4.2, we review existing algorithms in power and performance optimization. Section 5.2 presents the necessary background for this paper, including the RL algorithm, hardware and software stacks, and packages used in the experiments. Section 5.4 outlines the proposed algorithm and workflow in the context of compute nodes. In Section 4.5, we present the experimental design and implementation, along with the results and analysis. Finally, Section 5.8 provides conclusions including a brief summary, the impact of this work and future opportunities.

4.2 Related Work

Prior methods for regulating the performance on HPC nodes can be classified either as a Power-aware scheme Hsu and Feng (2005); Patki et al. (2015) or as a Time-aware scheme Sadrosadati et al. (2019); Qiu et al. (2012). A power-aware scheme is designed with the primary objective of minimizing power consumption while still achieving acceptable performance levels. The main focus of this scheme is to optimize energy usage and reduce power consumption in computing systems. Power-aware schemes may employ techniques such as dynamic voltage and frequency scaling (DVFS), power capping, power gating, and other power management methods to control and regulate power usage based on workload and system conditions. The goal is to maximize energy efficiency and reduce the overall carbon footprint of the computing system. Time-Aware Scheme on the other hand, prioritizes meeting specific time or performance requirements, often associated with critical real-time or time-sensitive applications. The main objective of a time-aware scheme is to ensure that tasks or processes are completed within predefined time constraints or deadlines. In time-aware schemes, performance and response time take precedence over power efficiency. These schemes may use techniques like aggressive clock frequency scaling, parallelism, or task prioritization to meet the time requirements of critical applications. This section provides an overview of some of these methods which also explains why there is a growing demand for machine learning based methods for the power and performance control that has lead to the proposed work.

Our prior work Caglar et al. (2014) researched software approaches for power optimization using duty cycle, voltage and frequency scaling, wherein we used machine learning techniques to model the application performance and hardware characteristics, and use these models to guide the placement decisions. But, the choice of method highly depended on the hardware under consideration. Similarly, Jung et al. Jung and Pedram (2010) proposes a supervised learning based power management for multi-core processors. However, such algorithms are not always reliable due to the need for training data prior to the implementation of software-based optimization.

On the other hand, methods that deal with system architecture that set the input power based on the sys-

tem requirements converges faster. For example, a RAPL-based scheme supports a variety of algorithms that relies on hardware-based control for power capping on Intel processors David et al. (2010). But the automation/feedback required for the processor to determine the power requirements based on the application it runs is not detailed in the paper. Similarly Raghavendra et al. Raghavendra et al. (2008) utilizes a controller to stop the supplied power that is being delivered to the subsystems, by detecting the no-load zones in the application during its execution. Utilizing the technique, they are able to regulate the power to finer granularities. But, the efficiency of detecting the workload characteristics is not at the best, since the research focuses only on the hardware centric approach which is the main contribution of the paper.

Zhang et al. Zhang and Hoffmann (2016) later came up with a hybrid method, combining software and hardware based methods to control the power supply, which showed significant improvement over the other methods. The authors combined the efficiency of the software-based approaches like DVFS, Task Scheduling etc. and the timeliness of the hardware approaches like Power Gating, Voltage regulators etc. together by reducing the time it takes to implement a power cap on the RAPL sensor from the moment its value is set. But a workload-based adaptive control of power was not considered here. Petoumenos et al. Petoumenos et al. (2015), through their research give a comparative analysis of the software, hardware and hybrid approaches towards the power optimization in compute nodes.

Cerf et al. in her recent research Cerf et al. (2021b) proposed a control theoretical approach to understand the possible operating regions of a compute node at which the performance decay is minimized under a reduced power supply. A classical proportional-integral-derivative (PID) controller was designed to impose power cap (PCAP) for RAPL actuators on an Intel(R) Xeon(R) Gold 6126 CPU by using performance feedback from the compute nodes Ramesh et al. (2019b). The RAPL actuators comprising a control knob or the PCAP knob and a time-window knob were used to regulate the average power given between two sampling intervals to a compute node. A mathematical model formulated using static characterization was used for the design of the controller. The controller was shown to be working for the compute nodes by making it track a user-computed set-point for the desired performance. While an adaptive controller Hawila et al. (2022b) addressed model inaccuracies, reliance on a user-defined set point calls for an automated approach.

Considering the limitations in prior work, we emphasize the design of an optimal controller by solving the underlying power and performance optimization problem. We propose a hybrid approach for determining the best operating power cap for a compute node by relying on the instantaneous performance analysis that is obtained using the state-of-the-art algorithms.

4.3 Background

To make the paper self-contained, this section provides the necessary background on the concepts and technologies used.

4.3.1 Reinforcement Learning

Reinforcement Learning (RL) iteratively improves a sequence of actions or strategy through continuous interactions with the environment over a finite or infinite amount of time traversed using discrete time steps. In this work we use an RL agent to learn an optimal control policy by constantly generating an action and evaluating it using a mathematical model that describes the relation between power cap and progress made by the compute node while running an application. The RL problem can be mathematically formulated as a Markov Decision Process (MDP), where the agent interacts with an environment that is modeled as a set of states, actions, and rewards. At each time step, the agent observes the current state, selects an action based on a policy, and receives a reward from the environment. The goal of the agent is to learn a policy that maximizes the expected cumulative reward over time. In this paper we use a model-based algorithm by employing a Proximal Policy Optimization (PPO) Schulman et al. (2017) agent to learn the state transition and the reward functions, thereby learning the policy.

4.3.2 Performance Measurement

Instantaneous measurement of the controlled variable is an important aspect of feedback control problems. The RL algorithm for maximizing performance under a controlled power cap being an optimization problem, also requires the measurement of instantaneous performance. During the measurement, care must be taken that it does not interfere with the overall working and thereby the performance of the system. Therefore, we use a lightweight instrumentation library Ramesh et al. (2019b) and follow the instrumentation steps outlined in Cerf et al. (2021b). We formally re-define the progress measurement equation introduced in the paper Ramesh et al. (2019b) considering the number of messages received between two sampling instants. Therefore in this paper the progress at the time instant t_i is given by:

$$progress(t_i) = \underset{\forall k, t_k \in [t_{i-1}, t_i]}{median} \left(\frac{N}{t_k - t_{k-1}} \right). \quad (4.1)$$

where N is the number of messages received between the time instants t_k and t_{k-1} .

4.3.3 Intel RAPL

The running average power limit (RAPL) is an interface available with modern Intel processors for power monitoring and controlling making it unavoidable in energy efficient computing. It also allows users to

specify a power cap on available hardware domains using model-specific registers or the associated Linux sysfs subsystem. The RAPL interface uses two knobs, the power-limit knob and a time window knob to supply the average power for a user-defined period of time. The internal controller then guarantees that the average power over the time window is maintained. This mechanism also offers sensors through the same interface to measure the total energy consumed since the processor was turned on. Consequently, RAPL can be used for both measuring and limiting power usage Rountree et al. (2012).

4.4 Methodology

This section describes details of our model-based reinforcement learning (RL) which relies on a mathematical model representing the relation between the progress and the power cap (PCAP) in a compute node for training the RL agent. In our previous work Cerf et al. (2021b), where the performance of a compute node under a given application was controlled using a suitable PID controller running alongside the application, itself utilized all the available cores of the Skylake processor. However, a similar approach is not possible in the current work for training the RL agent because of the time and computational complexity involved. Therefore, we rely on the mathematical model for training the RL agent.

Figure 4.1 depicts our methodology while comparing it with the steps involved in the design of the PI controller in our prior work Cerf et al. (2021b). The processes involved in the two approaches beginning from the formulation of problem statement and extending till the evaluation are shown in the block diagram in Figure 4.1. It can be observed that the initial problem statement, the steps leading to the control formulation, and the system analysis are same in both the approaches. But, when it comes to the controller block, the PI controller relies on standard tuning methods from the classical control theory to determine its control gains, (i.e tuning the proportional, integral and derivative constants, K_p , K_d and K_i). In contrast, the RL method uses an iterative, policy improvement and policy update process by using an actor-critic network. It is only in this stage of the controller design that the two algorithms take different routes in the implementation. The rest of this section details the steps used in our approach, i.e., the branch shown in red color.

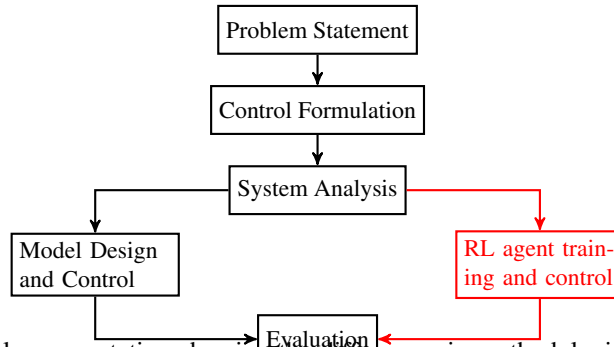


Figure 4.1: A pictorial representation showing the differences in methodologies implementing a classical control and an RL based control. The RL method discussed in this paper is colored in red.

4.4.1 Problem Definition

The first step is to define the objectives and the constraints involved in the optimization problem. Our objective is to maximize performance within a given power cap (PCAP) constraint, while taking into account the relationship between progress and PCAP as determined by the node dynamics. In other words, we aim to improve the execution time of the application while still using the minimum allowed energy. A simple way of ensuring this behavior is by making sure that the computational progress is at its peak under the given PCAP.

4.4.2 Mathematical modeling and MDP identification

Given that the current progress of our problem is dependent on both the previous progress and the current power cap, it is expected that the node dynamics follow the characteristics of a Markov Decision Process (MDP). Thus, to effectively apply a reinforcement learning to a problem, it is crucial to identify the MDP tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma)$ representing the state space, a set of actions available to an agent, the unknown transition kernel, the reward function and the discount factor, respectively. In addition to identifying the MDP, we also need to derive a mathematical model that represents the node dynamics so as to train the RL agent. To support the claim that the node dynamics have MDP characteristics and to derive the mathematical model, we use a static characterization approach and utilize line-fitting algorithms from SciPy Virtanen et al. (2020).

The experiment, which is part of the model building process, begins by collecting data points in the form of input-output pairs, where the input (i.e., PCAP) is provided at certain pre-defined instances and the output (i.e., progress made by the compute node on the workload) is calculated using Equation (6.6). These data points correspond to an entire benchmark execution where a constant PCAP is applied, and the progress signal is averaged. To model the time-averaged relationship between power cap and progress, we consider only stabilized situations of the progress value generated using PCAPs allowed by the RAPL actuators. At least 20 experiments are run for each compute node to generate enough data points. Based on these experiments, a static model is obtained by linking the time-stabilized power cap to the progress using line-fitting algorithms. We observed that the PCAP-progress relation follows an exponential relationship, as shown by Equation (4.2):

$$progress = K_L(1 - e^{-\alpha(a.PCAP+b-\beta)}), \quad (4.2)$$

where a and b parameters represent RAPL actuator accuracy of the node (slope and offset, respectively). Therefore, the actual power applied on a node can be approximated as: $a.PCAP + b$ which is also the measured power. α and β characterize the benchmark-dependent power-to-progress profile and K_L is the linear gain and is both benchmark- and node-specific. The effective values of the fitting coefficients given by α, β, a, b, K_L and τ are obtained using the above mentioned steps, where the line fitting problem follows a non-linear

least squares optimization. The linear characterization of dynamics can be then computed using least square optimization Cerf et al. (2021b) and is given as

$$\begin{aligned} progress_L(t_{i+1}) = & \frac{K_L \Delta t_i}{\Delta t_i + \tau} \cdot PCAP_L(t_i) \\ & + \frac{\tau}{\Delta t_i + \tau} progress_L(t_i), \end{aligned} \quad (4.3)$$

where $\Delta t_i = t_{i+1} - t_i$ is the control interval and τ is the time constant. The values of Δt_i and τ are chosen as 1s and $\frac{1}{3}$ s, respectively, for the experiments. We can use either of these models for training.

After defining the Markov Decision Process (MDP), we proceed to assign distinct characters to each role in the MDP, which include the state, action, and observation spaces that were previously established. In this study, the current state, denoted as $s' \in \mathcal{S}$, is represented by the progress value at time t , $progress_t$. On the other hand, the action, denoted as $a \in \mathcal{A}$, taken by the agent, is determined by the PCAP value at time t based on the observation $progress_{t-1}$ ($s \in \mathcal{S}$), which represents the progress value at the time $t - 1$.

4.4.3 Reward function

We now explain how the reward function is chosen to facilitate fast and accurate learning. The reward at a given time t reflects the quality of the control action applied to the system at that moment in time indicating how well the action helped achieve the goal or optimize the system's performance. The instantaneous reward, i.e., reward at a moment in time, is calculated by designing an appropriate reward function that maps the state-action space into the set of real numbers $\mathbb{R}$. The choice of an appropriate reward function depends on how effectively it can discriminate the changes happening in the environment when a control action is applied. As explained in the Subsection 4.4.1, our aim is to maximize the performance under a given PCAP. By negating the power consumption measured at time t during each of the sampling instance, we can easily take care of the power reduction i.e., $R(s, a) = -measured\ power(t)$. This reward function penalizes an action that can lead to an increased power consumption, but at the same time ignores its effect on performance.

An alternate choice could be $R(s, a) = -measured\ power(t) + progress(t)$, however, this is not a good choice for a reward function since it fails to record the trends in the power and performance. For example: A higher progress at a lower PCAP cannot be differentiated from another that may return a lower progress at a higher PCAP. Therefore, a suitable choice of reward function that can differentiate these modes of power and performance is $R(s, a) = \frac{progress}{measured\ power}$.

To give extra emphasis on the reduction of power, we decided to consider the following reward function

consisting of a linear combination of the total measured power and the instantaneous progress to PCAP ratio:

$$R(t) = \underbrace{-c_1 * PCAP}_{\text{minimize consumed power}} + c_2 * \underbrace{\frac{progress(t)}{measured\ power(t)}}_{\text{maximize performance per watt}} \quad (4.4)$$

where c_1 and c_2 are scaling factors which we will determine by observing a large number of training performances. In the mathematical model developed using the static characterization in Equation 4.2, the measured power is computed as $measured\ power = e^{-\alpha(a.PCAP+b-\beta)}$ which we will be using for the learning process. We tested our algorithm with a variety of values for c_1 and c_2 and observed that more than one value exists as a valid candidate for c_1 and c_2 . One interesting combination which we used for the training and testing is $c_1 = 0$ and $c_2 = 4.44$ which focused only on the maximization of the performance under the given power cap. Therefore, under the present simulation conditions, a reward function of $R(s, a) = \frac{progress(t)}{measured\ power(t)}$ is expected to be equally good when compared to an enhanced power reducing reward function given by Equation 5.3.

4.4.4 RL Agent Training

The RL system is trained using the mathematical model given by the Equation 4.3 which is linear in nature. Here, we follow a model-based training approach that is used in a variety of applications Williams et al. (2017). A model-based training has the advantage of not risking the safety and stability of hardware while evaluating a variety of control actions. Therefore, while training, we used the entropy term of the PPO agent to enable exploration. This helped in learning an optimal policy faster. The flow diagram in Figure 4.2 shows the general idea employed under the training using the mathematical model.

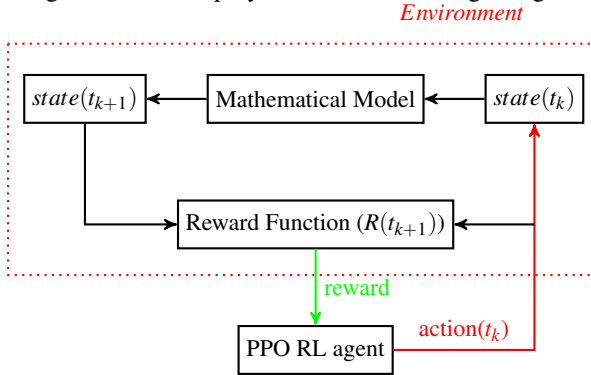


Figure 4.2: Training of the RL-agent as described in the paper. The RL-agent receives a reward, for every action it takes, calculated using the values of state variables obtained from the mathematical model.

4.5 Empirical Evaluation of Reinforcement Learning Approach

This section presents details of evaluating our approach for its efficacy in meeting its objectives. First we present the criteria using which we evaluate the proposed method for controlling a data center compute node.

We then present the results and our analysis.

4.5.1 Evaluation Strategy

We compare here the total time taken for executing the workload and the total energy consumption during that period, with two standard operating conditions of a data center compute node. The evaluation consists of the following analysis based on which we draw conclusions about the efficiency (percentage value of increase or decrease in the execution time and power consumption) of the proposed method:

- A comparative analysis of the proposed RL-based control method with the PI controller proposed in prior work Cerf et al. (2021b),
- A comparative analysis of the RL-based control with the efficiency of the system with minimum and maximum power caps,
- A repeatability analysis of the experiment on the same compute node, and
- A repeatability analysis involving different compute nodes.

For the comparative analysis with the PI controller, we test our RL agents that are trained using a variety of reward functions and generated using varying values of c_1 and c_2 while running the benchmark application. Similarly we use the optimal controller among these trained models to control the PCAP, while running the benchmark application, and compare it against the maximum and minimum PCAP runs. We repeat this experiment 10 different times on a node to analyze the statistics associated with the results. We also test the algorithm on different nodes and analyze the statistics for the repeatability of the results on varying hardware. Through these evaluations, we also comment on the execution time, total power consumed, and the related statistics for each experiment.

4.5.2 Implementation of RL Agent and Experimental Platform

The code for the training and testing the algorithms are written in Python3 with the required support packages installed on the Nix-environment manager Dolstra and Löh (2008). Raffin et al. (2021) introduces a Python3-based software package called stable-baselines-3 (SB-3) that give users a leverage to code RL algorithms seamlessly. With the support of PyTorch Paszke et al. (2019), we developed an easy to implement architecture for all the standard RL methods shown in the literature.

The RL agent resides inside the the Argo Node Resource Manager (NRM) stack Pete et al. (2021). Argo NRM is an infrastructure developed as part of the U.S. Department of Energy Exascale Computing Project called Argo, for the design of node-level resource management policies. It provides users an easy-to-use

and a unified interface (through Unix domain sockets), to the various monitoring and resource control knobs available on a compute node (e.g., RAPL, performance counters).

All experiments in this paper were performed on Skylake compute nodes using the STREAM benchmark. STREAM McCalpin (1997) is a standard application benchmark comprising a set of simple, portable programs that can be used to measure memory bandwidth on a variety of computer systems. The benchmark measures the memory bandwidth for four different types of operations: copy, scale, add, and triad. STREAM is chosen as it is representative of memory-bound phases of applications and shows a stable behavior. STREAM is also easy to modify into an iterative application: its four kernels are ran a configurable number of times in a loop, with a heartbeat being reported to the NRM each time the loop completes, i.e., after one run of the four kernels.

All four tests are performed on arrays of floating-point numbers and the results are reported in terms of the achieved memory bandwidth (in bytes/second). The problem size is set to 33,554,432 with 10,000 iterations for the experiment during its evaluation. In the context of the STREAM benchmark, the problem size refers to the amount of data in bytes that will be used to execute the benchmark. The problem size can be varied to measure the memory bandwidth at different data set sizes. The iterations refer to the number of times that the benchmark will be executed for a given problem size, and the results of the iterations are averaged to reduce the impact of measurement variability. Typically, a large number of iterations are used to obtain accurate and consistent results.

Note that availability of RAPL actuators and sensors on the compute nodes are important for our experiments. Moreover, each node should have a reasonable range of operation region under the given power cap before it attains saturation so that a number of them can execute the STREAM benchmark seamlessly. We use Chameleon Cloud Keahey et al. (2020b), a server facility made available for researchers to perform experiments that requires low-level access to hardware. Hosted on a variety of locations spanning across the country, it offers hardware components of various specifications. We use the Skylake nodes from the available servers for our experiments that use an Intel Xeon Gold 6126 processor with 12 dual threaded cores. Above all, it belongs to the modern series of the Intel processors with RAPL actuators and sensors.

4.5.3 Experiment Details

For the first part of the experiment involving the static characterization, described in the Subsection 4.4.2, we used a Chameleon Cloud node to run the STREAM benchmark while the daemon collected the progress measurement. We actuated the RAPL power knobs with test PCAP values in the form of step signals. A total of 17 step signals were given and the progress values were calculated using Equation 6.6 and recorded once the steady state was attained. We then collected these datasets consisting of the PCAP-progress pairs and used

the Scipy line fitting algorithm to obtain the best fit equation. Figure 4.3 depicts the non-linear and linear representations of data points and the curve that was obtained using static characterization, respectively. The parameters obtained after the line fitting are given in Table 4.1.

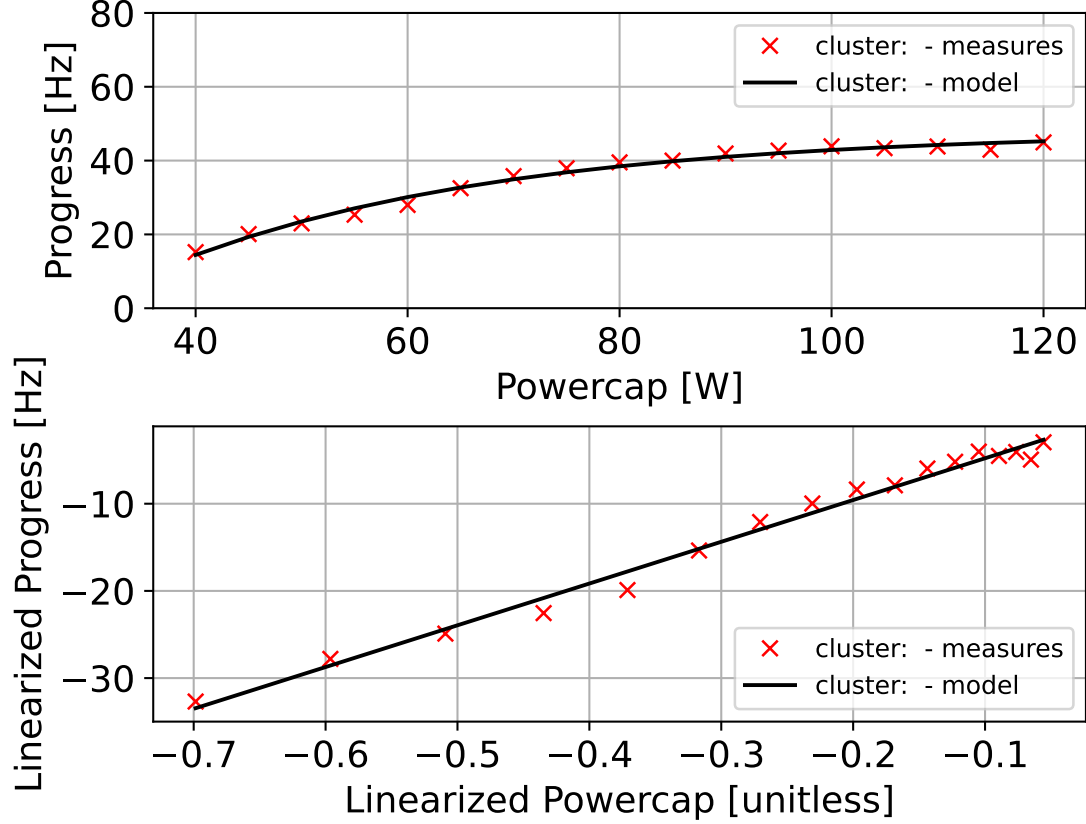


Figure 4.3: Mathematical model of an HPC node representing the non-linear and the corresponding linear relations existing between PCAP and progress, obtained using static characterization, showing the data points (red 'x') and the best fit curve (black '-').

Description	Notation	Unit	Value
RAPL slope	a	[1]	0.95
RAPL offset	b	[W]	0.15
	α	[W ⁻¹]	0.041
power offset	β	[W]	24.3
linear gain	K_L	[Hz]	47.9
time constant	τ	[s]	1/3

Table 4.1: Model parameters obtained using static characterization of an Intel Xeon Gold 6126 processor.

In the next part of the experiment, the obtained parameters were used in the mathematical model shown in Equation 4.2 for generating the next state (progress) in response to an action (PCAP). We employed a PPO-RL agent, to generate a policy and execute it on the mathematical model thereby generating the next state followed by the corresponding reward given by Equation 5.3. At this point, we would like to bring to the

reader's attention that we are not running the application nor the NRM daemon while using the mathematical model. We can implement this training algorithm on any standard processor and do not require any specific hardware. Therefore a simple Python code deploying the PPO-RL agent evaluated the policy, followed by periodic updates which assured its convergence to an optimal value.

For determining the best reward function that could generate an optimal policy, we performed the training using different reward functions, obtained by varying the c_1 and c_2 in the Equation 5.3. We then collected all the models and tested it on the mathematical model to determine its impact on the theoretical performance and power, and determined the region of interest. We also tested these models on the hardware running the application and daemon to compare it against the PI controller.

Having determined the optimal reward function, we used the optimal model on the hardware for testing followed by evaluation. For the testing and evaluation, we reintroduced the hardware and the workload to execute, with the help of NRM. This time, the trained RL-agent provided the PCAP for controlling the power knob of the RAPL actuators. Following this, the corresponding progress values were measured and fed back into the RL network to compute the next PCAP. This cycle is repeated till the entire work load is finished execution. We repeated the experiment for a variety of scenarios described in the Subsection 4.5.1.

4.5.4 Empirical Results and Discussion

In this section we will explain the results obtained after following the experiment design described in the previous section. We will also perform the evaluation of the algorithm followed by the interpretation of the results obtained.

Firstly for choosing an appropriate reward function we trained the RL-agent using the reward function given by the equation (5.3) with varying values of c_1 and c_2 followed by testing on the mathematical model for its performance. The theoretical values of performance (execution time) and the total energy consumed during the execution, were obtained using the control generated by each of the agents and are as shown in the Figure 4.4. By using a series of values for c_1 and c_2 starting from 0 followed by increments of 0.1, till 10, we were able to observe the variations in the performance and power for each of the designed reward. As expected we obtained a combination of values for c_1 and c_2 that yielded respective results ranging from the fastest to the slowest performance as shown in the figures 4.4. Observing the figure we can see that there is a point in the maximum curvature region (marked in blue circle) in figure 4.4 that can be considered as an ideal region to look for a candidate reward function. This point corresponds to the minimum energy consumption and the maximum performance. Note that, there were a couple of ideal candidates for the reward function that repeated across multiple execution of this experiment, for example, $(c_1=1.052, c_2=2.22)$ and $(c_1=0, c_2=4.44)$ gave the same responses. We chose the model, that was obtained after training using the reward function,

$R(t) = -1.052PCAP(t) + 2.22 \frac{progress(t)}{measured\ power(t)}$, as the RL-agent to control the HPC nodes during the testing phase.

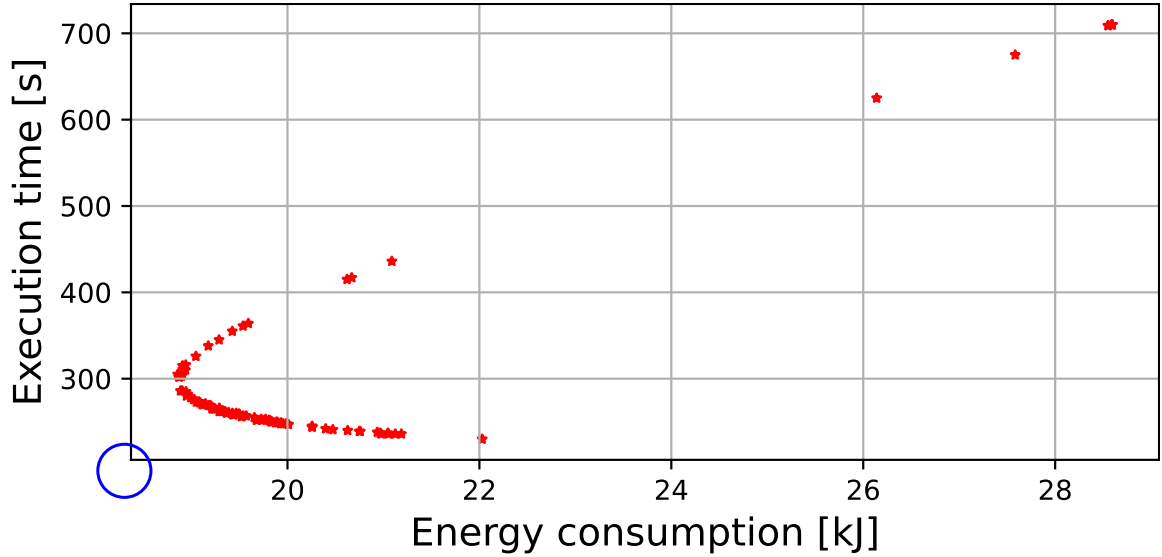


Figure 4.4: A Pareto of execution time vs energy consumed, generated by varying the RL-agents trained with different reward functions. The x-axis shows the total energy consumed during the execution, and the time of execution is plotted on the y-axis. The ideal characteristics are inside the blue circle.

4.5.5 Analysis of Results

Before we proceeded with the analysis and evaluation of the proposed optimal controller for the PCAP, we needed to verify its performance by comparing it against the existing PI controller. For that, we needed to run both the algorithms on the same machine. We first executed the PI controller on the HPC node with varying set-points as shown in the Figure 4.5. The pareto of points with gradients of colors ranging from yellow to blue depicts the result obtained for the PI controller with varying setpoints. The setpoint was specified using the parameter ϵ , representing the allowed tolerance in the maximum performance value. The ϵ value was varied from 0 to 0.6 randomly, during the experiments using PI controller. The shortest execution time of 240s (corresponding to faster execution) was obtained at an energy consumption of 36kJ at an ϵ value of 0.1. The setpoint value is expected to change based on the hardware being tested, whose dependency on the controller is being removed while using an RL controller. We then tested the trained RL controllers on the same hardware running the application and created a similar pareto of points. In the Figure 4.5, the red points represent the response of the HPC node controlled using the trained RL-agent. We observed that the optimal region consisted of points vary close to the best performance obtained using the PI controller. It was observed that the $R(t)$ proposed in the previous subsection fell within a closer range of the best performance.

Through these comparisons we were able to justify the use of a trained RL-agent as an optimal controller aimed to minimize the power consumption. Using the proposed RL controller, we were able to regulate the HPC node performance around those regions of operation, where a PI controller was regulating the performance, given a user-defined set-point. We also were able to excite the HPC node at some other regions where a slower response was obtained but at a very low energy consumption. We will now look for points where optimal performance and power consumption was recorded and will check for the repeatability of the experiments using the same hardware as well as different hardware.

We used the RL-agent trained using the reward function chosen previously, for the remaining part of the experiment consisting of:

- Repeatability on the same node and
- Repeatability on different nodes.

Within each of these analysis, we considered two different scenarios to compare with the performance of the proposed method, namely:

- Maximum PCAP: allowing the node to utilize the maximum allowed capabilities of the system,
- Minimum PCAP: limiting the performance, by allowing only the minimum power at the actuators.

For the analysis of the repeatability of the experiment on the same node, we conducted a total of 30 executions of the experiment with RL controller being used for 10 of the executions. In the remaining 20 executions we used the maximum allowed and minimum allowed values for the PCAP to record the statistics. We didn't consider the PI controller for the comparison here, since we are only evaluating the repeatability of the proposed method. We then observed the statistics associated with each set of executions and the values are tabulated. Figure 4.6 depicts the results of the experiment with the execution time plotted against the energy consumed during the each of the experiments. Figure 4.7 shows the values of instantaneous PCAPs sensed during each time step, by the RAPL sensors for three different executions using maximum, minimum and optimal PCAP respectively. In Figure 4.6, ensuring the maximum PCAP for the RAPL actuators, that put no limit on the amount of energy drawn into the system, the workload execution was completed in 245s. But when we look at the total energy consumed over the period it was in the range of an average value of 60.97kJ. On the other hand, enforcing the minimum PCAP on RAPL actuators, the power consumption was minimal, i.e in the range of an average value of 47.49kJ of energy, but compromising on the total execution time which lasted for 501.14s. On the other hand, the RL based controller was able to limit the power consumption and comparable execution time. It was observed that the average power consumption, over 10 experiments using the optimal PCAP, was 48.23kJ, while the average execution time was 261.19s. This shows a reduction of

21% on the consumed power, while the performance was compromised only by 6.5%. The statistics of the experiment are given in the table 4.2

	Minimal	Maximal	Optimal
μ Execution Time [s]	501.14	245.23	261.19.
σ Execution Time	26.20	3.6	5.89
μ Energy Consumption [kJ]	47.49	60.97	48.23
σ Energy Consumption	1.98	0.77	0.744

Table 4.2: Statistics of repeatability using the same node over 10 executions. μ and σ depict mean and standard deviation respectively.

For the next part, we performed a total of 21 executions consisting of 7 sets with 3 executions each with maximum, minimum and optimal PCAP. Each set was executed on different hardware. We made the reservation for 7 nodes from the Chameleon Cloud using a single lease in-order to assure diversity in hardware. The results of the execution was then gathered for analysis of its statistics. Figure 4.8 shows the obtained results of the experiment. It was observed that the proposed RL based controller was able to bring down the energy consumption by 19.7% at a cost of 7.3% in performance. It was also observed that the variance among each set of the experiments have gone high due to the changes in hardware, but in effect the optimal controller performed with the minimum variance. The related statistics are presented in the table 4.3. Through

	Minimal	Maximal	Optimal
μ Execution Time [s]	459.14	242.85	260.28
σ Execution Time	40.21	8.25	12.03
μ Energy Consumption [kJ]	43.3	59.52	47.78
σ Energy Consumption	3.78	3.77	2.22

Table 4.3: Statistics of repeatability over 5 executions on different HPC nodes. μ and σ have the usual meaning.

the repeatability experiments we were able to prove the reliability of the proposed approach for generating an optimal control for HPC nodes. We were also able to verify that the energy consumption reduction is consistent throughout its execution irrespective of the hardware.

4.6 Conclusion

This paper addresses the problem of managing power consumption in heterogeneous data center compute nodes by focusing on the potential of dynamically adjusting power across compute elements to save energy with almost no impact on performance. Our approach involves model-based reinforcement learning that learns using a mathematical model, which is generated using static characterization of a specific hardware node running a standard application benchmark. The control of the hardware, then uses the trained RL

agent to ensure optimal performance. Experimental validation shows promising results for systems running a memory-bound benchmark. The repeatability of results showed a standard deviation of 5.89 with a mean energy consumption of 48.23 kJ during 261.19s of mean execution time. These results indicate that the RL-based approach can serve as an alternative to the PI controllers used in our past experiments in order to remove the dependency on operational setpoints. The approach presented in this paper is most suitable for cloud providers and high performance compute node administrators who seek solutions to reduce power consumption of their server resources yet not impact application performance. In our future work, we aim to resolve the dependency of the proposed work on the mathematical model for training. We also seek to design a generalized RL-based controller, controlling PCAPs on nodes executing a variety of applications.

Acknowledgments

Results presented in this paper were obtained using the Chameleon testbed supported by the National Science Foundation. Argonne National Laboratory’s work was supported by the U.S. Department of Energy, Office of Science, Advanced Scientific Computer Research, under Contract DE-AC02-06CH11357. This research was supported by the Exascale Computing Project (17-SC-20-SC), a collaborative effort of the U.S. Department of Energy Office of Science and the National Nuclear Security Administration. The dataset and the codes used for the entire experiment, including the static characterization, training and testing, are hosted on our GitHub repository. We also provide readers with complete access to our Chameleon Cloud image, which will enable them to reproduce the results.

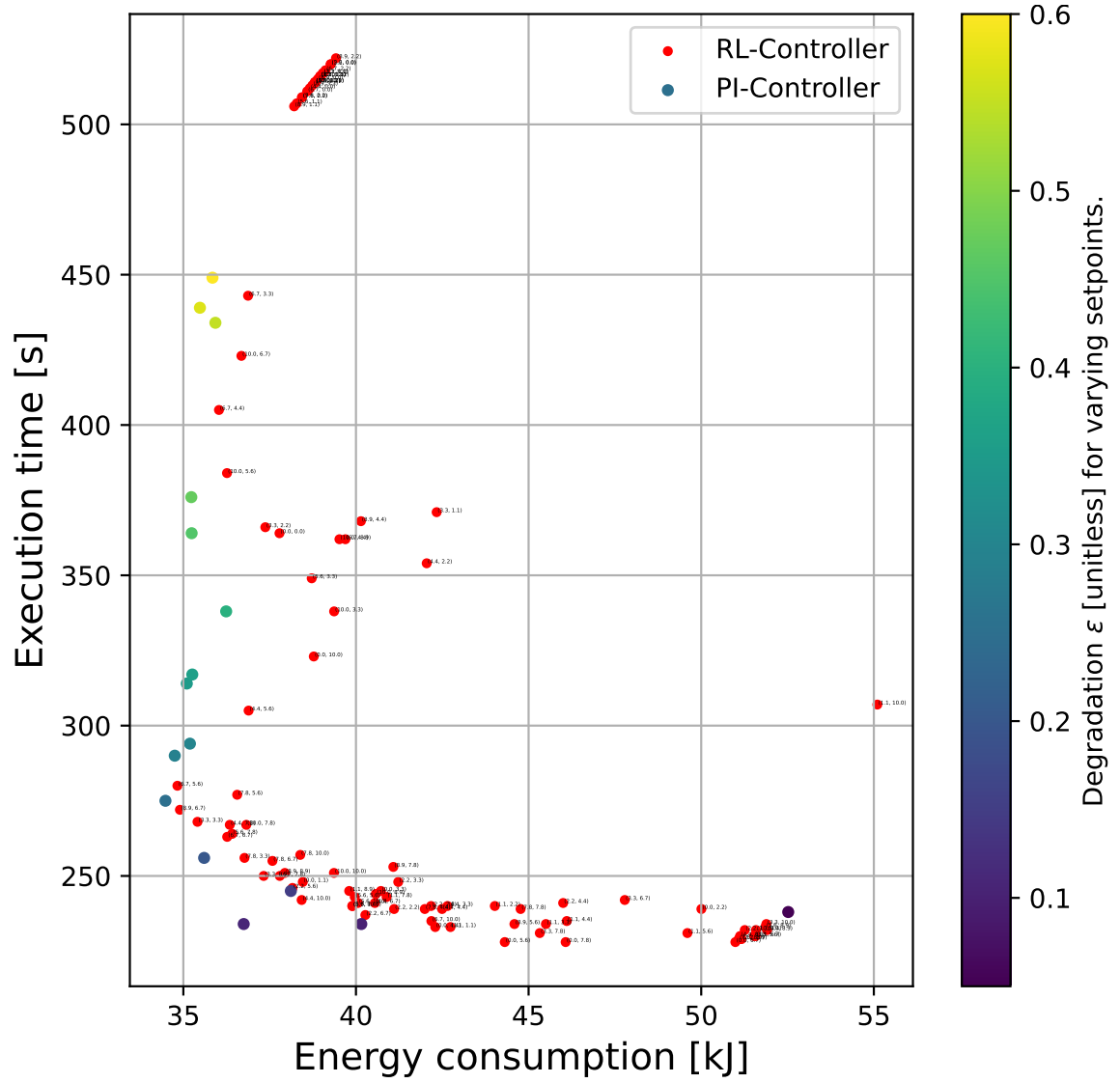


Figure 4.5: Plot showing the comparison of PI controller with varying setpoints (values for $1 - \epsilon$) and RL controller for varying reward functions. The PI control objective is given as a degradation factor ϵ , that is, the tolerable loss of performance

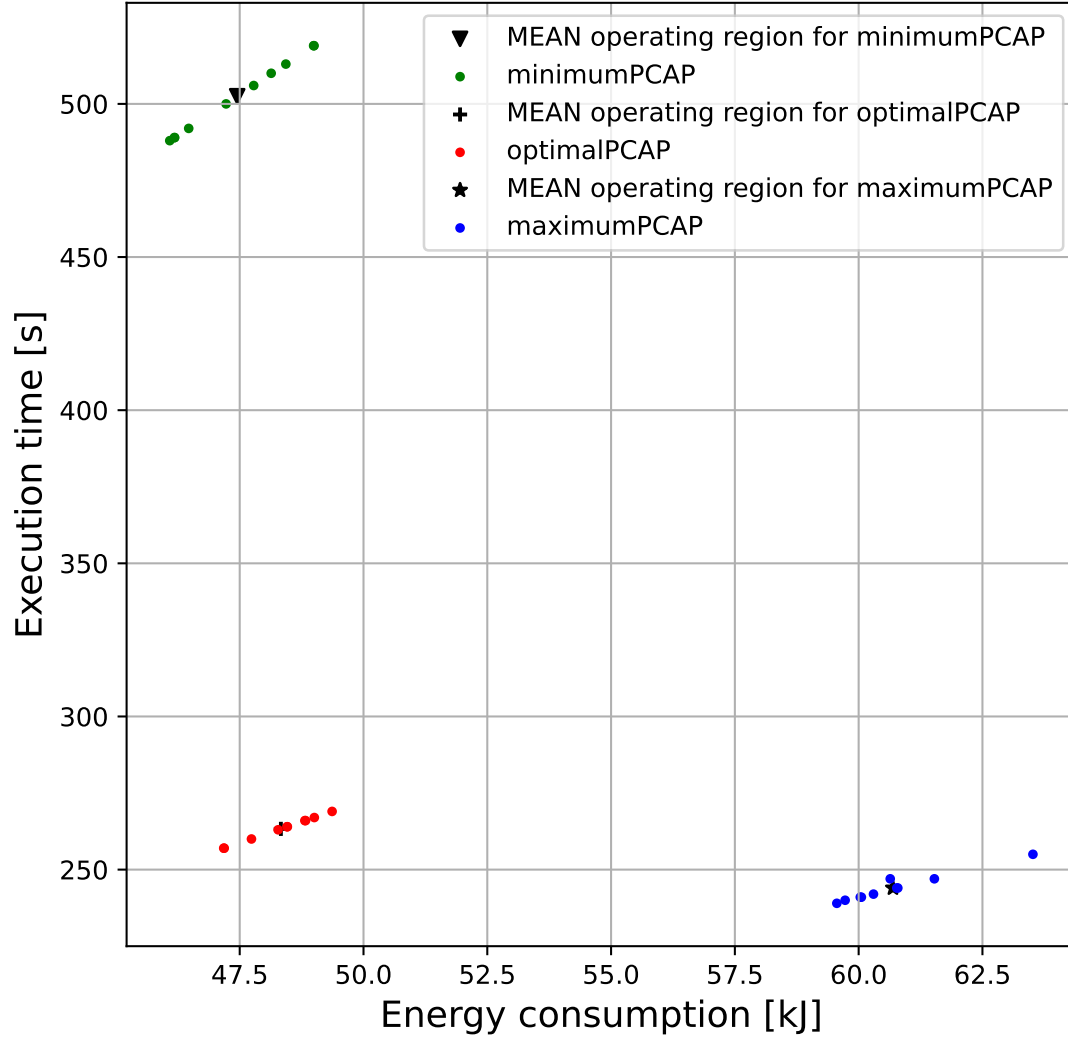


Figure 4.6: Comparison of the execution-time and the energy consumed during the execution of the experiments performed on the same hardware employing three different methods. The statistics of the executions are shown in the table 4.2

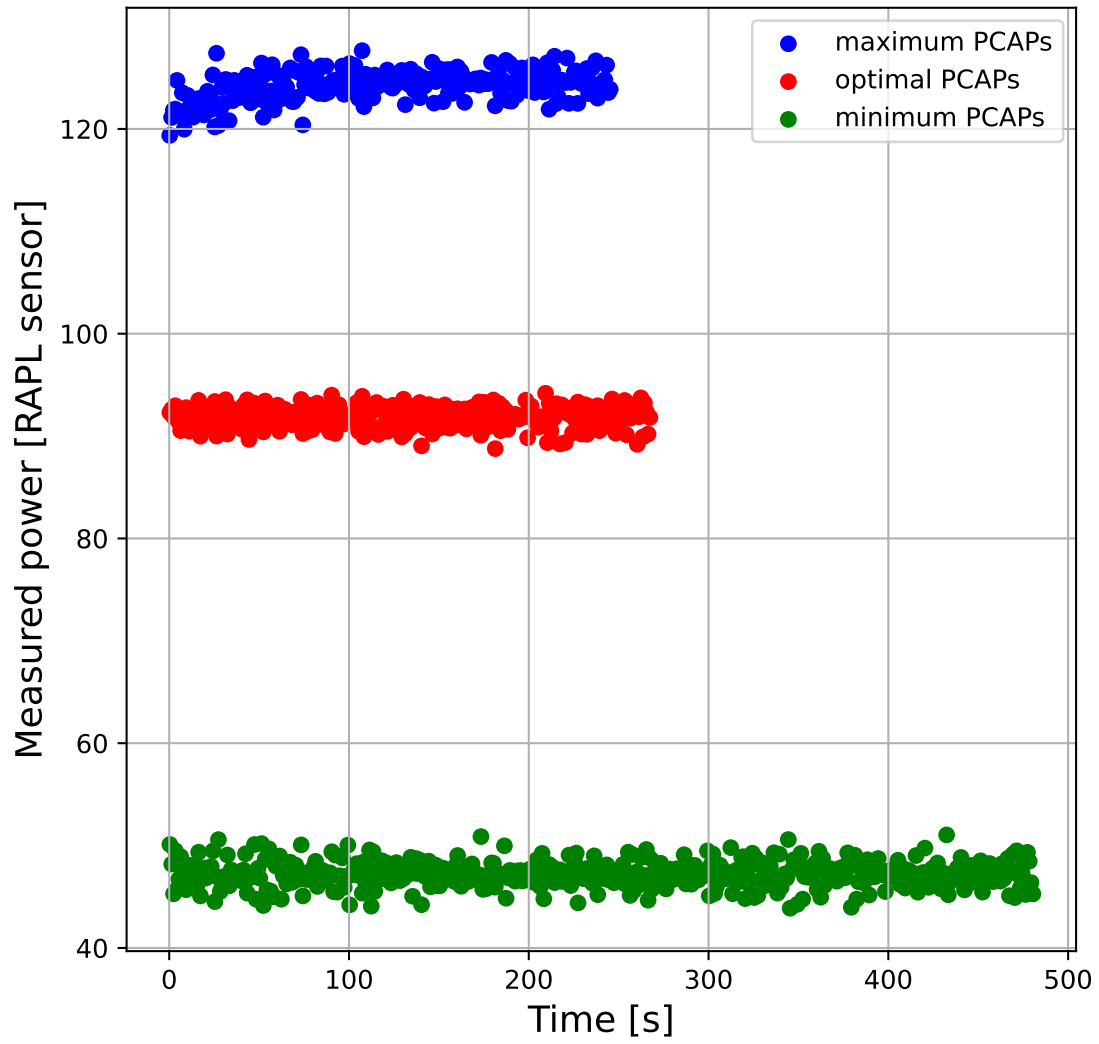


Figure 4.7: Instantaneous power vs time-steps plots for a single execution of the experiment using optimal, maximum and minimum PCAP controllers.

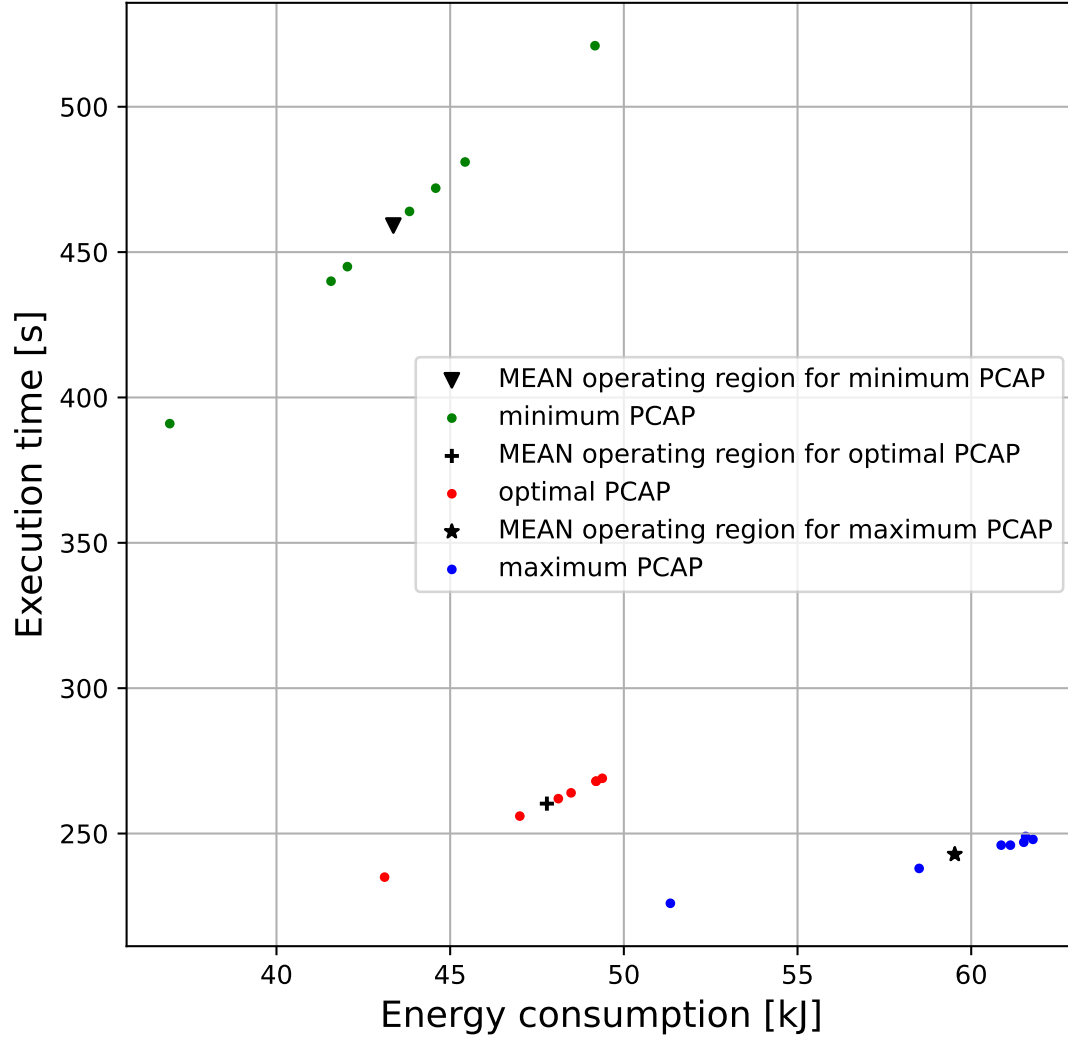


Figure 4.8: Comparison of the execution-time and the energy consumed during the execution of experiments, performed on different nodes using the three methods. The statistics of the executions are shown in the table 4.3

CHAPTER 5

Offline Reinforcement-Learning-Based Power Control for Application-Agnostic Energy Efficiency

Abstract

As the compute intensity of modern AI/ML and high performance computing (HPC) workloads grows, so does their energy consumption, making energy efficiency a major sustainability challenge. Although substantial prior research has dealt with the design of more energy-efficient hardware or more efficient algorithms, not much work exists on improving the energy efficiency of already deployed applications. In this chapter, we present an offline reinforcement learning (RL) approach for application-agnostic power control of HPC compute nodes. The controller is trained from pre-collected execution traces, capturing hardware performance counter data alongside power consumption measurements, and is deployed without retraining on diverse test workloads. To the best of our knowledge, this is the first work to apply offline RL for dynamic power regulation in HPC/data center environments. We demonstrate significant energy savings while maintaining application performance across a range of memory-bandwidth-bound and compute-bound benchmarks.

5.1 Introduction

As the compute intensity of modern AI/ML and high performance computing (HPC) workloads grows, so does their energy consumption, thus making energy efficiency a major sustainability challenge. Although substantial prior research has dealt with the design of more energy-efficient hardware or more efficient algorithms, not much work exists on improving the energy efficiency of already deployed applications. Achieving energy efficiency, then, is a matter of identifying opportunities for lowering energy consumption (e.g., via power capping) without negatively impacting the desired performance of already deployed compute intensive workloads.

A fundamental requirement for energy efficiency optimization is identifying resource imbalance within a system and focusing on reducing the power consumption of those resources that have minimal impact on application performance. However, identifying these resources is a hard problem because it requires an approach that is grounded in understanding both the application performance and the impact of hardware power management strategies on that performance.

Figure 5.1 illustrates this relationship for a memory-bound application, showing instantaneous performance (measured via instrumentation) and corresponding power consumption under varying processor power caps. A closer examination of the application’s runtime behavior reveals that performance does not scale linearly with increasing power caps. Instead, there exist operating regions in which similar performance levels

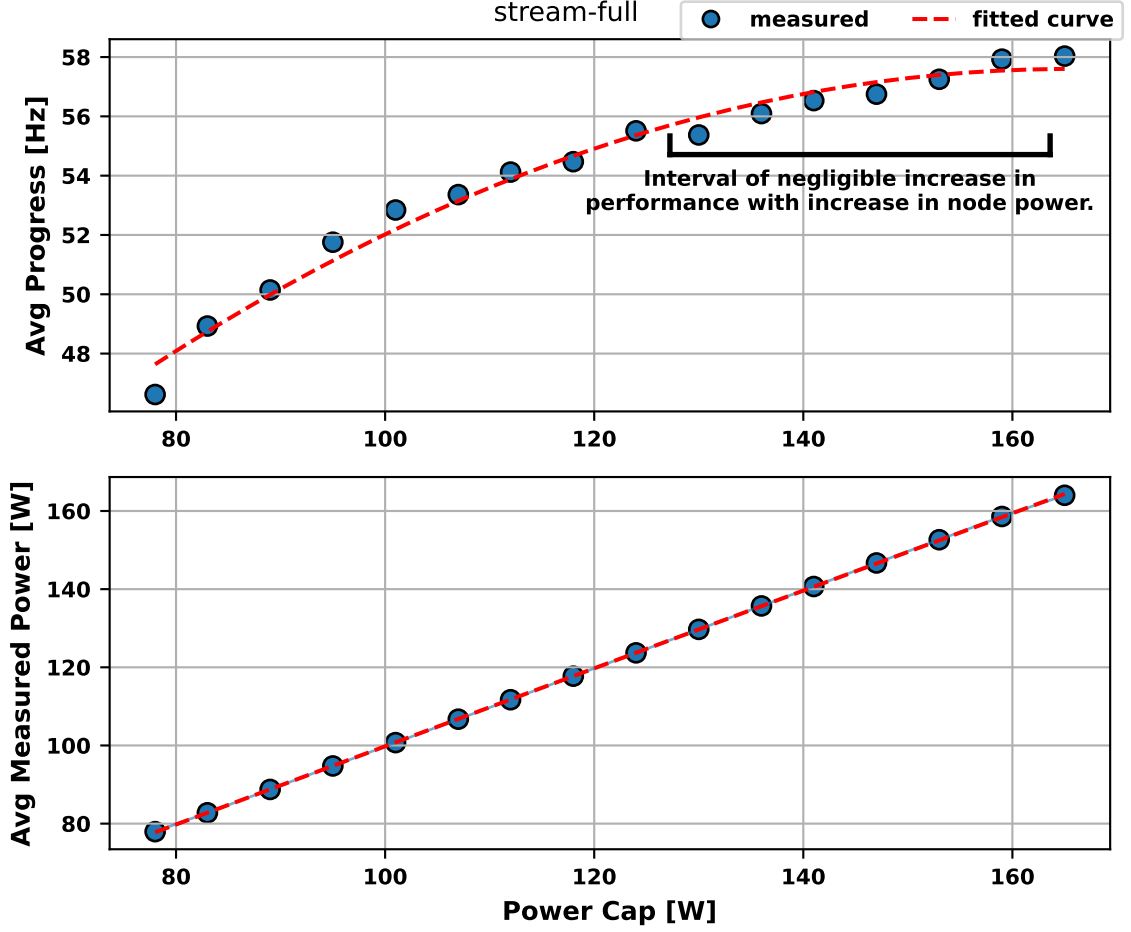


Figure 5.1: Power vs instantaneous performance (progress) for the `stream-full` benchmark on an Intel Cascadelake node. Progress, defined in Eq. (6.6), is a proxy for the instantaneous performance of an application toward its scientific objective.

are achieved at substantially lower power. These regions represent opportunities for energy optimization, which motivate the need for adaptive power and performance control strategies.

Modern processors with energy optimization capabilities such as dynamic voltage and frequency scaling (DVFS) Kim et al. (2008), Intel’s Running Average Power Limit (RAPL) David et al. (2010) and AMD’s thermal design power cap Roberts et al. (2019) have enabled application-oriented power optimization. Advanced power control strategies using these technologies range from classical control theory to approaches based on machine learning (ML) Czarnul et al. (2019). These methods can regulate power at a constant value, which is chosen according to hardware and application characteristics. Methods such as Dynamic Energy-Performance Optimizer *DEPO* Krzywaniak et al. (2023) can even dynamically adjust power by observing changes in application performance. Such approaches are, however, either hardware-centric or application-specific: they require recomputing all the parameters whenever the application or hardware changes. To avoid

this, we require a solution that is hardware- and application-agnostic. Therefore, in this paper, we design a power-capping controller that operates without application-specific tuning.

Reinforcement Learning (RL), which is an online machine learning method, is generally used for control applications, however, like any other ML algorithm, is computationally expensive to train. Therefore, researchers use surrogate models or digital twins of their environments Raj et al. (2023b) to train RL-agents when interactive learning is not possible. In such cases, the accuracy of the attained control objective depends on the accuracy of the surrogate models used for learning. Unfortunately, in data center or HPC compute clusters, where application and hardware power-to-performance relationship changes across application and hardware architectures, generating a black-box model Witt et al. (2019) using available datasets is difficult.

Consequently, we propose an *off-the-node training method* for the RL agent using non-interactive or offline learning Levine et al. (2020a), with the data generated from existing arbitrary policies. Here, the training is offline in nature, i.e., the actual compute node is never queried for evaluating an action during the training phase. Thus, offline RL requires a pre-collected dataset of state transitions, actions and rewards, using which the agent can be trained elsewhere. By training the RL model offline, we reduce the computational burden on the actual compute node and eliminate the need for surrogate models to train the agent.

We evaluate our approach by having it regulate the node-level power cap (PCAP) across multiple compute nodes using 12 benchmark applications. We compare it against several state-of-the-art power management systems Huang et al. (2020a); Eastep et al. (2017a); Cerf et al. (2021b). Experimental results show that our scheme outperforms existing methods by reducing average energy consumption by 20% with an average performance loss of only 7.4% and a worst-case performance degradation of 14%. When compared with the manufacturer-provided *on-demand* frequency governor—which dynamically adjusts core frequencies—our method demonstrates superior performance in reducing energy consumption while maintaining performance.

Note that although this work illustrates control design for multisoocket CPU-only systems, it is applicable to other resources, and GPUs in particular will be the focus of future work. Further, we posit that a single-node controller can then be integrated into a full-system solution.

The remainder of this paper is structured as follows: In Section 6.7 we provide background material and review existing algorithms in power and performance optimization. Section 6.3 formulates the problem. Section 5.4 outlines the proposed algorithm and workflow in the context of compute nodes. Section 5.5 presents the experimental setup used in both training and evaluations. In Section 5.6 we present the experimental design and implementation, along with the results and analysis. Finally, Section 5.8 provides a brief summary, the impact of this work, and future opportunities.

5.2 Background

This section offers an overview of the concepts and technologies employed in this paper.

5.2.1 Offline Reinforcement Learning

Reinforcement learning (RL) formulates control as a sequential decision-making problem, typically modeled as a Markov decision process (MDP) $\mathcal{M} := (\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathbf{T}, \gamma)$, where an agent observes the system state $s \in \mathcal{S}$, selects an action $a \in \mathcal{A}$, and receives a reward $r \in \mathcal{R}$. The objective is to learn a policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$ that maximizes the expected discounted return.

In an offline RL setting such as the Q-learning approach used in this paper, unobserved state–action pairs cannot be evaluated during training. Instead, learning relies on the ability of the Q-network to approximate a value function from the available dataset. A common challenge in this setting is distributional shift, which arises when the distribution of states and actions encountered during training differs from that observed at deployment, potentially degrading performance. Another limitation of offline RL is suboptimal learning when trained solely from random policies.

To address these challenges, we employ Conservative Q-Learning (CQL) Levine et al. (2020a), which regularizes Q-value estimation to mitigate overestimation under distributional shift. By carefully selecting learning parameters, we train an offline CQL agent from a previously collected dataset without overfitting. Moreover, incorporating domain knowledge of the controlled system enables the dataset to be populated with informative (greedy) behaviors, improving the quality of the learned policy.

5.2.2 Measuring Application Performance through Hardware Performance Counters

Training the offline models as suggested above requires a dataset of application performance on different hardware. However, different applications can illustrate differing performance on a given hardware architecture. This performance is influenced by hardware characteristics such as memory bandwidth, CPU cores, and cache size. To capture this complex application-hardware interaction in the dataset, we leverage a set of hardware performance counters while running applications on the target system. Specifically, we rely on the Performance Application Programming Interface (PAPI) Mucci et al. (1999) to provide portable access to hardware performance counters. PAPI offers easy access to a variety of counters through direct queries on the hardware model-specific registers. These measurements are representative of hardware-specific and application-specific data movement, or a combination of both.

To optimize energy efficiency while preserving performance, we focus on a small set of key performance metrics: total instructions issued (*TOT_INS*), total clock cycles (*TOT_CYC*), total last-level cache accesses (*L3_TCA*), last-level cache misses (*L3_TCM*), and cycles stalled because of resource contention (*RES_STL*).

We chose these key metrics for two reasons. First, these readings are available instantaneously and simultaneously during every sampling interval. Second, they are direct and sufficient indicators of the rate of data flow within and between the CPU cores and memory. Furthermore, because direct usage of these metrics would call for a large trained network and necessitate additional data processing, we follow insights from other research Wittmann et al. (2016); Rane and Browne (2014) in computing a set of *derived metrics*—instructions per cycle (*IPC*), the ratio of stalled cycles to total cycles (often equated with CPU utilization, noted as *STL*), and last-level cache miss rate (*CMR*)—which facilitate a more detailed analysis of runtime behavior.

Key Insight: Based on findings from previous works, we note that these metrics should not be viewed in isolation as definitive indicators of performance, since varying IPC values, for example, can still lead to the same execution time. Nonetheless, these derived metrics serve as valuable proxies to discern application characteristics, including its arithmetic intensity or whether the application is memory-bounded and thus can operate efficiently within a lower power budget.

5.2.3 Capturing Application Progress in the Dataset

We augment our monitoring of application behavior by adding a notion of *progress*, or *heartbeats*, to our system. This measure is obtained by a lightweight instrumentation of the applications under study. Our approach follows the work of Ramesh et al. Ramesh et al. (2019b) and follows the instrumentation steps outlined by Cerf et al. Cerf et al. (2021b). The role of this progress measurement is to highlight the potential for an application to be busy-waiting for some resource in the system and thus be active but not making useful use of compute power.

We redefine the progress measurement equation introduced in Ramesh et al. (2019b) by considering the number of heartbeats received between two sampling (control) instants. The progress at time instant t_i is given by Equation (6.6):

$$progress(t_i) = \underset{\forall k, t_k \in [t_{i-1}, t_i]}{\text{median}} \left(\frac{N}{t_k - t_{k-1}} \right), \quad (5.1)$$

where N represents the number of heartbeats received between the time instants t_k and t_{k-1} . This measure of progress can be understood as a proxy for monitoring the figure of merit of an application at runtime. Equation (6.6) enables us to accurately measure progress while considering the time interval and the number of heartbeats received. It also allows us to limit the heartbeat-rate of instrumentation to avoid overflowing our infrastructure with messages, since it is sufficient to accumulate a heartbeat counter on the application side and to only periodically send this counter to our data collection and control infrastructure.

5.3 Problem Statement

Given an application and a compute node of a datacenter or HPC cluster, the goal is to reduce energy consumption during the application’s execution while allowing little to no degradation in performance. We formulate this problem as a multiobjective optimization problem (balancing energy and performance) that we linearize by focusing on minimizing the energy delay squared product (ED^2P). As noted by Schöne et al. (2014), ED^2P is widely used to measure performance degradation relative to the energy saved. Compared with just the energy-delay product (EDP) where equal importance is given to the optimization of energy and execution time, ED^2P puts extra emphasis on maximizing the performance against minimization of energy. The formula is $ED^2P = E \times ET^2$, where ET represents execution time and E represents energy. Note that both ET and E are available only at the end of the application execution.

Referring to Figure 5.1, we define the term performance per watt (PPW) as a measure of useful work done per unit power. For example, the PPW corresponding to any operating point shown in Figure 5.1 is the square of the progress value per the measured power. Using a progress-based reparameterization of time and applying Cauchy-Schwarz inequality, the non separable objective $E \times ET^2$ can be upper-bounded (up to a constant factor) by an additive surrogate of the form $\int \frac{P_{avg}(t)}{progress^2(t)} dt$. Consequently, minimizing ED^2P is equivalent to minimizing this surrogate, which admits an online decomposition and corresponds to maximizing instantaneous PPW .

5.4 Methodology

This section describes the methods, beginning with data collection, followed by the training of the RL-agent and the deployment of the algorithm. To the best of our knowledge, this paper is the first to use an offline RL to train an agent to dynamically regulate the power of a datacenter/HPC cluster node. The different phases of the approach are depicted pictorially in Figure 6.2. As shown in the figure, there are no closed loops between the RL agent and the evaluation process, meaning the agent is trained without feedback from direct interaction with the target system.

5.4.1 Dataset Collection: Capturing Application Performance through Hardware Performance Counters

Unlike existing works Raj et al. (2023b) that use a mathematical model to describe the relationship between application performance and power for training an RL agent, our approach learns directly from a collected dataset, which consists of performance and power data. By analyzing a set of pre-collected data, the RL agent aims to improve decision-making processes and optimize performance over the course of training iterations.

Dataset collection itself is a challenging problem because different applications can illustrate differing

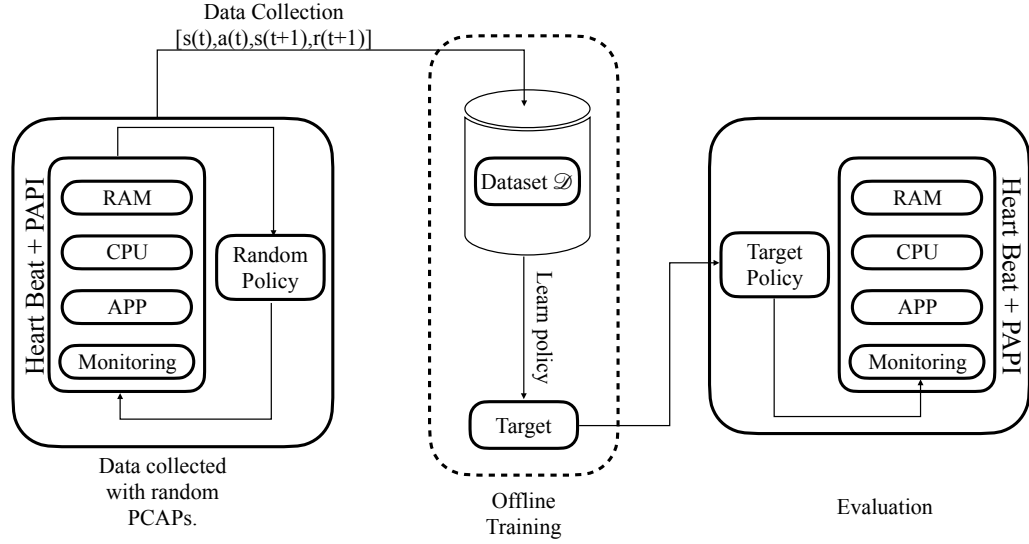


Figure 5.2: Flow diagram for offline reinforcement learning: data is collected using an arbitrary policy to train the agent without system , and the trained agent is later deployed for evaluation. Note that the lines in the diagram are not closed to form a loop, indicating offline training and online testing.

performance on a given hardware architecture. This performance is influenced by hardware characteristics such as memory bandwidth, CPU cores, and cache size. To capture this complex application-hardware interaction in our dataset, we leverage a set of hardware performance counters while running applications on the target system. Specifically, we rely on the Performance Application Programming Interface (PAPI) Mucci et al. (1999) to provide portable access to hardware performance counters. PAPI offers easy access to a variety of counters through direct queries on the hardware model-specific registers. These measurements are representative of hardware-specific and application-specific data movement, or a combination of both.

To optimize energy efficiency while preserving performance, we focus on a small set of key performance metrics: total instructions issued (TOT_INS), total clock cycles (TOT_CYC), total last-level cache accesses ($L3_TCA$), last-level cache misses ($L3_TCM$), and cycles stalled because of resource contention (RES_STL). We chose these key metrics for two reasons. First, these readings are available instantaneously and simultaneously during every sampling interval. Second, they are direct and sufficient indicators of the rate of data flow within and between the CPU cores and memory. Furthermore, because direct usage of these metrics would call for a large trained network and necessitate additional data processing, we follow insights from other research Wittmann et al. (2016); Rane and Browne (2014) in computing a set of *derived metrics*—instructions per cycle (IPC), the ratio of stalled cycles to total cycles (often equated with CPU utilization, noted as STL), and last-level cache miss rate (CMR)—which facilitate a more detailed analysis of runtime behavior.

Key Insight: Based on findings from previous works, we note that these metrics should not be viewed in isolation as definitive indicators of performance, since varying IPC values, for example, can still lead to

the same execution time. Nonetheless, these derived metrics serve as valuable proxies to discern application characteristics, including its arithmetic intensity or whether the application is memory-bounded and thus can operate efficiently within a lower power budget.

5.4.2 Augmenting the Dataset with Application Progress

To train the RL model, we must define the observables that constitute the states. In our case, we represent one aspect of the state of the RL agent related to application behavior, which is identified through hardware performance counters using the data collection steps shown above. Additionally, the instantaneous performance of the application and useful work it does, measured through application instrumentation, and the ongoing power consumption are also integral to the state. These characteristics can vary across applications.

We augment our monitoring of application behavior by adding a notion of application *progress*, or *heartbeats*, to our system. This measure is obtained by a lightweight instrumentation of the applications under study. Our approach follows the work of Ramesh et al. Ramesh et al. (2019b) and follows the instrumentation steps outlined by Cerf et al. Cerf et al. (2021b). The role of this progress measurement is to highlight the potential for an application to be busy-waiting for some resource in the system and thus be showing up as being active but in reality is not making useful use of compute power.

We redefine the progress measurement equation introduced in Ramesh et al. (2019b) by considering the number of heartbeats received between two sampling (control) instants. The progress at time instant t_i is given by Equation (6.6):

$$progress(t_i) = \underset{\forall k, t_k \in [t_{i-1}, t_i]}{\text{median}} \left(\frac{N}{t_k - t_{k-1}} \right), \quad (5.2)$$

where N represents the number of heartbeats received between the time instants t_k and t_{k-1} . This measure of progress can be understood as a proxy for monitoring the figure of merit of an application at runtime. Equation (6.6) enables us to accurately measure progress while considering the time interval and the number of heartbeats received. It also allows us to limit the heartbeat-rate of instrumentation to avoid overflowing our infrastructure with messages, since it is sufficient to accumulate a heartbeat counter on the application side and to only periodically send this counter to our data collection and control infrastructure.

5.4.3 Training the RL Agent using Offline Collected Dataset

The resulting collected dataset consists of quadruples of state, action, next state, and reward, corresponding to different benchmarks (training/testing).

Our objective, as defined in Section 6.3, is to identify the maximum efficiency point (max *PPW*) of operation for the application on a compute node, which will be represented by a suitable reward function. The

reward function in RL helps the agent learn a policy that aligns with the optimization goal. In our case, the reward should be a function of application progress and power consumption, which together constitute *PPW* (performance per watt). The reward function $r(t)$ is thus defined as

$$reward(t+1) = \frac{progress^3(t+1)}{P_{avg}(t+1) + 1e^{-3}}, \quad (5.3)$$

where the reward is an explicit function of $progress(t+1)$ and $P_{avg}(t+1)$ representing the progress and measured power, respectively. A small constant (0.001) is added to the denominator to avoid division by 0. The rest of the notations follow the earlier definitions. The overall structure of the reward function is designed to maximize the reward for improved progress while penalizing high actual power consumption. The $progress(t+1)$ term cubed in the numerator shows that more progress leads to a higher reward. The squared $PCAP(t)$ in the denominator suggests that the reward decrease is more than proportional to the increase in input power; thus, using significantly more input power than necessary reduces the reward substantially.

We note that in CQL the reward is part of the dataset and can be calculated before training, since it is a function of the next state and measured power. The state variables include characteristics that provide insights into potential bottlenecks occurring during the application runtime. The reward function determines the efficiency or quality of *PCAP* deployed to achieve the measured state.

Consistent with the offline RL setting described in Section III, we populate the acquired training dataset with random action samples spanning the admissible *PCAP* range and train the agent using CQL to mitigate distributional shift. The RL agent is implemented using a single Q-network, where the input dimension corresponds to the state space and the output dimension corresponds to the cardinality of the action space. During training, random mini-batches are sampled from the dataset, and the Q-network parameters are updated to learn the action policy that maximizes the expected reward. Training proceeds until the loss function of CQL converges.

5.4.4 RL Model Evaluation

We evaluate the trained model by running it alongside test applications that the training phase has not encountered before, as well as some applications with phases (to evaluate adaptability). During evaluation, the Q-network computes the action Q -values corresponding to the observed state provided by the daemon process and takes the greedy action, that is, $PCAP = \arg \max_a (Q(s, a))$. This *PCAP* is deployed by using the *RAPL* actuators and is maintained at a steady throughput throughout the sampling interval. The sampling interval plays an important role in determining the adaptability of the controller during actuation. In our experiments we have intentionally kept the sampling interval smaller to detect minute changes in performance.

5.5 Applying the Methodology to a Use Case

We now demonstrate how we have applied our approach to a concrete use case of high performance computing applications requiring effective power-performance trade-offs.

5.5.1 Experimental Hardware and Tools Used

All the experiments in this paper, including data collection and evaluation, were performed on a bare-metal Cascadelake node from Chameleon Cloud Keahey et al. (2019), equipped with Intel Xeon Gold 6240R processors. The node has 2 processors, each with 24 cores and 2 hyperthreads per core, and 192 GiB of memory. The L3 cache size is 35.75 MiB per processor.

Provisioning of these nodes included a fully installed and configured Global Extensible Open Power Manager (GEOPM) Eastep et al. (2017b) service (version 3.1.0) to enable reading and writing of power sensors, but without GEOPM being configured to provide any power control policy itself. PAPI version 6.0.0 was used to access hardware performance counters. A custom software infrastructure handles the collection of data from GEOPM, PAPI, and application heartbeats, while also providing the basis for our control policy deployment.

5.5.2 Compute-intensive Workloads used in Offline RL Model Training

We use a collection of well-established application benchmarks to represent the behaviors of various applications and their response to power capping. Table 6.1 presents the characteristics of various benchmarks used throughout this paper, including problem size, iterations, average *IPC*, average *CMR*, average *STL*, average progress, and average execution time (*ET*) for each kernel, calculated over 10 independent runs. The benchmarks listed here are used both for training and for validating our control agent, with some of them being kept out of our training dataset and used only for validation. By using these well-established benchmarks, we hope to train our model to recognize and predict optimal power management strategies that will be applicable to a wide range of application behaviors.

Table 5.1: Detailed description of our benchmarks, including their type, parameters, and average performance metrics (uncapped).

Benchmark	Prob Size	Iter.	Avg. IPC	Avg. CMR	Avg. STL	Avg. Prog. [Hz]	Avg. ET[s]	Avg. Energy [kJ]	Ari. Int. (FLOPs/Byte)
STREAM SCALE	33554432	10000	0.20	0.89	0.84	285.20	34.40	5.59	1.62
STREAM TRIAD	33554432	10000	0.18	0.94	0.83	200.03	50.80	7.93	1.41
NPB-EP	Class-W	1000	0.57	0.13	0.48	13.61	73.80	10.33	53.99
NPB-FT	Class-B	500	0.82	0.43	0.296	13.23	35.10	45.47	1.53
STREAM-ADD	33554432	10000	0.15	0.94	0.85	201.06	50.40	7.84	1.36
STREAM-COPY	33554432	10000	0.17	0.89	0.84	282.43	35.50	5.66	1.63
STREAM-FULL	33554432	10000	0.16	0.93	0.70	49.31	222.40	34.99	1.45
NPB-BT	Class-C	1000	0.97	0.87	0.30	6.22	166.00	23.168	0.54
STREAM-PHASE	33554432	10000	0.17	0.91	0.88	240.16	83.00	13.48	1.45
NPB-CG	Class-C	1000	0.49	0.37	0.62	9.97	114.00	14.74	3.61
NPB-IS	Class-B	1000	0.50	0.86	0.68	8.44	117.33	18.99	1.21
NPB-MG	Class-B	1000	0.47	0.809	0.54	40.3	22.23	55.5	1.45

STREAM McCalpin (1995) is a well-known benchmark designed to evaluate the memory bandwidth of a system. We consider six versions of this benchmark, corresponding to its four inner kernels (add, copy, scale, and triad) with varying arithmetic intensity, as well as one version (`full`) that executes all four kernels as part of its outer loop. The last version, with phases, repeats the full benchmark a number of times, resulting in changes in progress rate and behavior across its execution. For all these versions, the input problem size (the size of the working arrays in memory) has a direct impact on the performance and is chosen here so that these arrays do not fit in the last level cache on the system under study.

NAS Parallel Benchmarks (NPB) are a collection of programs created to benchmark the performance of parallel processors using kernels that are representative of scientific applications. We include six of the benchmarks here: Embarrassingly Parallel (EP) for being a representative of a compute-bound application, Integer Sort (IS) for its focus on random memory accesses, Fourier Transform (FT) for its straining of the cache for memory access, Multi-Grid (MG) for its computational intensity, Conjugate Gradient (CG) for memory-intensive and irregular communication patterns, and Block Tridiagonal (BT) for its compute intensive and irregular communication. Our implementation is based on the NPB-3.4-OpenMP version.

We modified these benchmarks to be iterative so that application progress is more easily identifiable: When not already a part of the benchmark, an outer loop is introduced, repeating their kernels a configurable amount of time. Each outer loop also contains our instrumentation for heartbeats, resulting in each benchmark reporting one heartbeat per iteration. Note that the progress rate of each benchmark is different and depends on the runtime of its kernel.

Since this paper is concerned with demonstrating energy efficiency, we select each benchmark problem size to be bigger than caches, with the goal of exhibiting performance trade-offs between CPU power capping (and the resulting frequency variation) and stable memory performance. These problem size values are provided in Table 6.1. The table also provides information on the arithmetic intensity for each application, defined as the ratio between total computational operations and total instructions, which serves as a measure of computational diversity among the applications. A double line in the table separates applications that are part of the training dataset (top) and those used only during validation (bottom).

5.5.3 Data Collection and Offline Training

We collect our training dataset by executing each benchmark under a structured stochastic control policy and sampling all measurements that constitute the state vector during execution. Prior work in offline reinforcement learning suggests that randomly generated datasets can be sufficient to recover near-optimal policies Levine et al. (2020a). However, uniformly random sampling over the action space can lead to sub-optimal learned policies.

To address this, we employ a weighted sampling strategy based on a Gaussian-shaped random walk over the discrete *PCAP* action space rather than uniform sampling. At fixed control intervals, the next power cap is sampled from a Gaussian distribution centered at the constant action index, thereby inducing smooth transitions between neighboring power levels. The smoothness parameter (σ) controls the locality of exploration. For memory-bound applications such as *stream*, the operating point was selected at 89.0 W with $\sigma = 2$, since empirical profiling shows that application progress begins to saturate near this power cap.

To ensure that the training dataset fits the conditions under which the trained controller will operate, we set up the experiments to sample at the same rate as the control will activate: 1 Hz (once per second). In this paper we use a random control policy during data collection: *PCAP* values for RAPL, chosen randomly from a discretization of the action space of allowed power caps for the system. Indeed, while the system allows for a power capping range of 78 W to 165 W, we apply only 16 values distributed uniformly in that range. By reducing the action state to such a small and finite set of actions, we avoid the need for a much larger Q-Network (in the output dimension) to train. This choice also significantly reduces the size of the training dataset while preserving the controller’s performance. We ensure in our data collection that all 16 possible power caps are sampled, since CQL, like most RL algorithms, does not extrapolate on the action space observed. Nevertheless, not all transitions between two actions corresponding to all state observations are present in the dataset.

This collection strategy is repeated for each of the benchmarks selected as part of the training set for the problem size and iterations given in Table 6.1. To ensure an independent and identically distributed (i.i.d.) dataset across applications, we execute applications with lower average execution times more frequently than others. Rewards are computed from these state vectors and appended to the dataset and normalized as explained in Section 5.4.

In our case the state variable at time t , $s(t)$ consists of the progress of the system, the measured power (the measure of how much of the supplied power is being utilized) and the characteristics of the application.

$$s(t) = [\text{progress}(t), P_{\text{avg}}(t), \text{IPC}(t), \text{STL}(t), \text{CMR}(t)].$$

The action is the CPU power supplied using the RAPL actuators, that is, $a(t) = \text{PCAP}(t)$. Upon observing $s(t)$ the system transitions to $s(t+1)$ under the action $a(t)$ following the transition dynamics under the policy.

The resulting dataset consists of filtered data points $(s(t), a(t), r(t+1), s(t+1))$, on which offline training is performed. Since the action space is finite, we use a Q-network consisting of output dimensions equivalent to the cardinality of the action space (i.e., 16). The input dimension is the dimension of the state space or observation space, which is 5. The 2 intermediate layers consist of 20 neurons each. A total of 10,000

iterations, with a discount factor $\gamma = 0.9$ and the CQL constant $\alpha = 0.1$, was used for training, with a buffer size of 128. It is recommended to keep the α at larger values (0.3) for larger datasets to compensate for out-of-distribution observations. The number of iterations is chosen based on the time it takes for the Q-network to converge.

5.6 Empirical Evaluations

We now evaluate the proposed controller by deploying it to regulate the node’s power while executing both training and testing applications from our use case. Later, we also compare our method with state-of-the-art controllers.

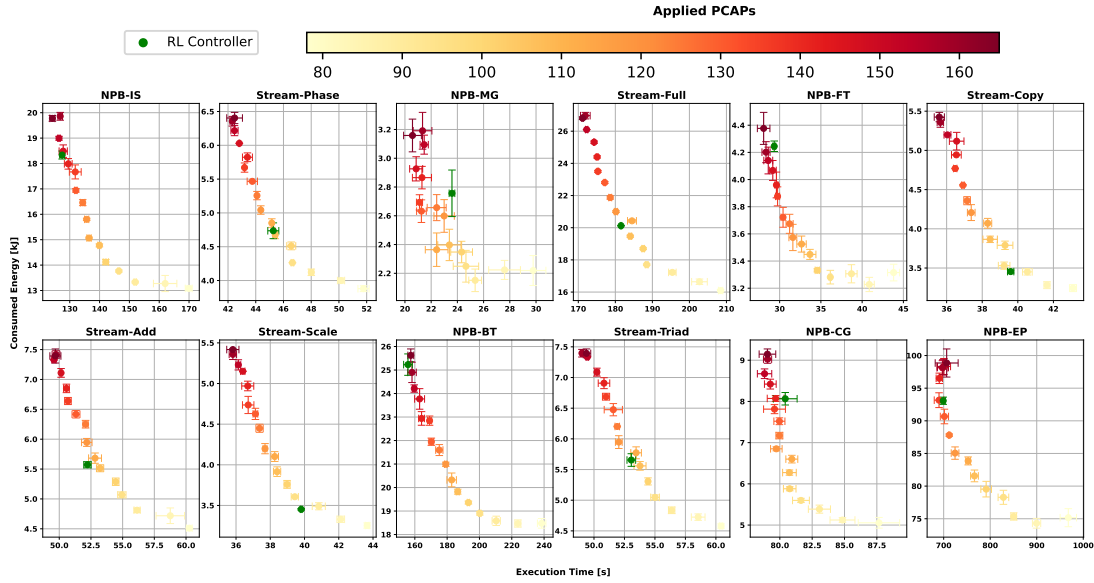


Figure 5.3: Execution time vs. energy for twelve benchmarks. Six applications were unseen during training (Table 5.2). Yellow-red gradients show fixed $PCAP$ values; the green dot shows the proposed controller. Each point is averaged over five runs.

5.6.1 Evaluation of the Proposed Framework

We assess the execution time and energy consumption in comparison with the performance of an uncapped system. In all the testing scenarios consisting of different applications, the agent evaluates states that it has never encountered during training. More out-of-distribution states are likely to appear when the proposed controller is used to regulate the power of testing applications.

For testing, we deploy our infrastructure, the trained agent, and the application together on the node of interest. The infrastructure is designed to collect the measurements (*progress*, *power*, and *PAPI*) and to provide them to the controller, which, upon postprocessing and observing the required states, makes an

informed decision. The resulting action is then translated into RAPL control values and applied on the system through GEOPM.

We first test the controller on the applications used for training, namely, the STREAM TRIAD, STREAM SCALE, NPB-IS, and NPB-EP benchmarks, which are selected to provide a large range of memory boundedness behavior. We then collect the runtime metrics and calculate the execution time and total energy consumed during the runtime for each experiment. Furthermore, we assess the performance and energy consumption of the controller with applications it has never encountered during training. For this analysis, we use the testing applications listed in Table 6.1. For comparison, we generate a Pareto curve of energy versus execution time for all the applications used in both training and testing, as shown in Figure 5.3 using various static power caps.

Table 5.2: Execution time and energy across benchmarks for Min *PCAP*, Max *PCAP*, and the proposed RL agent (mean $\pm$ std. over ten runs). RL results including energy consumed and execution time are reported relative to Max *PCAP*.

Benchmark	Min <i>PCAP</i>				Max <i>PCAP</i>				RL					
	Execution Time		Energy		Execution Time		Energy		Execution Time		Energy		ET Degradation (%)	Energy Saved (%)
	μ [s]	σ	μ [kJ]	σ	μ [s]	σ	μ [kJ]	σ	μ [s]	σ	μ [kJ]	σ	(%)	(%)
STREAM SCALE	37.73	2.61	4.17	0.61	34.33	0.47	5.65	0.08	37.33	0.47	3.81	0.05	8.74	32.61
STREAM TRIAD	52.86	3.26	5.83	0.88	47.67	0.47	7.67	0.07	53.67	0.47	5.15	0.05	12.59	32.90
NPB-EP	78.74	8.76	8.54	0.84	67.67	0.47	9.80	0.06	68.67	0.47	9.22	0.06	1.48	5.92
NPB-FT	37.52	5.81	4.30	0.35	30.60	0.49	5.05	0.08	31.33	0.47	4.96	0.08	2.39	1.78
STREAM ADD	53.69	3.88	5.92	0.86	48.33	0.47	7.96	0.08	55.00	0.82	5.27	0.08	13.80	33.74
STREAM COPY	38.09	2.92	4.20	0.60	34.67	0.47	5.72	0.08	37.67	0.47	3.84	0.05	8.66	32.91
STREAM FULL	189.77	12.24	20.86	3.13	172.33	0.47	28.27	0.07	183.00	2.52	20.50	1.22	6.20	27.47
NPB-BT	169.00	24.89	19.43	1.68	140.00	1.67	22.99	0.25	149.00	1.41	21.07	0.33	6.43	8.34
STREAM PHASES	93.31	6.26	10.22	1.52	84.33	0.47	13.83	0.07	94.00	0.00	9.22	0.01	11.46	33.34
NPB-CG	109.96	15.83	13.79	2.04	101.00	5.97	15.08	0.93	100.17	2.91	14.87	0.43	-0.82	1.37
NPB-IS	134.85	11.89	14.67	1.90	116.67	1.25	19.17	0.22	120.67	0.94	16.88	0.14	3.43	11.94
NPB-MG	22.75	2.60	2.60	0.34	19.80	0.40	3.18	0.09	22.67	0.94	2.49	0.09	14.47	21.70

The comparison shows that the RL controller (green dot) consistently regulates the node’s operating region between the maximum and minimum execution extremes, particularly for memory-bound applications. For compute-bound applications, the green dot typically appears on the darker end of the color bar, indicating operation under higher *PCAP* settings. Notably, the green dot always lies on or outside the Pareto curve, demonstrating that the RL controller achieves improved or comparable performance under static workloads. This plot also highlights that the RL controller does not introduce any noticeable computational overhead on the node.

Table 5.2 presents statistics related to the repeatability of our experiments. Together with Figure 5.3, it demonstrates that the proposed controller achieves significantly greater energy savings than any corresponding loss in performance. On average, the controller saves 20.2% energy, with only a 7.5% performance decay when compared with uncapped execution.

Some exceptions can be observed in the *NPB – MG* and *NPB – IS* cases, which are not controlled as

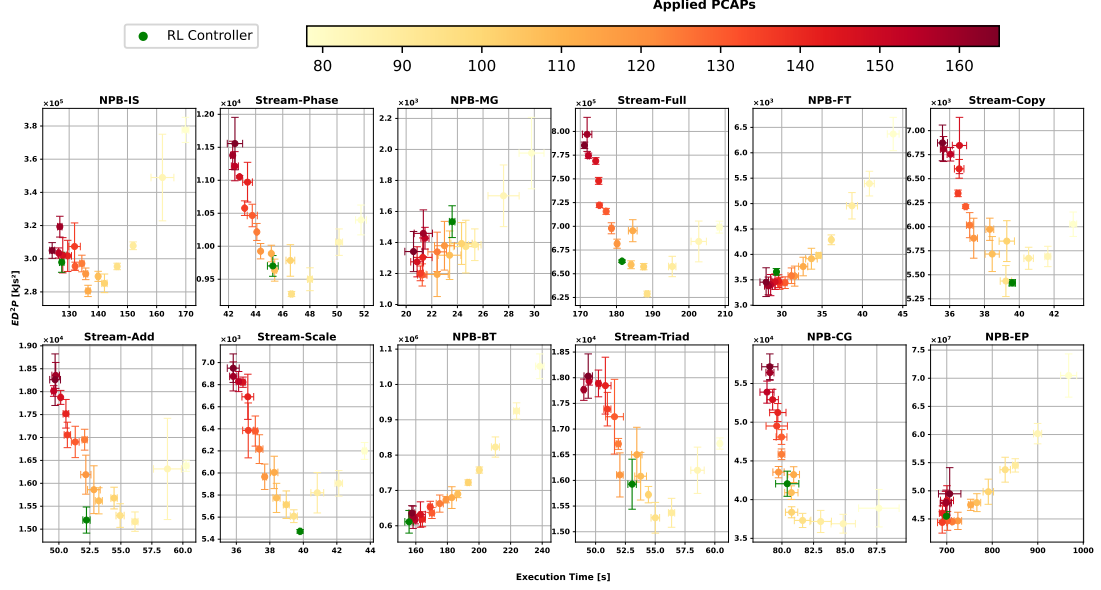


Figure 5.4: Mean and standard deviation of ED^2P vs. execution time for the experiments in Fig. 5.3. The proposed controller consistently maintains lower average ED^2P across applications.

Benchmark	DVFS Controller			PI-Global Model			PI-App-Specific Model			Ondemand Governor			RL (Ours)		
	Energy Saved (%)	Perf Degrad. (%)	ED^2P	Energy Saved (%)	Perf Degrad. (%)	ED^2P	Energy Saved (%)	Perf Degrad. (%)	ED^2P	Energy Saved (%)	Perf Degrad. (%)	ED^2P	Energy Saved (%)	Perf Degrad. (%)	ED^2P
NPB-BT	-13.50	13.90	6.63×10^5	-98.43	-0.21	8.90×10^5	-5.12	7.14	5.44×10^5	-13.50	13.90	6.63×10^5	8.34	6.43	4.68×10^5
NPB-CG	-7.74	-1.44	1.61×10^5	-13.56	9.57	2.10×10^5	2.23	0.99	1.53×10^5	-11.47	11.17	2.12×10^5	1.37	-0.83	1.49×10^5
NPB-EP	-0.80	-1.56	4.38×10^4	-124.00	16.87	1.37×10^5	6.33	4.43	4.58×10^4	-9.97	15.63	6.60×10^4	5.93	1.47	4.35×10^4
NPB-FT	-5.70	36.42	9.30×10^4	-123.59	16.31	1.43×10^5	0.02	13.07	6.04×10^4	10.51	4.65	4.63×10^4	1.78	2.40	4.87×10^4
NPB-IS	-8.17	23.42	4.30×10^4	-127.69	17.41	8.19×10^4	10.45	4.28	2.54×10^4	-5.54	7.65	3.19×10^4	11.96	3.43	2.46×10^4
NPB-MG	-11.04	27.19	2.24×10^4	-162.41	33.39	5.82×10^4	6.23	-1.32	1.14×10^5	-12.72	15.20	1.87×10^5	21.79	14.48	1.28×10^5
STREAM-FULL	7.76	1.50	7.98×10^4	-47.49	19.33	1.76×10^5	9.21	4.06	8.25×10^4	1.23	0.29	8.34×10^4	27.50	6.19	6.86×10^4
STREAM-TRIAD	7.66	12.86	2.05×10^4	-128.48	20.24	5.76×10^4	11.17	10.48	1.89×10^4	-4.44	6.45	2.06×10^4	32.82	12.58	1.48×10^4
STREAM-ADD	10.97	13.96	2.15×10^4	-147.56	27.35	7.47×10^4	35.17	4.60	1.32×10^5	0.60	2.87	1.96×10^4	33.77	13.80	1.59×10^4
STREAM-SCALE	9.61	15.77	8.07×10^3	-131.15	22.24	2.30×10^4	12.17	8.75	6.92×10^3	-0.84	6.49	7.62×10^3	32.65	8.75	5.30×10^3
STREAM-COPY	5.63	16.86	8.86×10^3	-143.02	28.76	2.77×10^4	11.65	11.53	7.56×10^3	1.23	4.82	7.46×10^3	32.83	8.64	5.45×10^3
STREAM-PHASES	11.15	14.51	1.15×10^5	-65.80	24.02	2.51×10^5	2.20	7.12	1.10×10^5	-0.35	3.38	1.05×10^5	33.35	11.47	8.14×10^4
Average	0.49	14.45	1.90×10^5	-109.43	19.61	3.56×10^5	8.48	6.26	1.66×10^5	-3.77	7.71	1.88×10^5	20.34	7.40	1.43×10^5

Table 5.3: Controllers compared with uncapped (Max PCAP). Only rowwise extremes are colored: **green** = best (max Energy Saved, min Perf Degradation, min ED^2P); **red** = worst (min Energy Saved, max Perf Degradation, max ED^2P). Lower ED^2P is better.

efficiently or as stably as the other benchmarks. We suspect that their characteristics, including a smaller average progress than most other benchmarks, might represent a challenge when not including such applications as part of the training dataset. A more exhaustive search for the ideal training set, however, becomes prohibitively expensive as the number of applications increases. Yet, these benchmarks do not result in a performance degradation higher than that of the other benchmarks.

To understand the impact of the proposed controller in optimizing the target objective, we plot the ED^2P values corresponding to each of these experiments. The results appear in Figure 5.4.

The results show that the proposed controller was able to regulate power across different application scenarios, often where the ED^2P value was close to its global minimum. Compared with the uncapped power setting, the controller gave better results in all cases.

5.6.2 Comparison with Existing Methods

We compare our proposed method against several state-of-the-art approaches. The baseline scheme is defined as uncapped power consumption, with the CPU frequency governor set to *performance*. The other methods compared against this baseline are described here.

PI Controller Cerf et al. (2021b): This implementation of PCAP control employs a classical proportional–integral (PI) controller with user-defined set points to specify the minimum allowable performance degradation. It uses both application- and hardware-based performance measurements to construct a model that serves as the basis for controller design. We use this controller in two variants: one based on a global model for all applications and hardware configurations and another that employs application- and hardware-specific models tailored to each case.

Ondemand CPU Governor Wysocki (2017): This scheme relies on the manufacturer-provided *ondemand* governor to dynamically adjust CPU frequencies during execution. Here, applications run without explicit power capping, with power managed implicitly by the governor based on workload demand.

RL-Based DVFS Control Wang et al. (2025): This method uses an online reinforcement learning approach to regulate core frequency through dynamic voltage and frequency scaling (DVFS). The RL agent interacts with the system at runtime to train its model and determine suitable core and uncore power settings for the node executing the application. This method, however, relies solely on IPC as a measure of performance, which has been shown to be an unreliable indicator of scientific progress Ramesh et al. (2019b). Furthermore, the runtime overhead of on-node training perturbs application execution and contaminates the very performance counter readings the agent relies on.

Comparison Approach: For the PI controllers, we tuned application-specific parameters of the common mathematical model from Cerf et al. (2021b) based on the performance data collected for offline training. Then, each PI controller regulates node power under the same experimental conditions as the RL controller, for each application. Because PI controller design depends heavily on the underlying application and hardware characteristics, a separate parametrization was required for each case. To evaluate generalization, we also developed a single, combined global PI shared across all applications, fitting the model as best as possible across applications. However, performance and power regulation using this global model is significantly worse, primarily because of model inaccuracy, which led to noisy actuation and unstable control behavior.

Similarly, we trained a DVFS-based RL controller using the same dataset employed for training our proposed offline RL method but considering only the instructions per cycle as the performance metric. Unlike the original paper Wang et al. (2025) this training was performed offline since online training on the compute node can interfere with application runtime and introduce noisy measurements. The resulting DVFS

controller was evaluated under the same testing conditions as the proposed RL controller. For the *ondemand* governor, the Linux kernel autonomously selected the optimal core frequency during execution while keeping the node power uncapped.

Among all the compared methods, the PI controller with a global model performed the worst, exhibiting frequent PCAP oscillations that resulted in higher energy consumption and longer execution times. The application-specific PI controller performed better, achieving the second-lowest average ED²P among the baseline methods. The DVFS-based RL controller performed well on compute-bound applications, as its frequency regulation aimed to maximize IPC, an effective proxy for such workloads. It remained a close competitor to the proposed RL controller on memory-bound applications, outperforming it only on the STREAM FULL benchmark.

As shown in Figure 5.5, the proposed RL controller achieves a better ratio of energy savings to performance loss and a substantial improvement in minimizing the overall ED²P across the benchmark applications.

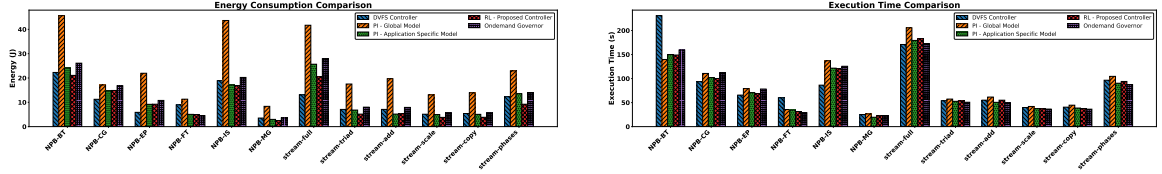


Figure 5.5: Aggregate comparison between the proposed RL-based power-capping controller and four baselines: global PI, application-specific PI, DVFS, and the *ondemand* governor.

5.7 Background and Related Work

This section delves into the background needed to understand the presented research and compare it with prior efforts.

5.7.1 Background on Offline Reinforcement Learning

Reinforcement learning (RL) formulates control as a sequential decision-making problem, typically modeled as a Markov decision process (MDP) $\mathcal{M} := (\mathcal{S}, \mathcal{A}, \mathcal{R}, \mathbf{T}, \gamma)$, where an agent observes the system state $s \in \mathcal{S}$, selects an action $a \in \mathcal{A}$, and receives a reward $r \in \mathcal{R}$. The objective is to learn a policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$ that maximizes the expected discounted return.

In an offline RL setting, such as the Q-learning approach used in this paper, unobserved state-action pairs cannot be evaluated during training. Instead, learning relies on the ability of the Q-network to approximate a value function from the available dataset. A common challenge in this setting is distributional shift, which arises when the distribution of states and actions encountered during training differs from that observed at deployment, potentially degrading performance. Another limitation of offline RL is suboptimal learning

when trained solely from random policies.

To address these challenges, we employ Conservative Q-Learning (CQL) Levine et al. (2020a), which regularizes Q-value estimation to mitigate overestimation under distributional shift. By carefully selecting learning parameters, we train an offline CQL agent from a previously collected dataset without overfitting. Moreover, incorporating domain knowledge of the controlled system enables the dataset to be populated with informative (greedy) behaviors, improving the quality of the learned policy.

5.7.2 Comparison with Related Efforts

Energy consumption of compute-intensive applications has been the focus of substantial research in different domains of science and technology including semiconductor design, architecture design, and application design. In this section we discuss relevant practices and techniques in the literature to reduce the energy consumption on compute nodes given the hardware architecture and application.

Power profiling is the conventional and convenient practice to understand the application phases prior to actual usage. For example, Ryan et al. Grant et al. (2017) surveyed different methods used in the analysis of an application’s runtime profile, where input power consumption changes. The authors classified the power-profiling methods into in-band and out-of-band, based on the approach used to measure the power consumption of a node.

For instance, the job-wide energy consumption returned during or at the end of an application runtime, measured using external meters, such as WattsUp! and WattProf, is considered out-of-band, while the same data collected by analyzing the measurements from device-level integrated measurement sensors is considered in-band. From the perspective of an application during its execution, in-band measurements are more accurate and useful for dynamic power control.

Simmendinger et al. Simmendinger et al. (2024) highlighted the importance of dynamic power capping. They showed that adjusting the power available dynamically can allow a management policy to deal with changes in applications’ behaviors and also to redistribute available power to other nodes or resources within a datacenter.

Several works have explored system software for node-level energy optimization. The Energy Aware Runtime (EAR) Corbalan et al. (2020) regulates power at runtime using phase detection based on MPI activity, while the Bull Dynamic Power Optimizer Stoffel and Mazouz (2018) rely on profiling and instruction-per-cycles heuristics for control.

In contrast, our approach overcomes the application- and hardware-specific nature of these controllers by being amenable to control of unknown applications, while gathering application-agnostic performance data (heartbeats) for increased adaptability.

In another study, Cerf et al. Cerf et al. (2021b) used a control theoretical approach to attenuate disturbances in performance under reduced power supply. They designed a classical proportional-integral-derivative controller to impose power caps on RAPL actuators using performance feedback from the compute nodes. The controller was shown to track a user-defined performance set point.

More recently, ML techniques have demonstrated increased efficacy in managing online job scheduling and hardware configurations Chen and Marculescu (2015); Li et al. (2024), offering broad applicability across various applications, where modeling the relationship between cluster node configurations and performance is impractical.

Prior efforts using RL in the control of compute node energy efficiency exist. Wang et al. Wang et al. (2025) proposed an online power control algorithm by observing the total instructions per cycle (*IPC*) while running an application and thereby regulating core and uncore frequencies with online interactions during the execution of applications to enhance learning. However, Ramesh et al. Ramesh et al. (2019b) show that *IPC* is an unsuitable performance metric for compute-intensive applications as it fails to distinguish useful application progress from idle-time activity, both of which contribute to the total instruction count.

Likewise, the authors in Raj et al. (2023b) used RL trained on a mathematical model of a modified STREAM benchmark to regulate power of HPC nodes through RAPL actuators. But the process of generating a mathematical model for each application’s behavior to be trained is cumbersome.

In relation to these works, we formally introduce an offline RL-based dynamic power management algorithm that is independent of phase detectors and surrogate models and is capable of managing power across diverse hardware and applications to ensure optimal energy utilization at runtime.

5.8 Conclusions

This paper addresses the critical challenge of power consumption and performance management within datacenter and HPC cluster nodes by presenting a dynamic power adjustment approach that seeks to minimize energy use while preserving application performance. Leveraging offline reinforcement learning, our strategy uses datasets collected from static characterization of compute nodes across various benchmarks to train an effective RL agent that dynamically optimizes hardware configuration (*PCAP*).

Experimental validations demonstrate our system’s robustness, yielding standard deviations in execution time ranging from a minimum of 0.47s to a maximum of 2.91s and for energy consumption from a minimum of 0.01kJ to a maximum of 1.22 kJ, indicating consistent energy savings across both memory-bound and compute-bound benchmark tasks. Remarkably, the controller proved its efficacy when applied to new and phase-varied applications, achieving an average energy saving of 20.34% with only a minimal average performance degradation of 7.4%. Repeated tests have further solidified the controller’s capability to significantly

enhance energy efficiency. These findings underscore the potential of RL-based control for overcoming the limitations of application- and hardware-centric power management designs. Our research bears particular importance for cloud service providers and HPC cluster administrators aiming to significantly reduce energy expenditure with only a marginal compromise on performance. In the future, we intend to explore the deployment of our algorithms in GPU-intensive tasks and wider cluster job environments.

While our approach has successfully reduced energy consumption with minimal impact on performance, we have so far limited our use of progress as an application-agnostic signal for online performance to applications that can exhibit an iterative behavior, with a heartbeat that occurs often enough during a control interval and is somewhat stable in rate for a given kernel. Future work will concentrate on widening this approach, from extracting a stable signal for a wider set of metrics (like OpenMP callbacks or MPI activity) and dealing with applications than iterate over a heterogeneous workload.

The code and experimental data are available in open source in our [GitHub page](#).

Acknowledgment

Results presented in this paper were obtained by using the Chameleon testbed supported by the National Science Foundation. We also acknowledge the use of AI tools for language improvements and text generation in certain sections of this paper. All results and generated content were carefully reviewed by the authors for accuracy. Responsibility for the content of this paper lies solely with the authors.

Decay-Driven Reinforcement-Learning-Based Power Control for Single Node Energy Efficiency

Abstract

Energy efficiency in high-performance computing (HPC) systems is increasingly critical, yet balancing energy savings against application performance remains a fundamental challenge. Existing reinforcement learning (RL)-based approaches either require costly online exploration or train separate agents for each desired performance-energy trade-off, limiting their practicality in production HPC environments. In this chapter, we present *Decay-Driven Multi-Objective Reinforcement Learning* (DDMORL), a single offline-trained controller that allows users to specify their maximum tolerable performance degradation as a percentage, which is automatically converted to a preference vector aligned with the corresponding operating point on the Pareto front. This eliminates the need for multiple agents, manual preference tuning, or online retraining. DDMORL extends the Preference-Driven MORL (PDMORL) framework to an offline setting by replacing the conventional preference-alignment interpolator with a lightweight, application-specific progress-to-PCAP model. The controller is trained on pre-collected execution traces from a mixed dataset of eight benchmarks, comprising STREAM memory-bandwidth variants and NAS Parallel Benchmarks. It is then deployed without retraining on out-of-distribution test workloads, where it produces well-structured Pareto fronts while keeping performance degradation within the user-specified limit.

6.1 Introduction

Energy efficiency has emerged as a fundamental constraint in modern high-performance computing (HPC) systems. Large production systems execute long sequences of computational jobs under strict power budgets, where electricity costs can dominate total cost of ownership and, in many cases, overtake hardware costs by a large margin. At the same time, HPC applications are typically subject to performance constraints such as deadlines, turnaround time requirements, or bounded performance degradation. These competing objectives make energy-aware runtime control a central challenge in today's HPC environments.

A common abstraction used to study this problem is that of executing a sequence of jobs on a variable-speed processor so as to minimize energy consumption while satisfying performance constraints. In this setting, jobs often have strict deadlines but do not necessarily require a fixed execution time. Instead, applications can tolerate controlled slowdowns as long as they complete within user-specified or system-defined limits. Dynamic speed scaling, enabled by modern processors through mechanisms such as frequency control and power capping, provides a principled way to trade performance for energy savings by adjusting processor speed according to demand.

Dynamic speed scaling introduces a fundamental multi-objective optimization problem. Across different hardware platforms and applications, one constraint remains constant: energy consumption must be reduced. At the same time, performance requirements must be met. In current HPC systems, schedulers are responsible for deciding which resources to allocate to a job and, in some cases, selecting operating parameters such as node type or power limits. However, schedulers typically operate with limited knowledge of application-level performance sensitivity and must make decisions under uncertainty.

Much of the foundational work assumes an offline setting, where all jobs, along with their characteristics, are known in advance. This allows the computation of optimal or near-optimal schedules using global optimization techniques. In contrast, real HPC systems largely operate in an online setting, where jobs arrive dynamically and decisions must be made instantaneously without knowledge of future workloads. Online algorithms are commonly evaluated using competitive analysis, where an algorithm is said to be c -competitive if its objective value is within a factor c of the optimal offline solution for the same input.

To bridge theory and practice, HPC research often relies on proxy applications that capture representative computation and communication patterns of real workloads. Studying these proxy applications under different power caps allows us to understand how application performance degrades with reduced power and how these relationships vary across applications and hardware platforms. Such empirical characterization is essential for designing practical runtime controllers.

Reinforcement learning (RL) has gained attention as a promising framework for addressing this trade-off due to its ability to learn control policies directly from data. However, online RL is often impractical in HPC environments because exploration can degrade application performance and interfere with production workloads. Offline RL has therefore been explored as an alternative, where policies are trained using pre-collected system and application traces. Many existing approaches formulate the problem as a multi-objective optimization and convert it into a single objective by fixing weights on performance and energy, which often requires training multiple agents to convert different operating points.

In this paper, we focus on preference-driven control of HPC application runtime using a modified version of Decay-Driven Multi-Objective Reinforcement Learning (DDMORL) framework. Our goal is to provide a single, application- and hardware-agnostic control that allows users to explicitly specify

their acceptable performance degradation through a simple preference input. By leveraging mathematical models of application behavior on HPC nodes to generate preference signals for augmenting the training data, we remove the need for interpolation models during training and enable efficient offline learning. This approach eliminates the requirement to train separate RL agents for different performance-energy trade-offs.

Through extensive experiments on multiple applications and compute nodes, we demonstrate that user-defined performance degradation levels can be reliably mapped to power caps that achieve high energy efficiency. The results show that the proposed methods delivers significant energy savings while maintaining performance within the user’s specified limits, making it suitable for deployment in production HPC environments.

6.2 Background

6.2.1 Offline Preference-Driven Multi-Objective Reinforcement Learning

Many real-world control problems, including energy-aware scheduling and power-performance management in HPC systems, naturally involve multiple conflicting objectives. We discussed some of these in Section 6.1. Multi-objective reinforcement learning (MORL) formulates such problems by extending the scalar reward in standard RL to a vector-valued reward, enabling explicit reasoning over trade-offs between objectives such as performance and energy consumption.

Formally, a multi-objective Markov decision process (MOMDP) is defined as

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathbf{r}, \Omega \rangle, \quad (6.1)$$

where $\mathcal{S}$ and $\mathcal{A}$ denote the state and action spaces, $\mathcal{P}(s'|s, a)$ is the transition kernel, and $\mathbf{r}(s, a) \in \mathbb{R}^L$ is an L -dimensional reward vector. User preferences are encoded by a weight vector

$$\omega \in \Omega = \left\{ \omega \in \mathbb{R}_{\geq 0}^L \mid \sum_{i=1}^L \omega_i = 1 \right\}, \quad (6.2)$$

which induces a scalarized reward via $f_\omega(\mathbf{r}) = \omega^T \mathbf{r}$.

Traditional scalarization-based MORL methods learn a separate policy for each fixed preference vector, which is inefficient when preferences change dynamically and impractical in offline settings where retraining is costly or impossible. To address this limitation, Preference-Driven Multi-Objective Reinforcement Learning (PD-MORL) Basaklar et al. (2023) proposes learning a single universal value function that conditions on preferences and covers the entire preference space in one training phase.

6.2.1.1 Multi-Objective Value Functions and Preference Conditioning

PD-MORL extends Q-learning by defining a vector-valued Q-function $Q(s, a, \omega) \in \mathbb{R}^L$, which estimates the expected discounted return for each objective under the preference ω . The optimal policy for any preference can then be recovered at inference time by conditioning the network on ω , eliminating the need to store multiple policies.

A scalarized distance metric over multi-objective Q-functions is defined as

$$d(\mathbf{Q}, \mathbf{Q}') = \sup_{s, a, \omega} |\omega^T (\mathbf{Q}(s, a, \omega) - \mathbf{Q}'(s, a, \omega))|, \quad (6.3)$$

which is a complete pseudo-metric space due to its one-to-many mapping nature. Intuitively, this metric bounds the maximum discrepancy a user may observe in the energy-runtime trade-off when switching between learned value functions, while ensuring convergence of the preference-driven Bellman operator.

6.2.1.2 Preference-Driven Bellman Optimality Operator

The PD-MORL Bellman optimality operator is defined as:

$$(\mathcal{T}\mathbf{Q})(s, a, \omega) = \mathbf{r}(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)} \left[\mathbf{Q} \left(s', \arg \max_{a' \in \mathcal{A}} (S_c(\omega, \mathbf{Q}(s', a', \omega)) \cdot \omega^T \mathbf{Q}(s', a', \omega)), \omega \right) \right] \quad (6.4)$$

where $S_c(\cdot, \cdot)$ denotes the cosine similarity between the preference vector and the multi-objective Q-value:

$$S_c(\omega, \mathbf{Q}) = \frac{\omega^T \mathbf{Q}}{\|\omega\| \|\mathbf{Q}\|}. \quad (6.5)$$

This alignment term prevents domination by objectives with larger numerical scales, a critical issue in energy-performance optimization where metrics often differ by orders of magnitude. The resulting operator is shown to be a contraction under metric d , guaranteeing convergence to a unique fixed point corresponding to the optimal multi-objective value function.

6.2.1.3 Offline Learning and the Preference Alignment Problem

PD-MORL is well suited for offline reinforcement learning settings, where policies are learned from pre-collected system traces without further interaction with the environment. Since environment dynamics and objective-wise reward vectors are independent of user preferences, each recorded transition can be consistently reused across multiple preference vectors. PD-MORL leverages this property through hindsight experience replay, augmenting transitions $(s, a, \mathbf{r}, s', \omega)$ with alternative preference labels ω' , enabling efficient reuse of historical data to learn a single preference-conditioned value function that generalizes across the full performance and energy trade-off space.

However, PD-MORL as originally formulated Basaklar et al. (2023) is oriented toward online learning, where user-defined preferences directly guide experience generation. Extending it to a purely offline setting introduces a key challenge: preference alignment. PD-MORL addresses this through an interpolation mechanism that projects user preference vectors into a normalized solution space, mitigating misalignment between nominal preferences and the resulting Pareto-optimal solutions. This is particularly important offline, where corrective exploration is unavailable. However, constructing such an interpolator requires data that explicitly relates preference vectors to their corresponding Pareto-optimal points, which is cumbersome to collect and, in many practical HPC systems, infeasible.

6.2.1.4 From Preference Alignment to Decay-Driven Control

The preference alignment problem becomes even more acute in HPC settings, because users do not naturally think in terms of abstract preference weights. Instead, HPC practitioners reason about acceptable performance degradation: how much slowdown is tolerable before a job misses its objectives. This motivates a fundamentally different formulation.

Rather than asking users to specify a preference vector ω and then attempting to align it to the Pareto front through a learned interpolator, we propose a decay-driven approach in which the user directly specifies a maximum tolerable performance degradation $\delta \in [0, 1]$ as a percentage. This decay value is then converted into a preference vector $\tilde{\omega}$ that is analytically aligned to the corresponding operating point on the Pareto front. The alignment exploits a domain-specific property of HPC workloads: for a fixed application, instantaneous progress is approximately monotonic with respect to the enforced power cap (PCAP). This monotonic relationship is directly observable in the training dataset and can be captured by a lightweight application-specific model, eliminating the need for an episodically updated data-driven interpolator entirely.

This modification not only resolves the preference alignment problem in a principled way, but also makes the controller directly interpretable to HPC users. The full formulation of this decay-driven preference construction and its integration into the offline Q-learning framework is described in Section 6.3.

6.2.2 Application-Aware Performance Measurement

We augment our monitoring of application behavior by adding a notion of *progress*, or *heartbeats*, to our system. This measure is obtained through lightweight instrumentation of the applications under study, following the work of Ramesh et al. (2019b) and the instrumentation steps outlined by Cerf et al. (2021). The role of this progress measurement is to highlight the potential for an application to be busy-waiting for some resource in the system, and thus be active but not making useful use of compute power.

We redefine the progress measurement equation of Ramesh et al. (2019b) by considering the number of heartbeats received between two sampling instants. The progress at time instant t_i is given by:

$$progress(t_i) = \text{median}_{\forall k, t_k \in [t_{i-1}, t_i]} \left(\frac{N}{t_k - t_{k-1}} \right), \quad (6.6)$$

where N is the number of heartbeats received between time instants t_k and t_{k-1} . This measure serves as a runtime proxy for the figure of merit of an application. It accurately captures progress relative to the sampling interval and allows rate-limiting of the instrumentation, since it is sufficient to accumulate a heartbeat counter on the application side and periodically forward it to the data collection and control infrastructure.

Critically, progress also plays a central role in our decay-driven formulation. As defined in Section ??, the user-specified performance degradation δ is expressed relative to the maximum achievable progress $g_{\max}$, and the preference vector $\tilde{\omega}$ is constructed from the target progress $g^* = (1 - \delta) g_{\max}$. Progress is therefore not merely a monitoring signal but a first-class input to the preference alignment mechanism of DDMORL.

6.2.3 Measuring Application Performance through Hardware Performance Counters

Applications vary significantly in their performance on a given hardware architecture, depending on characteristics such as memory bandwidth, CPU core count, and cache size. To understand this complex application-hardware interaction, we capture a set of hardware performance counters while running applications on the target system using the Performance Application Programming Interface (PAPI) Mucci et al. (1999), which provides portable access to hardware model-specific registers.

We focus on five key counters: total instructions issued (TOT_INS), total clock cycles (TOT_CYC), total last-level cache accesses ($L3_TCA$), last-level cache misses ($L3_TCM$), and cycles stalled due to resource contention (RES_STL). These counters are chosen for two reasons. First, they are available instantaneously and simultaneously at every sampling interval. Second, they are direct indicators of data flow within and between the CPU cores and memory subsystem. To reduce input dimensionality and avoid the need for a larger network, we follow insights from prior work Wittmann et al. (2016); Rane and Browne (2014) and compute three derived metrics: instructions per cycle (IPC), the ratio of stalled cycles to total cycles (STL), and the last-level cache miss rate (CMR). These derived features facilitate a compact and informative characterization of runtime behavior.

Key Remark: These metrics should not be viewed in isolation as definitive indicators of performance. Varying IPC values, for example, can still lead to the same execution time. Nonetheless, they serve as valuable proxies for discerning application characteristics, including arithmetic intensity and memory boundedness, both of which directly influence how aggressively an application can be power-capped without significant performance loss.

6.2.4 Proxy Applications and Dataset Partitioning

To rigorously evaluate generalization, we partition the benchmark set into training and testing groups using a Calder-inspired similarity analysis ?, rather than relying on random or arbitrary splits. Calder quantifies similarity between applications based on hardware performance counter behavior, enabling principled control over the distributional gap between training and evaluation workloads. It computes similarity by (i) selecting informative features using Laplacian score, (ii) removing redundant features through correlation filtering, and (iii) comparing applications using statistical distance metrics including cosine similarity, Jensen-Shannon divergence, Wasserstein distance, and Mahalanobis distance. These metrics capture differences in execution characteristics such as instruction mix, memory behavior, and communication patterns.

We construct two types of dataset partitions. The in-distribution (ID) split groups applications with high behavioral similarity, using representative traces for training and held-out traces from similar applications for testing. This evaluates interpolation within known behavioral regimes. The out-of-distribution (OOD) split intentionally separates dissimilar applications, placing workloads from distinct regions of the similarity space into training and test sets respectively. This evaluates the ability of the learned controller to generalize to unseen application behaviors.

This partitioning is critical because the proposed offline RL controller is trained entirely from logged traces and must generalize without online exploration. A random split would likely overestimate performance by inadvertently mixing similar behaviors across train and test sets. The Calder-guided split ensures the test set represents genuinely unseen system dynamics, providing a more realistic assessment of deployment performance.

Table 6.1 summarizes the full benchmark set. The eight benchmarks above the double line constitute the training set, comprising five STREAM memory-bandwidth variants (Scale, Triad, Add, Copy, and Full) and three NAS Parallel Benchmarks (EP, FT, and BT). The four benchmarks below the double line form the out-of-distribution test set, consisting of Stream-Phase and three additional NPB variants (CG, IS, and MG). This mixed training set is intentional: by including both memory-bound (STREAM) and compute-intensive (NPB) workloads during training, the controller is exposed to a broader range of hardware counter signatures, which we show in Section 6.6 improves generalization to the OOD test benchmarks.

Table 6.1: Detailed description of our benchmarks, including their type, parameters, and average performance metrics (uncapped). Benchmarks above the double line form the training set; those below form the out-of-distribution test set.

Benchmark	Prob Size	Iter.	Avg. IPC	Avg. CMR	Avg. STL	Avg. Prog. [Hz]	Avg. ET [s]	Avg. Energy [kJ]	Ari. Int. (FLOPs/Byte)
STREAM-SCALE	33554432	10000	0.20	0.89	0.84	285.20	34.40	5.59	1.62
STREAM-TRIAD	33554432	10000	0.18	0.94	0.83	200.03	50.80	7.93	1.41
NPB-EP	Class-W	1000	0.57	0.13	0.48	13.61	73.80	10.33	53.99
NPB-FT	Class-B	500	0.82	0.43	0.296	13.23	35.10	45.47	1.53
STREAM-ADD	33554432	10000	0.15	0.94	0.85	201.06	50.40	7.84	1.36
STREAM-COPY	33554432	10000	0.17	0.89	0.84	282.43	35.50	5.66	1.63
STREAM-FULL	33554432	10000	0.16	0.93	0.70	49.31	222.40	34.99	1.45
NPB-BT	Class-C	1000	0.97	0.87	0.30	6.22	166.00	23.168	0.54
STREAM-PHASE	33554432	10000	0.17	0.91	0.88	240.16	83.00	13.48	1.45
NPB-CG	Class-C	1000	0.49	0.37	0.62	9.97	114.00	14.74	3.61
NPB-IS	Class-B	1000	0.50	0.86	0.68	8.44	117.33	18.99	1.21
NPB-MG	Class-B	1000	0.47	0.809	0.54	40.3	22.23	55.5	1.45

6.2.4.1 Calder Based Similarity Analysis

To guide dataset partitioning, we use a Calder-inspired similarity analysis based on PCA-reduced feature vectors and Mahalanobis distance. This analysis allows us to quantify how application behavior changes both across power-cap settings and across workloads, and it provides a principled basis for separating training and testing data.

Figure ?? shows the pairwise Mahalanobis distance matrix for different PCAP settings of the same application, `ones-stream-full`. The distances are computed after projecting the behavioral features into a lower-dimensional PCA space. The resulting matrix exhibits a clear structured trend: nearby PCAP levels have small distances, while larger separations in PCAP correspond to progressively larger distances. For example, adjacent settings such as 147 W and 153 W remain close in the feature space, whereas the distance between low and high caps such as 89 W and 159 W is substantially larger.

This pattern indicates that power capping induces a smooth and continuous shift in application behavior rather than abrupt regime changes. In other words, the execution characteristics of the application evolve gradually as the power cap is adjusted. This observation supports treating PCAP as a continuous control dimension in the reinforcement learning formulation and justifies grouping nearby power-cap regimes together when constructing the training dataset.

Cosine Similarity: Measures the cosine of the angle between two vectors in a multi-dimensional space. In this context, it quantifies how similar the performance profiles (vectors of averaged metrics like *TOT_INS_PER_CYC*, *progress_per_cycle*, etc.) of two applications are. The code converts it to degrees (0° = identical profiles, 90° = completely dissimilar/orthogonal). Higher similarity (lower angle) indicates applications behave more alike in terms of the measured attributes.

JS-Divergence (Jensen-Shannon Divergence): A symmetric measure of the difference between two probability distributions. It quantifies how much information is lost when one distribution is used to approximate another. Lower values indicate greater similarity between the application profiles, with 0 meaning identical distributions. It's more stable than KL divergence and bounded between 0 and 1 (scaled to 0-100 in the plots).

Wasserstein Distance: Also known as Earth Mover's Distance, it measures the minimum "work" needed to transform one distribution into another by moving probability mass. In the application comparison, it assesses how different the metric distributions are between apps. Lower distances mean more similar profiles; the values are scaled by 1000 in the code for readability, and annotated in scientific notation on the plots.

PCA + Mahalanobis Distance: First reduces the dimensionality of the data using Principal Component Analysis (PCA) to capture the most variance, then computes the Mahalanobis distance, which accounts for correlations between variables and scales them appropriately. This provides a correlation-aware measure of dissimilarity. Lower distances indicate more similar applications after dimensionality reduction. Values are scaled by 10 in the code.

Figure ?? complements this view by measuring pairwise similarity across different applications at a fixed power cap of 159 W. Unlike the smooth variation observed within a single application, the cross-application matrix shows substantially higher diversity, indicating that application identity has a strong effect on the observed behavior even under the same power constraint. This supports the use of Calder-based similarity analysis to distinguish training and testing workloads and to evaluate generalization under realistic distribution shifts.

Figures 6.1(a)–(d) jointly illustrate the interplay between application behavior and power capping. Figure 6.1a shows that cosine similarity across power caps remains near zero for Stream-Full, confirming that the intrinsic hardware behavior of an application is invariant to power allocation. In contrast, Figure 6.1b highlights clear cross-application diversity at a fixed power cap, where memory-bound Stream variants form a tight cluster while NPB-BT and NPB-IS remain distinct. However, cosine similarity alone fails to capture magnitude variations: Figures ?? demonstrate that PCA-based Mahalanobis distance reveals substantial shifts across power caps, with NPB-BT exhibiting a smooth monotonic progression and NPB-EP showing saturation effects. Together, these results establish that while application characteristics remain structurally consistent across power caps, their performance (progress) varies significantly, motivating the use of PCAP as a controllable parameter in our learning framework.

6.3 Problem Statement

We consider an HPC application executing on a compute node where the controller selects a power-cap action (PCAP) at each control interval to balance performance and power/energy. Let s_t denote the system observation at time t (hardware counters + progress), and let $a_t \in \mathcal{A}$ denote the selected PCAP action.

Instead of directly optimizing raw performance, we formulate the problem in a *decay-driven* manner, where the objective is to control the degradation in performance relative to a nominal high-performance execution. Let $progress_{\max}$ denote the maximum achievable progress (e.g., at the highest PCAP), and define the performance decay as

$$d(t_i) = \frac{progress_{\max} - progress(t_i)}{progress_{\max}}. \quad (6.7)$$

The environment provides a two-dimensional reward vector

$$\mathbf{r}(t_i) = \begin{bmatrix} r_{\text{decay}}(t_i) \\ r_{\text{energy}}(t_i) \end{bmatrix} = \begin{bmatrix} \text{progress}(t_i) \\ P_{\text{avg},t_i} \end{bmatrix}, \quad (6.8)$$

where maximizing progress corresponds to preserving performance, and minimizing P_{avg,t_i} corresponds to reducing power consumption.

A user specifies a preference in terms of acceptable performance degradation. Let $\delta \in [0, 1]$ denote the maximum tolerable decay. This formulation is more natural in HPC settings, where users typically reason in terms of allowable slowdown rather than absolute performance or abstract reward weights.

The objective is to learn, from offline execution traces, a single controller that produces near Pareto-optimal power-cap decisions while respecting the user-specified decay constraint, without requiring separate controllers for different operating points.

In standard preference-driven MORL, preference conditioning often relies on an interpolator that maps user preferences to regions of the Pareto front. However, constructing such interpolators requires extensive labeled data across multiple objectives and power settings, which is impractical in HPC environments.

Instead, we exploit a key domain property: for a fixed application, performance (progress) is approximately monotonic with respect to PCAP. This allows us to establish a direct relationship between performance decay and power cap.

Although execution time and energy are defined as integrals over the execution trajectory, they are monotonically related to instantaneous progress and power. Specifically, higher progress rates reduce execution time, while higher power increases energy consumption.

Under the assumption that progress is monotonic with respect to PCAP, preferences over (T, E) are consistently aligned with preferences over $(\text{progress}, P)$. Therefore, optimizing in the *progress - power* space provides a tractable surrogate for controlling the *execution time - energy* trade-off.

For small control intervals, execution time and energy can be approximated as $T \approx \sum \Delta t$ and $E \approx \sum P_i \Delta t_i$, implying that instantaneous progress and power serve as locally proportional proxies for runtime and energy, respectively.

Let $\mathcal{G}_{\text{app}}$ denote an application-specific mapping that captures the relationship between progress and power-cap (PCAP). This mapping is constructed from training traces by fitting a lightweight model (e.g., a polynomial) to the observed progress-PCAP relationship.

Given a user-specified decay tolerance $\delta \in [0, 1]$, we first define the target progress as

$$g^* = (1 - \delta) g_{\text{max}}, \quad (6.9)$$

where g_{max} denotes the maximum achievable progress under the highest PCAP setting.

Using the fitted model $\mathcal{G}_{\text{app}}$, the corresponding desired power-cap is obtained as

$$\text{PCAP}_{\text{des}} = \mathcal{G}_{\text{app}}(g^*). \quad (6.10)$$

Let P^* denote the average power induced by PCAP_{des} . This defines a target operating point in the objective space:

$$\mathbf{x}^* = \begin{bmatrix} g^* \\ P^* \end{bmatrix}. \quad (6.11)$$

The aligned preference vector is then constructed to reflect this operating point:

$$\tilde{\omega} \propto \begin{bmatrix} g^* \\ -P^* \end{bmatrix}, \quad (6.12)$$

or equivalently, using a positive energy-savings formulation,

$$\tilde{\omega} \propto \begin{bmatrix} g^* \\ P_{\text{max}} - P^* \end{bmatrix}. \quad (6.13)$$

This construction aligns the preference vector directly with the desired operating point induced by the decay constraint, eliminating the need for a learned preference interpolator and enabling a principled connection between user intent and system behavior.

Given this alignment, action selection is performed using a preference-aware Q-function:

$$\text{score}(s', a') = \max(0, \cos(\tilde{\omega}, \mathbf{Q}(s', a'))) \cdot \tilde{\omega}^\top \mathbf{Q}(s', a'), \quad (6.14)$$

where $\tilde{\omega}$ encodes the decay-aligned trade-off between performance and energy.

Overall, this decay-driven formulation provides an interpretable and practically meaningful control objective, while enabling efficient offline learning and generalization across user-specified performance constraints.

6.4 Methodology

In this section we highlight the underlying methodology and steps involved in this paper. The entire methodology can be broadly classified into three phases as shown in the figure 6.2, namely: i) Data collection, ii) Training and iii) Testing/ evaluation.

6.4.1 Data Collection

As defined in the section 6.2.1, a data sample for any RL method consists of state, action, next state and reward. The state is a representative of the environment and the agents action on it. The RL agent constantly observe this state and take action $a(t)$ based on the observation that results in the environment transition from state $s(t)$ to the state $s(t+1)$ while returning the reward $r(t)$. The HPC node environment is defined by the hardware and the application that runs on it. Therefore we use the features engineered using the measurements from hardware performance counters along with progress and measured power as the representative of the state. Apart from these we also give the user defined decay in performance as one of the feature of the state. Therefore $s(t) = [\text{progress}(t), \text{power}(t), \text{IPC}(t), \text{STL}(t), \text{CMR}(t), d(t)] \in \mathbb{R}^6$. The action $a(t)$ in case of the HPC node is the node power cap represented as $\text{PCAP}(t) \in \mathbb{R}$. Reward in case of a multiobjective reinforcement learning is not any more scalar. Rather it is a vector whose dimension equals the number of objectives. In our case, energy and execution time being two objectives deciding the pareto front our reward vector $\mathbf{r}(t) = [\text{progress}, \text{power}]$ as defined in the equation 6.8. As the underlying learning is going to be offline it is important to include optimal policies in the dataset. As all actions are optimal corresponding to some decay factor $d(t_i)$ we calculate the delta from the collected data set. We calculate the delta using the equation 6.7 and append it to the state of the dataset. Followinglly, the state $s(t)$ and the next state $s(t+1)$ are both appended with the calculated decay to ensure a sequential state transition.

6.4.2 Offline Training

We use offline reinforcement learning to avoid the training model and run time application compete for the same node resources 6.1. The dataset collected is now used to train an RL agent defined using a Q network. The state observation consisting of the $s(t) = [\text{progress}(t), \text{power}(t), \text{IPC}(t), \text{STL}(t), \text{CMR}(t), d(t)]$ goes into a finite action spaced Q network and return the Q vector corresponding to each of the actions defined in the action space. This Q vector is compared against a weight vector calculated using the data points connecting the PCAP and progress for alignment and also evaluated for maximum performance per watt.

6.4.3 Evaluation

For the evaluation phase we use the trained model and deploy it alongside the runtime applications on the hardware having similar actuator configuration. We perform the evaluation using different applications as given in the table 6.1 that differentiates training and testing applications. During the evaluation the controller receives information from the hardware performance counters that are instantaneously converted to the useful features along with progress and average power consumed. Followinglly we append the user defined preference to this feature vector and let the controller implement the action. The action is enforced using the PCAP control knob through the intel RAPL actuators. We repeat the process multiple times and record the average of energy and execution time to get a good repeatability statistics. We evaluate the results by plotting the pareto front of the application on a given hardware alongside the proposed controller. We plot the weightage or the preferences on each of the objectives for the points obtained as well as the percentage decay each of these points corresponds to.

6.5 Experimental Setup

6.5.1 HPC Platform and Software Stack

All experiments are conducted on an HPC node managed by the Node Resource Manager (NRM) [?], which exposes a PCAP (power cap) actuator via its `libnrm` client interface. Power and energy measurements are collected through GEOPM sensors (`nrm.geopm.CPU_POWER` and `nrm.geopm.CPU_ENERGY`), while microarchitectural performance counters are instrumented using PAPI via `nrm-papiwrapper`. The five hardware events collected are: `PAPI_TOT_INS`, `PAPI_TOT_CYC`, `PAPI_L3_TCM`, `PAPI_L3_TCA`, and `PAPI_RES_STL`. Application progress is reported at runtime through the NRM progress sensor (`nrm.benchmarks.progress`).

Algorithm 1 Offline Preference-Aligned Q-Learning

Require: Offline dataset $\mathcal{D} = \{(s_t, a_t, r_t, s_{t+1})\}$

Require: Progress-PCAP model $\mathcal{G}_{app}$

Require: Learning rate η , discount γ , batch size B , soft update rate τ

Ensure: Trained network parameters θ

1: Initialize online network Q_θ and target network $Q_{\bar{\theta}} \leftarrow Q_\theta$

2: **for** each iteration **do**

3: Sample minibatch $\{(s_i, a_i, r_i, s'_i)\}_{i=1}^B \sim \mathcal{D}$

4: Construct state $s_i = [\text{progress}, \text{power}, \text{IPC}, \text{STL}, \text{CMR}, d]$

5: Compute decay-aligned target progress:

$$g_i^* = (1 - \delta_i) g_{\max}$$

6: Map to desired PCAP:

$$\text{PCAP}_{\text{des},i} = \mathcal{G}_{app}(g_i^*)$$

7: Construct aligned preference vector $\tilde{\omega}_i$

8: Compute target using preference-aware Bellman operator:

$$(\mathcal{T}\mathbf{Q})(s, a, \omega) = \mathbf{r}(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)} \left[\mathbf{Q} \left(s', \arg \max_{a' \in \mathcal{A}} (S_c(\omega, \mathbf{Q}(s', a', \omega)) \cdot \omega^\top \mathbf{Q}(s', a', \omega)), \omega \right) \right]$$

9: Approximate expectation using sampled s'_i and target network $Q_{\bar{\theta}}$

10: Form Bellman target:

$$y_i = (\mathcal{T}\mathbf{Q})(s_i, a_i, \tilde{\omega}_i)$$

11: Update network parameters:

$$\theta \leftarrow \theta - \eta \nabla_\theta \frac{1}{B} \sum_{i=1}^B \|Q_\theta(s_i, a_i) - y_i\|^2$$

12: Soft update target network:

$$\bar{\theta} \leftarrow \tau \theta + (1 - \tau) \bar{\theta}$$

13: **end for**

14: **return** Q_θ

6.5.2 Workloads

The training dataset is collected from five variants of the STREAM memory bandwidth benchmark: `ones-stream-full`, `ones-stream-triad`, `ones-stream-add`, `ones-stream-copy`, and `ones-stream-scale`. Each benchmark operates on an array of 33,554,432 double-precision elements (256 MB per array) and runs for 10,000 iterations. Evaluation is performed on `ones-stream-full` across nine fixed preference configurations.

6.5.3 Offline Data Collection

The training data is collected by `data_generation.py`, which runs each application under NRM management and applies a randomly selected PCAP value every 10 seconds from the discrete action space $\mathcal{A} = \{89, 95, 101, 107, 112, 118, 124, 130, 136, 141, 147, 153, 159, 165\}$ W. Each application is executed for 10 independent repetitions, yielding a dataset of 381 transitions after quality filtering.

At each PCAP interval $[t_1, t_2]$, a five-dimensional state vector is constructed by averaging sensor readings within the interval:

$$\mathbf{s} = [\bar{f}_{\text{prog}}, \bar{P}, \overline{\text{IPC}}, \overline{\text{L3_MISS_RATE}}, \overline{\text{STL_CYC}}]^\top \in \mathbb{R}^5, \quad (6.15)$$

where $\bar{f}_{\text{prog}}$ is the median progress rate, $\bar{P}$ is mean measured power, $\overline{\text{IPC}} = \text{TOT_INS}/\text{TOT_CYC}$, $\overline{\text{L3_MISS_RATE}} = \text{L3_TCM}/\text{L3_TCA}$, and $\overline{\text{STL_CYC}} = \text{RES_STL}/\text{TOT_CYC}$.

The two-objective reward vector for a transition (s, a, s') is:

$$\mathbf{r} = [r_{\text{pow}}, r_{\text{prog}}]^\top, \quad (6.16)$$

where each component is normalized to $[-1, +1]$:

$$r_{\text{pow}} = -\left(2 \frac{P - P_{\min}}{P_{\max} - P_{\min}} - 1\right), \quad (\text{low power} \Rightarrow +1) \quad (6.17)$$

$$r_{\text{prog}} = \left(2 \frac{f'_{\text{prog}}}{f_{\text{prog}}^{\max}} - 1\right), \quad (\text{high progress} \Rightarrow +1) \quad (6.18)$$

with $P_{\min}$, $P_{\max}$, and $f_{\text{prog}}^{\max}$ computed per application from the assembled dataset. The dataset columns are summarized in Table 6.2.

Table 6.2: Training dataset structure (381 transitions, 5 applications).

Field	Description	Range
App	Application identifier	5 classes
Progress	$\bar{f}_{\text{prog}}$	$[0, 300]$
Power	$\bar{P}(\text{W})$	$[0, 165]$
IPC	Instructions per cycle	$[0, 1]$
L3 miss rate	L3 TCM / L3 TCA	$[0, 1]$
Stall ratio	RES_STL / TOT_CYC	$[0, 1]$
Action	PCAP (W)	$\mathcal{A}$, $ \mathcal{A} = 14$
Reward	$[r_{\text{pow}}, r_{\text{prog}}]$	$[-1, +1]^2$
Next state	$\mathbf{s}'$ (same fields)	as above

6.5.4 Decay-Driven Preference

DDMORL uses a time-varying preference vector that decays from power-focused to progress-focused as the application nears completion:

$$\omega(t) = [1 - \phi(t), \phi(t)]^\top, \quad \phi(t) = \frac{f_{\text{prog}}(t)}{f_{\text{prog}}^{\max}} \in [0, 1], \quad (6.19)$$

where $f_{\text{prog}}(t)$ is the current progress rate. At the start of execution $\phi \approx 0$ so $\omega \approx [1, 0]^\top$ (conserve power); as the application progresses ω shifts toward $[0, 1]^\top$ (maximise throughput).

6.5.5 Policy Network Architecture

The Q-function is represented by a fully connected network $Q_\theta : \mathbb{R}^7 \rightarrow \mathbb{R}^{32}$ with architecture $7 \rightarrow 50 \rightarrow 20 \rightarrow 32$, where the input concatenates the five state features with the two-dimensional preference vector, and the output encodes a two-objective Q-vector for each of the 16 PCAP actions:

$$Q_\theta(\mathbf{s}, \omega) \in \mathbb{R}^{A \times 2}, \quad A = 16. \quad (6.20)$$

6.5.6 Offline Training

The policy is trained via Conservative Q-Iteration (CQI) on the offline dataset. At each of the 500 outer iterations, 50 gradient steps are taken on mini-batches of size 128 sampled uniformly from the dataset.

6.5.6.0.1 Bellman backup.

For each sampled transition the multi-objective Bellman target is:

$$\mathbf{y} = \mathbf{r} + \gamma Q_{\bar{\theta}}(\mathbf{s}', a^*), \quad a^* = \underset{a}{\operatorname{argmax}} \underbrace{\cos(\omega_{\text{perf}}, Q_{\bar{\theta}}(\mathbf{s}', a))}_{\text{alignment}} \cdot \underbrace{\omega^\top Q_{\bar{\theta}}(\mathbf{s}', a)}_{\text{magnitude}}, \quad (6.21)$$

where ω_{perf} is the preference vector projected onto the performance–power trade-off curve via a per-application polynomial model, and $\gamma = 0.9$.

6.5.6.0.2 Training objective.

The network is updated by minimising:

$$\mathcal{L}(\theta) = \underbrace{\mathcal{L}_{\text{Huber}}(Q_{\theta}(\mathbf{s}, a), \mathbf{y})}_{\text{TD loss}} + \alpha_{\text{CQL}} \underbrace{\left[\log \sum_{a'} \|Q_{\theta}(\mathbf{s}, a')\|_2 - \|Q_{\theta}(\mathbf{s}, a)\|_2 \right]}_{\text{CQL regularisation}}, \quad (6.22)$$

with $\alpha_{\text{CQL}} = 0.01$. The CQL term penalises large Q-norms for out-of-dataset actions, preventing the policy from exploiting overestimated values. The target network $Q_{\bar{\theta}}$ is updated via a soft Polyak average with $\tau = 0.005$.

6.5.6.0.3 Optimiser.

Parameters are updated with RMSprop at an initial learning rate of $\eta = 10^{-3}$, reduced by a factor of 0.5 every 5,000 gradient steps.

6.5.7 Online Evaluation

At deployment, `controller` runs the application under NRM and invokes the trained policy every 2 seconds. The current five-dimensional state is constructed from live sensor readings, the preference vector ω is recomputed from the current progress rate via Eq. (6.19), and the PCAP is set to the action selected by Eq. (6.21). The evaluation action space is extended to 16 PCAP levels by adding 78 W and 83 W to the original 14-level training space.

Each preference configuration is evaluated over 5 independent runs of `ones-stream-full`. Nine static preference values are tested, spanning the power–progress trade-off from $\omega = [0.9, 0.1]^\top$ (strongly power-saving) to $\omega = [0.1, 0.9]^\top$ (strongly progress-focused), for a total of 45 evaluation runs.

6.6 Results

We evaluate DDMORL across nine benchmarks — five STREAM memory-bandwidth variants (FULL, TRIAD, ADD, COPY, SCALE) and four NAS Parallel Benchmarks (EP, FT, IS, BT) — under nine static preference configurations $\omega = [\omega_{\text{pow}}, 1 - \omega_{\text{pow}}]^\top$ with $\omega_{\text{pow}} \in \{0.1, 0.2, \dots, 0.9\}$. Each preference was evaluated over five independent runs. Results are reported as the mean consumed energy (kJ) versus the mean performance degradation (%) relative to uncapped execution, shown in Figures ?? and ??.

6.6.1 Preference-Conditioned Operating Points

A central claim of DDMORL is that the preference vector ω acts as a user-controllable knob that steers the policy to a specific operating point on the energy–performance trade-off frontier. The results confirm this across all nine benchmarks: each value of ω_{pow} produces a *distinct* operating point, and the full set of points traces a monotone Pareto front in the energy–degradation plane.

Moving from $\omega_{\text{pow}} = 0.1$ (progress-focused, minimal power throttling) to $\omega_{\text{pow}} = 0.9$ (power-focused, aggressive throttling) consistently shifts each benchmark from the high-energy, low-degradation regime to the low-energy, high-degradation regime. Table 6.3 summarises the energy savings and the corresponding maximum performance degradation at the two extremes of the preference range.

6.6.2 The Decay Percentage as a User-Controllable Bound

The x-axis in each figure represents the *performance degradation*, which we term the *decay percentage*: it quantifies how much throughput the application sacrifices relative to uncapped execution. In the DDMORL framework this is not a hard constraint but an emergent property of the preference weight ω_{pow} : a higher power weight leads the policy to select lower PCAP values, which throttles the processor and increases execution time, thereby raising the decay percentage.

The results demonstrate that ω_{pow} provides a reliable proxy for the maximum tolerable decay:

- **Low decay regime** ($\omega_{\text{pow}} \leq 0.2$): all benchmarks remain within $\leq 10\%$ performance degradation while still reducing energy consumption relative to uncapped execution. Stream-Full at $\omega_{\text{pow}} = 0.1$ achieves only 3% degradation with 26.7 kJ consumed.

Table 6.3: Energy savings and performance degradation at the preference extremes ($\omega_{\text{pow}} = 0.1$ vs. $\omega_{\text{pow}} = 0.9$).

Benchmark	$\omega_{\text{pow}} = 0.1$		$\omega_{\text{pow}} = 0.9$	
	Deg. [%]	Energy [kJ]	Deg. [%]	Energy [kJ]
Stream-Full	3	26.7	95	16.2
Stream-Triad	10	6.9	88	4.6
Stream-Add	0	7.0	90	4.5
Stream-Copy	5	5.0	88	3.5
Stream-Scale	8	5.1	85	3.6
NPB-IS	5	19.2	63	13.8
NPB-EP	5	9.2	87	7.1
NPB-BT	5	23.5	70	18.5
NPB-FT	10	4.1	97	3.3

- **Intermediate regime** ($\omega_{\text{pow}} \in \{0.3, 0.4, 0.5\}$): the policy transitions through the knee of the Pareto front. Stream-Full moves from 10% degradation at $\omega = 0.3$ to 47% at $\omega = 0.5$, with energy dropping from 23.2 kJ to 17.8 kJ — a 23% reduction for a 37 pp increase in degradation tolerance.
- **High decay regime** ($\omega_{\text{pow}} \geq 0.6$): the policy saturates near maximum throttling. Operating points for $\omega_{\text{pow}} \in \{0.6, 0.7, 0.8, 0.9\}$ cluster tightly, indicating diminishing returns beyond $\omega_{\text{pow}} = 0.6$.

6.6.3 Application-Specific Observations

6.6.3.0.1 STREAM benchmarks.

The STREAM variants exhibit the cleanest Pareto fronts, with well-separated operating points across the full preference range. Stream-Full achieves the largest absolute energy savings: 10.5 kJ (39%) moving from $\omega_{\text{pow}} = 0.1$ to $\omega_{\text{pow}} = 0.9$. Stream-Triad and Stream-Add follow with savings of 37% and 35% respectively, while Stream-Copy and Stream-Scale yield 31% savings. The memory-bandwidth-bound nature of STREAM makes it particularly sensitive to PCAP throttling: lower power caps reduce memory controller frequency, significantly impacting throughput.

6.6.3.0.2 NPB benchmarks.

The NAS Parallel Benchmarks exhibit moderate but consistent energy savings of 20–31%, with NPB-IS achieving the highest savings at 31% and NPB-FT the lowest at 20%. A notable characteristic of the NPB benchmarks is the tendency for operating points with $\omega_{\text{pow}} \geq 0.5$ to cluster at high degradation values ($> 60\%$), with limited further energy reduction. This suggests that these compute-intensive workloads reach a minimum sustainable power floor beyond which additional throttling primarily extends execution time without reducing total energy consumption. NPB-EP and NPB-BT achieve savings of 23% each, while their operating points for $\omega_{\text{pow}} \geq 0.6$ converge to near-identical energy values, confirming the saturation effect.

6.6.4 Generalisation to Unseen Workloads

The policy was trained exclusively on five STREAM benchmark variants. The fact that it produces coherent and well-structured Pareto fronts on the four NPB benchmarks — which differ substantially in their computational patterns from STREAM — demonstrates that the PAPI-derived state features (IPC, L3 miss rate, stall ratio) carry sufficient information for the policy to generalise its power-management strategy to unseen workload classes. Across all nine benchmarks, the preference vector faithfully controls the energy–performance operating point, confirming that DDMORL learns a preference-conditioned policy rather than a workload-specific one.

6.7 Related Work

Energy–performance optimization in HPC systems has been studied extensively across scheduling theory, dynamic voltage and frequency scaling (DVFS), power capping, and learning-based control. Prior work can broadly be categorized into offline scheduling and speed-scaling theory, control-theoretic and model-based runtime power management, learning-based power control, and application-aware power modeling and tuning. Our work differs from these approaches by introducing a preference-driven, job-level controller that operates independently of the scheduler and avoids retraining for different energy–performance trade-offs.

Classical work on energy-aware scheduling formulates the problem as executing a set of jobs on a variable-speed processor to minimize energy consumption. In the strict-deadline setting, each job is defined by a release time r , deadline d , and workload w , representing the required number of CPU cycles. The optimal offline solution is given by the Yao–Demers–Shenker (YDS) algorithm, which identifies intervals of maximum workload density and assigns constant execution speeds within those intervals Bansal et al. (2011). Earliest Deadline First (EDF) is commonly used as an online heuristic in this setting Bansal et al. (2011).

For scenarios without explicit deadlines, prior work considers minimizing a combination of energy and flow time, leading to different optimality criteria and competitive guarantees Albers et al. (2015). Online algorithms are often evaluated using competitive analysis, where an algorithm is said to be c -competitive if its objective value is within a factor c of the optimal offline solution for the same input Bansal et al. (2011); Albers et al. (2015). While these formulations provide strong theoretical insights, they assume precise knowledge of job characteristics and speed–power relationships, and they do not directly translate to modern HPC systems where applications exhibit complex, hardware-dependent behavior.

DVFS has long been a primary mechanism for reducing energy consumption while meeting performance requirements. Control-theoretic approaches model the relationship between frequency, performance, and power, and use feedback control to regulate execution speed Poellabauer et al. (2005); Chen et al. (2014). Several works demonstrate that DVFS-based controllers can effectively meet timing constraints with reduced energy, particularly for compute-bound workloads Poellabauer et al. (2005).

However, recent processor designs increasingly move DVFS control into hardware, limiting the ability of software to directly set operating frequencies. This trend has motivated the use of hardware power capping interfaces such as Intel RAPL, where software specifies a power limit and the hardware manages voltage and frequency internally Imes et al. (2019). Control-based approaches using power caps have been proposed to meet soft real-time performance targets, but they must deal with non-linear and diminishing returns in the power–performance relationship Imes et al. (2019). These methods typically rely on adaptive control mechanisms and runtime feedback, and are often designed for single objectives or fixed performance targets.

Machine learning and reinforcement learning (RL) have been explored to address the limitations of fixed models and hand-tuned controllers. Early work applies Q-learning and Double-Q learning to DVFS governors, enabling systems to adapt frequency decisions based on observed workload behavior Huang et al. (2020b). These approaches demonstrate improved energy savings compared to static governors, but they require online exploration and are sensitive to training stability and convergence time.

More recent efforts employ model-based or multi-objective learning techniques to predict performance and energy under different power cap configurations Hao et al. (2021). These methods often construct performance and energy models using micro-benchmarks or application features and then identify Pareto-optimal configurations. While effective, such approaches typically require extensive profiling, retraining for new architectures or applications, and separate optimization runs for different objectives or user preferences.

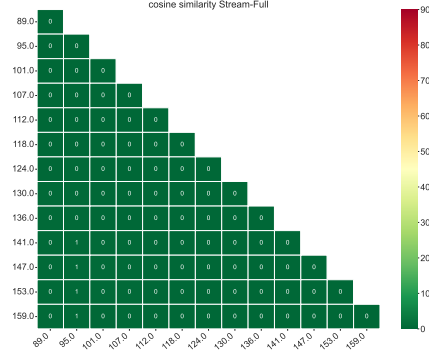
Offline RL has been proposed as a way to avoid unsafe or costly online exploration by training policies from pre-collected data Levine et al. (2020b). However, most existing RL-based approaches still produce a single policy optimized for a fixed objective, requiring multiple agents or retraining when the desired trade-off changes.

To make power management practical, many studies rely on HPC proxy applications and benchmark suites to capture representative computation and communication patterns Hao et al. (2021); Chen et al. (2025). Proxy applications enable controlled experiments to study how performance degrades under power caps and how energy efficiency varies across workloads. Empirical studies show that different applications respond very differently to power limiting, highlighting the need for application-aware control Hao et al. (2021).

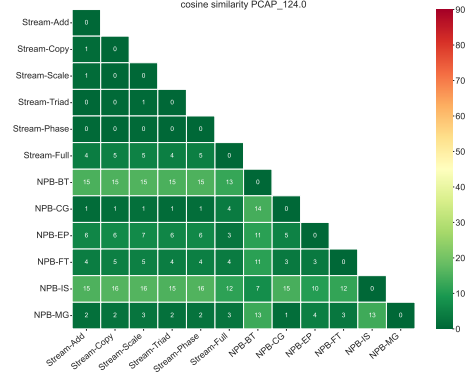
Several works combine proxy-based profiling with analytical or data-driven models to guide power allocation decisions at the node or cluster level Hao et al. (2021); Morari et al. (2011). While these approaches provide valuable insights into workload behavior, they often assume static optimization goals and do not expose an intuitive mechanism for users to express acceptable performance degradation.

In contrast to prior work, our approach focuses on job-level runtime control rather than scheduler-level resource allocation or cluster-level power distribution. We propose a preference-driven multi-objective reinforcement learning framework that allows users to specify their desired trade-off between performance and energy using a simple preference parameter. By leveraging mathematical models of application behavior to generate training data, we eliminate the need for interpolation models and enable efficient offline training.

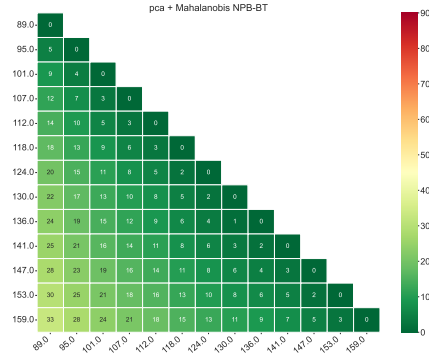
Unlike existing learning-based approaches, our method produces a single controller that can operate across a range of user-defined performance degradation levels, avoiding the need to train separate agents for different objectives. This design complements existing schedulers by providing fine-grained, application-aware power control during job execution, while preserving scheduler autonomy over resource allocation and job ordering.



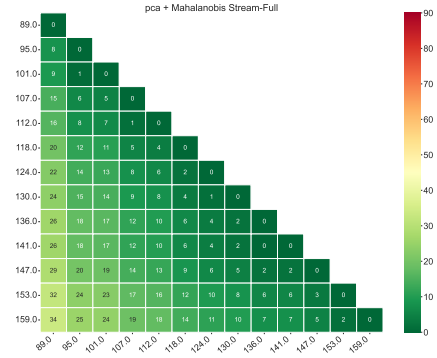
(a) Cosine similarity of Stream-Full across all 13 power cap levels (89W to 159W). The near-zero angles ($\leq 1^\circ$) across all PCAP pairs confirm that the application's behavioral fingerprint in hardware counter space is invariant to power capping. This supports using any power cap level for training without introducing distributional shift.



(b) Cosine similarity between applications at a power cap of 124W, expressed as the angle (in degrees) between feature vectors comprising IPC, L3 miss rate, stall fraction, and normalized progress rate. Values near 0° indicate near-identical hardware behavior. Stream variants (Add, Copy, Scale, Triad, Phase) form a tight cluster (0 to 1°), while NPB-BT and NPB-IS remain the most behaviorally distinct applications (13 to 16°).



(c) PCA-reduced Mahalanobis distance between power cap conditions for NPB-BT. Unlike cosine similarity, which captures only the direction of the feature vector, Mahalanobis distance in PCA space captures magnitude shifts, particularly in progress rate. The smooth monotonic increase, reaching a maximum distance of 33 between 89W and 159W, shows that power capping continuously impacts application throughput.



(d) PCA-reduced Mahalanobis distance between power cap levels for Stream-Full, showing non-uniform sensitivity. Behavior remains nearly invariant across mid to high power caps (107W to 159W, distance ≤ 10) but diverges at the lowest cap (89W versus 159W reaching a distance of 34). This indicates a saturation region beyond which increasing power does not significantly alter the application's behavior.

Figure 6.1: Application diversity and power-cap sensitivity across HPC workloads. Cosine similarity highlights invariant behavioral structure across power caps, while PCA-based Mahalanobis distance reveals significant variation in application progress under different power constraints.

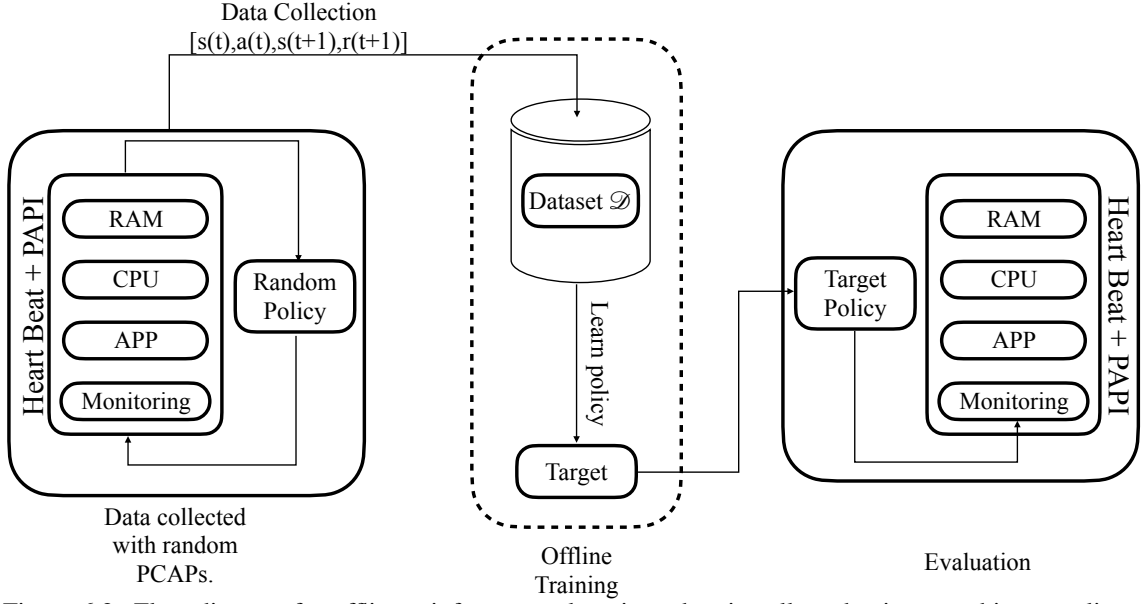


Figure 6.2: Flow diagram for offline reinforcement learning: data is collected using an arbitrary policy to train the agent without system access, and the trained agent is later deployed for evaluation. Note that the lines in the diagram are not closed to form a loop, indicating offline training and online testing.

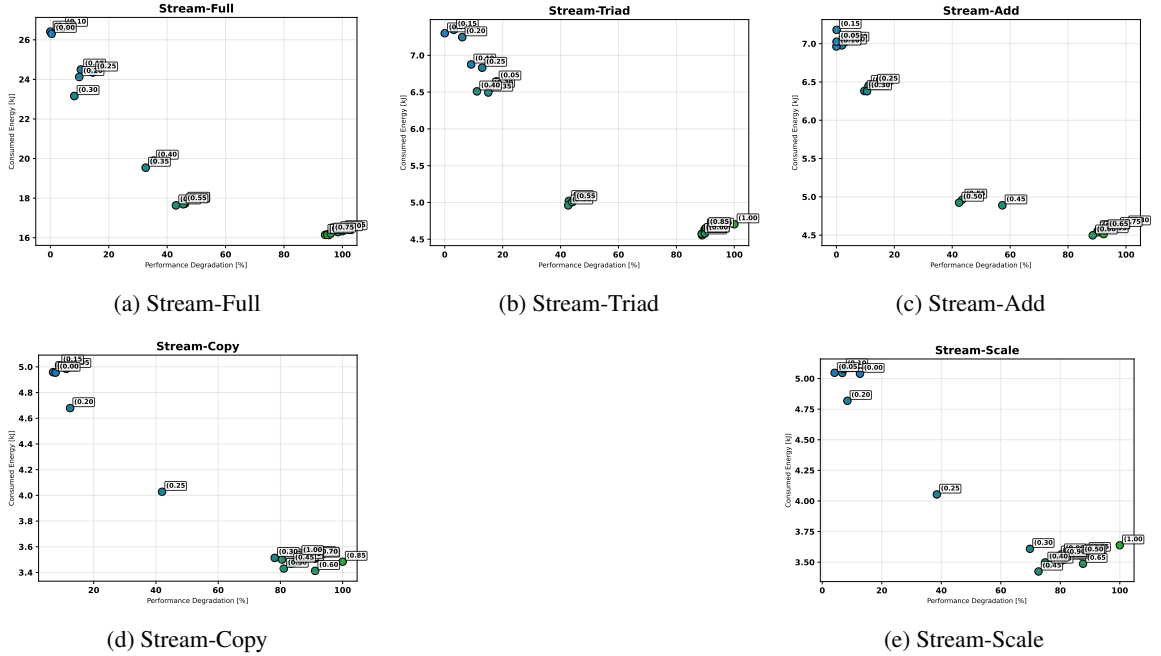


Figure 6.3: Performance degradation (%) vs. consumed energy (kJ) for the five STREAM memory-bandwidth benchmarks under DDMORL. Each point corresponds to a fixed static preference $\omega = [\omega_{\text{pow}}, 1 - \omega_{\text{pow}}]^\top$ (label shows ω_{pow}), averaged over five independent runs. Moving from $\omega_{\text{pow}} = 0.1$ (upper-left, progress-focused) to $\omega_{\text{pow}} = 0.9$ (lower-right, power-focused) traces the Pareto front, reducing energy by 31–39% at the cost of increased performance degradation. The decay percentage on the x-axis represents the maximum performance degradation tolerated under each preference setting.

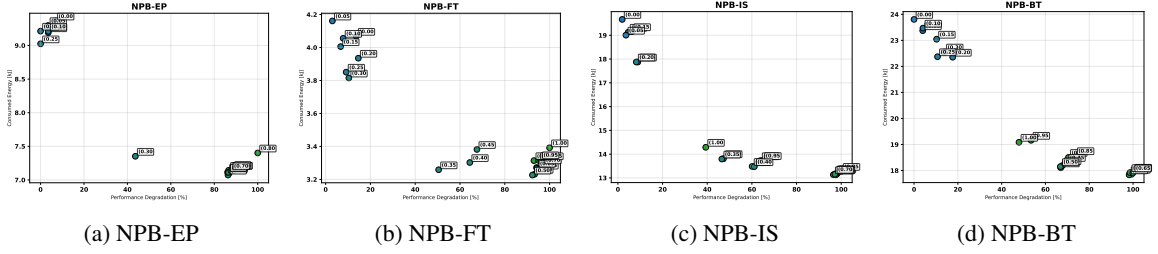


Figure 6.4: Performance degradation (%) vs. consumed energy (kJ) for four NAS Parallel Benchmarks ? under DDMORL. Despite the training dataset containing only STREAM variants, the policy generalises to these compute- and communication-intensive workloads, producing coherent Pareto fronts with energy savings of 20–31%. NPB-IS achieves the largest savings (31%) while NPB-FT saturates earliest, with operating points for $\omega_{\text{pow}} \geq 0.5$ clustering near 97% degradation.

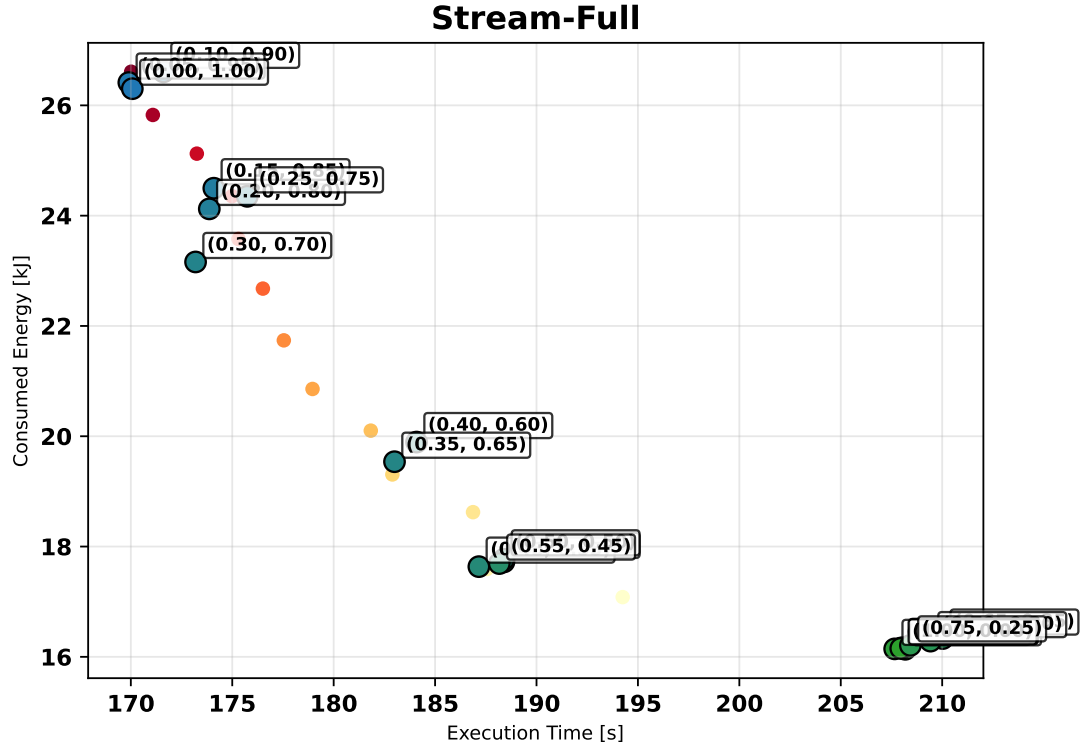


Figure 6.5: Execution time (s) vs. consumed energy (kJ) for STREAM-FULL ?. Filled circles show DDMORL operating points under nine preference configurations; marker colour encodes the mean PCAP level (W) selected by the policy (colour bar, right). Open circles show static fixed-PCAP baselines ? at each of the 16 discrete power cap levels. The RL policy consistently selects operating points on or near the lower-left convex hull of the static baseline scatter, confirming that the preference-conditioned policy learns to approximate the Pareto front rather than an arbitrary fixed cap.

References

- Acar, U. A., Charguéraud, A., Guatto, A., Rainey, M., and Sieczkowski, F. (2018). Heartbeat scheduling: Provable efficiency for nested parallelism. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 769–782.
- Adebayo, H. (2026). Human-in-the-loop explainable ai for reliable autonomous cybersecurity infrastructure.
- Advanced Photon Source (2021). APS scientific computing strategy.
- Albers, S., Antoniadis, A., and Greiner, G. (2015). On multi-processor speed scaling with migration. *Journal of Computer and System Sciences*, 81(7):1194–1209.
- Babu, A. V., Zhou, T., Kandel, S., Bicer, T., Liu, Z., Judge, W., Ching, D. J., Jiang, Y., Veseli, S., Henke, S., et al. (2022). Deep learning at the edge enables real-time streaming ptychographic imaging. *arXiv preprint arXiv:2209.09408*.
- Badia, R. M., Berti-Equille, L., da Silva, R. F., and Leser, U. (2024). Integrating HPC, AI, and workflows for scientific data analysis. *Dagstuhl Reports*, 13(8):129–164.
- Bansal, N., Bunde, D. P., Chan, H.-L., and Pruhs, K. (2011). Average rate speed scaling. *Algorithmica*, 60(4):877–889.
- Basaklar, T., Gumussoy, S., and Ogras, U. Y. (2023). Pd-morl: Preference-driven multi-objective reinforcement learning algorithm.
- Bastani, O., Pu, Y., and Solar-Lezama, A. (2018a). Verifiable reinforcement learning via policy extraction. *Advances in neural information processing systems*, 31.
- Bastani, O., Pu, Y., and Solar-Lezama, A. (2018b). Verifiable reinforcement learning via policy extraction. In *Advances in Neural Information Processing Systems*, volume 31.
- Beckman, P., Sankaran, R., Catlett, C., Ferrier, N., Jacob, R., and Papka, M. (2016). Waggle: An open sensor platform for edge computing. In *2016 IEEE SENSORS*, pages 1–3. IEEE.
- Caglar, F., Shekhar, S., and Gokhale, A. (2014). iplace: An intelligent and tunable power-and performance-aware virtual machine placement technique for cloud-based real-time applications. In *2014 IEEE 17th International Symposium on Object/Component/Service-Oriented Real-Time Distributed Computing*, pages 48–55. IEEE.
- Cerf, S., Bleuse, R., Reis, V., Perarnau, S., and Rutten, E. (2021a). Sustaining performance while reducing energy consumption: A control theory approach. In *International European Conference on Parallel and Distributed Computing (EUROPAR)*.
- Cerf, S., Bleuse, R., Reis, V., Perarnau, S., and Rutten, E. (2021b). Sustaining performance while reducing energy consumption: a control theory approach. In *European Conference on Parallel Processing*, pages 334–349. Springer.
- Chen, G., Huang, K., and Knoll, A. (2014). Energy optimization for real-time multiprocessor system-on-chip with optimal dvfs and dpm combination. *ACM Transactions on Embedded Computing Systems (TECS)*, 13(3s):1–21.
- Chen, S., Garcia de Gonzalo, S., Aaziz, O., Cook, J., and Wildani, A. (2025). Beyond guess and check: Quantifying the fidelity of proxy applications. In *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC Workshops '25*, page 1464–1477, New York, NY, USA. Association for Computing Machinery.
- Chen, Z. and Marculescu, D. (2015). Distributed reinforcement learning for power limited many-core system performance optimization. In *2015 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1521–1526. IEEE.
- Cherukara, M. J. et al. (2020). AI-enabled high-resolution scanning coherent diffraction imaging. *Applied Physics Letters*, 117(4).
- Corbalan, J., Alonso, L., Aneas, J., and Brochard, L. (2020). Energy optimization and analysis with ear.
- Czarnul, P., Proficjz, J., Krzywaniak, A., et al. (2019). Energy-aware high-performance computing: survey of state-of-the-art tools, techniques, and environments. *Scientific Programming*, pages 1–8.
- Darrow, K. and Hedman, B. A. (2009). *Opportunities for combined heat and power in data centers*. ICF International Arlington, VA.
- David, H., Gorbato, E., Hanebutte, U. R., Khanna, R., and Le, C. (2010). Rapl: Memory power estimation and capping. In *2010 ACM/IEEE International Symposium on Low-Power Electronics and Design (ISLPED)*, pages 189–194. IEEE.
- Dayarathna, M., Wen, Y., and Fan, R. (2015). Data center energy consumption modeling: A survey. *IEEE Communications surveys & tutorials*, 18(1):732–794.
- Dolstra, E. and Löh, A. (2008). Nixos: A purely functional linux distribution. In *Proceedings of the 13th ACM SIGPLAN international conference on Functional programming*, pages 367–378.
- Dube, N., Roweth, D., Faraboschi, P., and Milojicic, D. (2021). Future of HPC: The Internet of workflows. *IEEE Internet Computing*, 25(5):26–34.
- Eastep, J., Sylvester, S., Cantalupo, C., Geltz, B., Ardanaz, F., Al-Rawi, A., Livingston, K., Keceli, F., Maiterth, M., and Jana, S. (2017a). Global extensible open power manager: A vehicle for HPC community collaboration on co-designed energy management solutions. In *High Performance Computing: 32nd International Conference, ISC High Performance 2017, Frankfurt, Germany, June 18–22, 2017, Proceedings*, pages 394–412, Berlin, Heidelberg. Springer-Verlag.
- Eastep, J., Sylvester, S., Cantalupo, C., Geltz, B., Ardanaz, F., Al-Rawi, A., Livingston, K., Keceli, F., Maiterth, M., and Jana, S. (2017b). Global extensible open power manager: A vehicle for HPC community collaboration on co-designed energy management solutions. In *International Conference on High Performance Computing (ISC)*. Springer.

- Emerson, H. et al. (2024). Cyborg++: An enhanced gym for the development of autonomous cyber agents. *arXiv preprint arXiv:2410.16324*.
- Evertsz, R. and Thangarajah, J. (2020). A framework for engineering human/agent teaming systems. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 2477–2484.
- Grant, R. E., Laros, J. H., Levenhagen, M., Olivier, S. L., Pedretti, K., Ward, L., and Younge, A. J. (2017). Evaluating energy and power profiling techniques for HPC workloads. In *2017 Eighth International Green and Sustainable Computing Conference (IGSC)*, pages 1–8.
- Hao, M., Zhang, W., Wang, Y., Lu, G., Wang, F., and Vasilakos, A. V. (2021). Fine-grained powercap allocation for power-constrained systems based on multi-objective machine learning. *IEEE Transactions on Parallel and Distributed Systems*, 32(7):1789–1801.
- Hawila, I., Cerf, S., Bleuse, R., Perarnau, S., and Rutten, E. (2022a). Adaptive power control for sober high-performance computing. In *IEEE Conference on Control Technology and Applications (CCTA)*.
- Hawila, I., Cerf, S., Bleuse, R., Perarnau, S., and Rutten, E. (2022b). Adaptive Power Control for Sober High-Performance Computing. In *CCTA 2022 - 6th IEEE Conference on Control Technology and Applications*, pages 1–8, Trieste, Italy. IEEE.
- Hsu, C.-h. and Feng, W.-c. (2005). A power-aware run-time system for high-performance computing. In *SC'05: Proceedings of the 2005 ACM/IEEE Conference on Supercomputing*, pages 1–1. IEEE.
- Huang, H., Lin, M., Yang, L. T., and Zhang, Q. (2020a). Autonomous power management with double-Q reinforcement learning method. *IEEE Transactions on Industrial Informatics*, 16(3):1938–1946.
- Huang, H., Lin, M., Yang, L. T., and Zhang, Q. (2020b). Autonomous power management with double-q reinforcement learning method. *IEEE Transactions on Industrial Informatics*, 16(3):1938–1946.
- Imes, C., Zhang, H., Zhao, K., and Hoffmann, H. (2019). Copper: Soft real-time application performance using hardware power capping. In *2019 IEEE International Conference on Autonomic Computing (ICAC)*, pages 31–41, New York, NY, USA. Association for Computing Machinery.
- Ju, L., Singh, P., and Toor, S. (2021). Proactive autoscaling for edge computing systems with Kubernetes. In *Proceedings of the 14th IEEE/ACM International Conference on Utility and Cloud Computing Companion*, pages 1–8.
- Jung, H. and Pedram, M. (2010). Supervised learning based power management for multicore processors. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 29(9):1395–1408.
- Kalimuthu, N. (2025). Explainable ai (xai) for enhanced cyber threat intelligence: Building interpretable intrusion detection systems. *IJSAT-International Journal on Science and Technology*, 16(4).
- Kandel, S., Zhou, T., Babu, A. V., Di, Z., Li, X., Ma, X., Holt, M., Miceli, A., Phatak, C., and Cherukara, M. J. (2023). Demonstration of an AI-driven workflow for autonomous high-resolution scanning microscopy. *Nature Communications*, 14(1):5501.
- Keahey, K., Anderson, J., Zhen, Z., Riteau, P., Ruth, P., Stanzione, D., Cevik, M., Colleran, J., Gunawi, H. S., Hammock, C., Mambretti, J., Barnes, A., Halbach, F., Rocha, A., and Stubbs, J. (2020a). Lessons learned from the Chameleon testbed. In *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC '20)*. USENIX Association.
- Keahey, K., Anderson, J., Zhen, Z., Riteau, P., Ruth, P., Stanzione, D., Cevik, M., Colleran, J., Gunawi, H. S., Hammock, C., Mambretti, J., Barnes, A., Halbach, F., Rocha, A., and Stubbs, J. (2020b). Lessons learned from the chameleon testbed. In *Proceedings of the 2020 USENIX Annual Technical Conference (USENIX ATC '20)*. USENIX Association.
- Keahey, K., Riteau, P., Stanzione, D., Cockerill, T., Mambretti, J., Rad, P., and Ruth, P. (2019). Chameleon: a scalable production testbed for computer science research. In *Contemporary High Performance Computing*, pages 123–148. CRC Press.
- Kiely, M., Bowman, D., Standen, M., and Moir, C. (2023). On autonomous agents in a cyber defence environment. *arXiv preprint arXiv:2309.07388*.
- Kim, W., Gupta, M. S., Wei, G.-Y., and Brooks, D. (2008). System level analysis of fast, per-core DVFS using on-chip switching regulators. In *2008 IEEE 14th International Symposium on High Performance Computer Architecture*, pages 123–134.
- Kim, Y., Park, S., Shahkarami, S., Sankaran, R., Ferrier, N., and Beckman, P. (2022). Goal-driven scheduling model in edge computing for smart city applications. *Journal of Parallel and Distributed Computing*, 167.
- Koomey, J. et al. (2011). Growth in data center electricity use 2005 to 2010. *A report by Analytical Press, completed at the request of The New York Times*, 9(2011):161.
- Koot, M. and Wijnhoven, F. (2021). Usage impact on data center electricity needs: A system dynamic forecasting model. *Applied Energy*, 291:116798.
- Krzywaniak, A., Czarnul, P., and Proficz, J. (2023). Dynamic GPU power capping with online performance tracing for energy efficient GPU computing using DEPO tool. *Future Generation Computer Systems*.
- Lazer, S. J., Aryal, K., Gupta, M., and Bertino, E. (2026). A survey of agentic ai and cybersecurity: Challenges, opportunities and use-case prototypes.
- Levine, S., Kumar, A., Tucker, G., and Fu, J. (2020a). Offline reinforcement learning: Tutorial, review, and perspectives on open problems.
- Levine, S., Kumar, A., Tucker, G., and Fu, J. (2020b). Offline reinforcement learning: Tutorial, review, and perspectives on open problems.
- Li, B., Gadepally, V., Samsi, S., and Tiwari, D. (2022). Characterizing multi-instance GPU for machine learning workloads. In *2022 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, pages 724–731. IEEE.
- Li, X., Chen, L., Chen, S., Jiang, F., Li, C., Zhang, W., and Xu, J. (2024). Deep reinforcement learning-based power management for chiplet-based multicore systems. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 32(9):1726–1739.
- Liu, Z. et al. (2022). BraggNN: Fast X-ray Bragg peak analysis using deep learning. *IUCrJ*, 9(1).

- Lundberg, S. M. and Lee, S. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30.
- McCalpin, J. D. (1995). Memory bandwidth and machine balance in current high performance computers. *IEEE Computer Society Technical Committee on Computer Architecture (TCCA) Newsletter*.
- McCalpin, J. D. (1997). A survey of memory bandwidth and machine balance in current high performance computers. *Newsletter of the IEEE Technical Committee on Computer Architecture (TCCA)(December 1995)*.
- Morari, A., Gioiosa, R., Wisniewski, R. W., Cazorla, F. J., and Valero, M. (2011). A quantitative analysis of os noise. In *2011 IEEE International Parallel & Distributed Processing Symposium*, pages 852–863, New York, NY, USA. Association for Computing Machinery.
- Mucci, P. J., Browne, S., Deane, C., and Ho, G. (1999). PAPI: a portable interface to hardware performance counters. In *Proceedings of the Department of Defense HPCMP users group conference*, volume 710.
- Neema, H., Balasubramanian, D., Vardhan, H., Nine, H., and Neema, S. (2024). RAMPART: Reinforcement Against Malicious Penetration by Adversaries in Realistic Topologies. In *2024 IEEE Workshop on Design Automation for CPS and IoT (DESTION)*, pages 33–39. IEEE Computer Society.
- Neema, H., Wang, L., Koutsoukos, X., Tang, C., and Stouffer, K. (2021). Model-based risk analysis approach for network vulnerability and security of the critical railway infrastructure. In *International Conference on Critical Information Infrastructures Security*, pages 79–98. Springer.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al. (2019). Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32.
- Patki, T., Lowenthal, D. K., Sasidharan, A., Maiterth, M., Rountree, B. L., Schulz, M., and De Supinski, B. R. (2015). Practical resource management in power-constrained, high performance computing. In *Proceedings of the 24th international symposium on high-performance parallel and distributed computing*, pages 121–132.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Matthaus, F., and Muller, E. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.
- Perarnau, S., Zounmevo, J. A., Dreher, M., Essen, B. C. V., Gioiosa, R., Iskra, K., Gokhale, M. B., Yoshii, K., and Beckman, P. (2017). Argo NodeOS: Toward unified resource management for exascale. In *International Parallel and Distributed Processing Symposium (IPDPS)*, pages 153–162.
- Pete, B., Nicolas, D., Rinku, G., Kamil, I., Swann, P., John, T., Brice, V., and Kazutomo, Y. (2021). Argo Node Resource Manager.
- Petoumenos, P., Mukhanov, L., Wang, Z., Leather, H., and Nikolopoulos, D. S. (2015). Power capping: What works, what does not. In *2015 IEEE 21st International Conference on Parallel and Distributed Systems (ICPADS)*, pages 525–534. IEEE.
- Poellabauer, C., Singleton, L., and Schwan, K. (2005). Feedback-based dynamic voltage and frequency scaling for memory-bound real-time applications. In *Proceedings of the 11th IEEE Real Time on Embedded Technology and Applications Symposium, RTAS '05*, page 234–243, USA. IEEE Computer Society.
- Qiu, M., Ming, Z., Li, J., Liu, S., Wang, B., and Lu, Z. (2012). Three-phase time-aware energy minimization with dvfs and unrolling for chip multiprocessors. *Journal of Systems Architecture*, 58(10):439–445.
- Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., and Dormann, N. (2021). Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*.
- Raghavendra, R., Ranganathan, P., Talwar, V., Wang, Z., and Zhu, X. (2008). No” power” struggles: coordinated multi-level power management for the data center. In *Proceedings of the 13th international conference on Architectural support for programming languages and operating systems*, pages 48–59.
- Raj, A., Perarnau, S., and Gokhale, A. (2023a). A reinforcement learning approach for performance-aware reduction in power consumption of data center compute nodes. In *IEEE Conference on Cloud Engineering (IC2E)*.
- Raj, A., Perarnau, S., and Gokhale, A. (2023b). A reinforcement learning approach for performance-aware reduction in power consumption of data center compute nodes. In *IEEE International Conference on Cloud Engineering (IC2E)*.
- Ramesh, S., Perarnau, S., Bhalachandra, S., Malony, A. D., and Beckman, P. (2019a). Understanding the impact of dynamic power capping on application progress. In *33rd International Parallel and Distributed Processing Symposium (IPDPS 2019)*, pages 793–804. IEEE.
- Ramesh, S., Perarnau, S., Bhalachandra, S., Malony, A. D., and Beckman, P. (2019b). Understanding the impact of dynamic power capping on application progress. In *2019 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pages 793–804. IEEE.
- Rane, A. and Browne, J. (2014). Enhancing performance optimization of multicore/multichip nodes with data structure metrics. *ACM Transactions on Parallel Computing (TOPC)*.
- Roberts, S. I., Wright, S. A., Fahmy, S. A., and Jarvis, S. A. (2019). The power-optimised software envelope. *ACM Trans. Archit. Code Optim.*, 16(3).
- Rountree, B., Ahn, D. H., De Supinski, B. R., Lowenthal, D. K., and Schulz, M. (2012). Beyond dvfs: A first look at performance under a hardware-enforced power bound. In *2012 IEEE 26th International Parallel and Distributed Processing Symposium Workshops & PhD Forum*, pages 947–953. IEEE.
- Rutten, E., Cerf, S., Bleuse, R., Reis, V., and Perarnau, S. (2021). Sustaining performance while reducing energy consumption: A control theory approach. *arXiv preprint arXiv:2107.02426*.
- Sadrosadati, M., Ehsani, S. B., Falahati, H., Ausavarungrun, R., Tavakkol, A., Abaee, M., Orosa, L., Wang, Y., Sarbazi-Azad, H., and Mutlu, O. (2019). Itap: Idle-time-aware power management for gpu execution units. *ACM Transactions on Architecture and Code Optimization (TACO)*, 16(1):1–26.
- Savard, C., Manganelli, N., Holzman, B., Gray, L., Perloff, A., Pedro, K., Stenson, K., and Ulmer, K. (2024). Optimizing high-throughput inference on graph neural networks at shared computing facilities with the NVIDIA Triton Inference Server. *Computing and Software for Big Science*, 8(1):14.

- Schöne, R., Treibig, J., Dolz, M. F., Guillen, C., Navarrete, C., Knobloch, M., and Rountree, B. (2014). Tools and methods for measuring and tuning the energy efficiency of hpc systems. *Scientific programming*, 22(4):273–283.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- Shalf, J. (2020). The future of computing beyond moore’s law. *Philosophical Transactions of the Royal Society A*, 378(2166):20190061.
- Shehabi, A., Smith, S., Sartor, D., Brown, R., Herrlin, M., Koomey, J., Masanet, E., Horner, N., Azevedo, I., and Lintner, W. (2016). United states data center energy usage report. 2016. *DOI*, 10:1372902.
- Simmendinger, C., Marquardt, M., Mäder, J., and Schneider, R. (2024). Powersched - managing power consumption in overprovisioned systems. In *2024 IEEE International Conference on Cluster Computing Workshops (CLUSTER Workshops)*, pages 1–8.
- Sokol, K. and Flach, P. (2019). Desiderata for interpretability: Explaining decision tree predictions with counterfactuals. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33.
- Stoffel, M. and Mazouz, A. (2018). Improving power efficiency through fine-grain performance monitoring in HPC clusters. In *2018 IEEE International Conference on Cluster Computing (CLUSTER)*.
- Tran, M.-N., Vu, D.-D., and Kim, Y. (2022). A survey of autoscaling in kubernetes. In *2022 Thirteenth International Conference on Ubiquitous and Future Networks (ICUFN)*, pages 263–265. IEEE.
- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., Carey, C. J., Polat, İ., Feng, Y., Moore, E. W., VanderPlas, J., Laxalde, D., Perktold, J., Cimrman, R., Henriksen, I., Quintero, E. A., Harris, C. R., Archibald, A. M., Ribeiro, A. H., Pedregosa, F., van Mulbregt, P., and SciPy 1.0 Contributors (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272.
- Wang, Y., Zhang, W., Hao, M., Kong, W., and Wen, Y. (2025). Dynamic power management through multi-agent deep reinforcement learning for heterogeneous systems. *ACM Trans. Archit. Code Optim.*, 22(2).
- Williams, G., Wagener, N., Goldfain, B., Drews, P., Rehg, J. M., Boots, B., and Theodorou, E. A. (2017). Information theoretic mpc for model-based reinforcement learning. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 1714–1721. IEEE.
- Witt, C., Bux, M., Gusew, W., and Leser, U. (2019). Predictive performance modeling for distributed batch processing using black box monitoring and machine learning. *Information Systems*, 82:33–52.
- Wittmann, M., Hager, G., Zeiser, T., Treibig, J., and Wellein, G. (2016). Chip-level and multi-node analysis of energy-optimized lattice Boltzmann CFD simulations. *Concurrency and Computation: Practice and Experience*, 28(7).
- Wysocki, R. J. (2017). The concept of cpu performance scaling. <https://www.kernel.org/doc/html/latest/admin-guide/pm/cpufreq.html>. Linux Kernel Documentation, © Intel Corporation.
- Ye, X., Iyer, S., Celikyilmaz, A., Stoyanov, V., Durrett, G., and Pasunuru, R. (2023). Complementary explanations for effective in-context learning. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 4469–4484.
- Zhang, H. and Hoffmann, H. (2016). Maximizing performance under a power cap: A comparison of hardware, software, and hybrid techniques. *ACM SIGPLAN Notices*, 51(4):545–559.